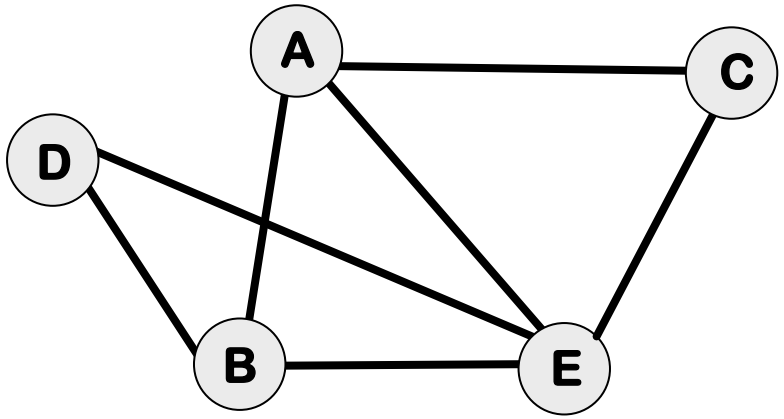
GREEDY ALGORITHMS: KRUSKAL, PRIM, DIJEKSTRA

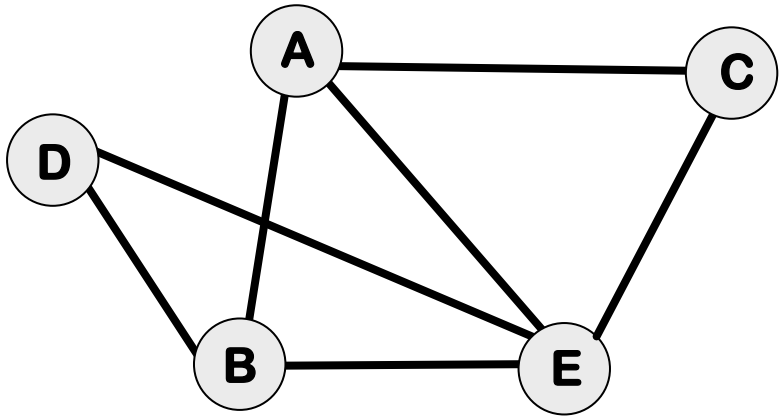
MARZIEH ESKANDARI

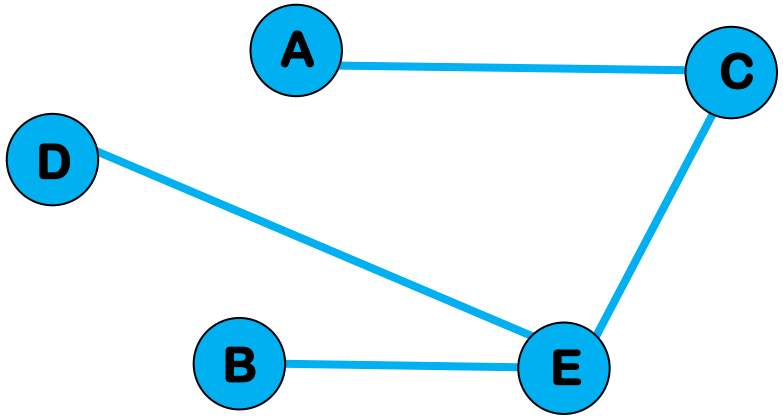
NJIT

SPANNING TREE

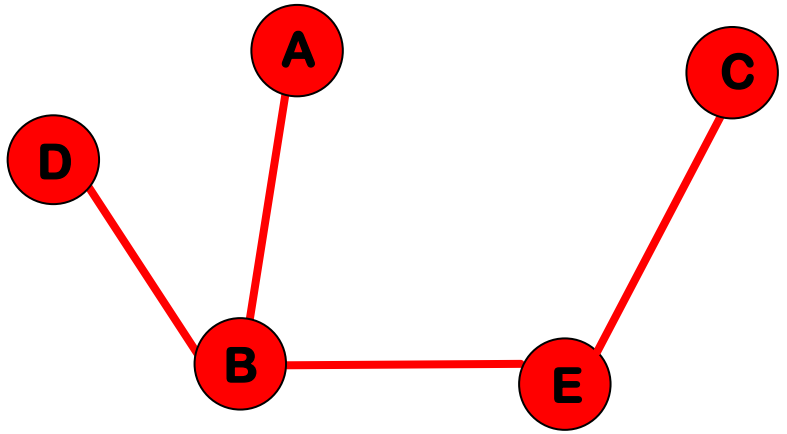


SPANNING TREE

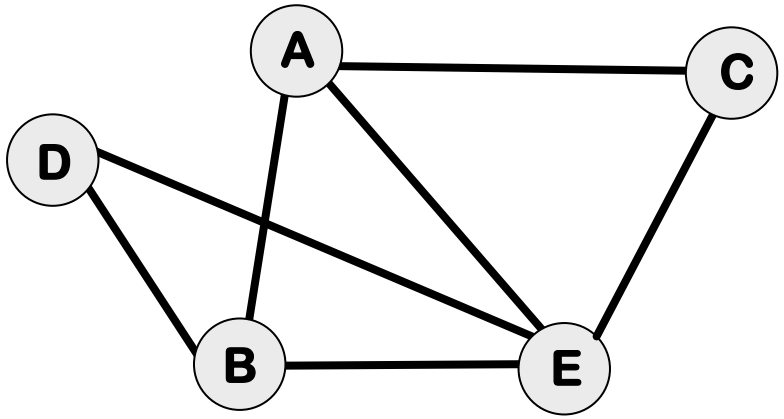


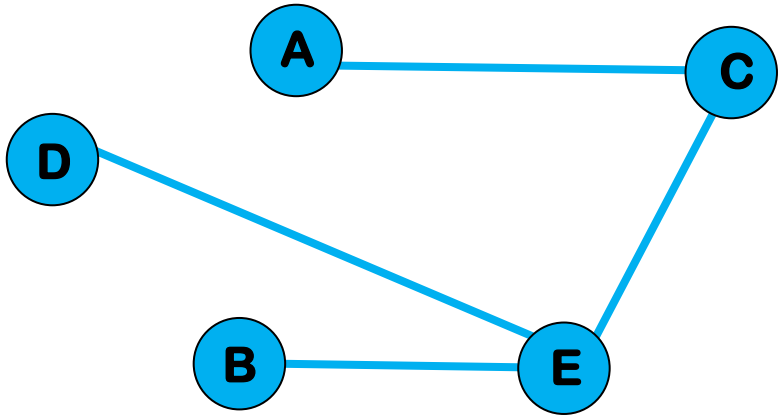


n nodes → edges

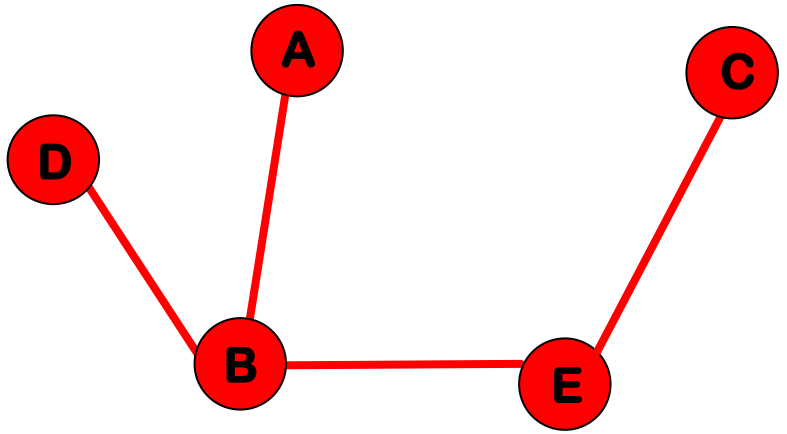


SPANNING TREE

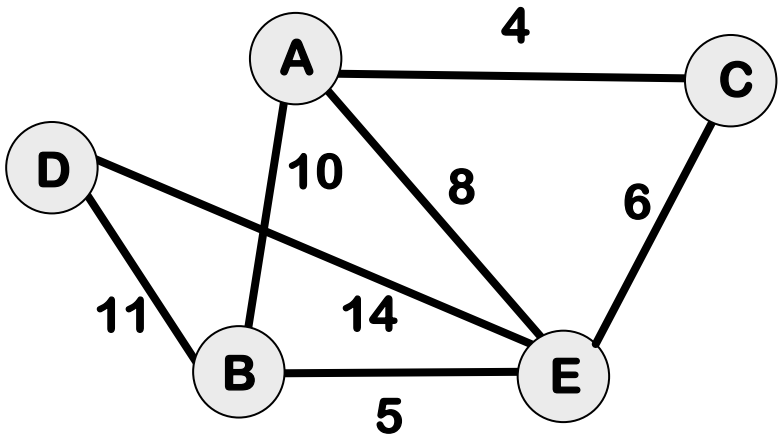




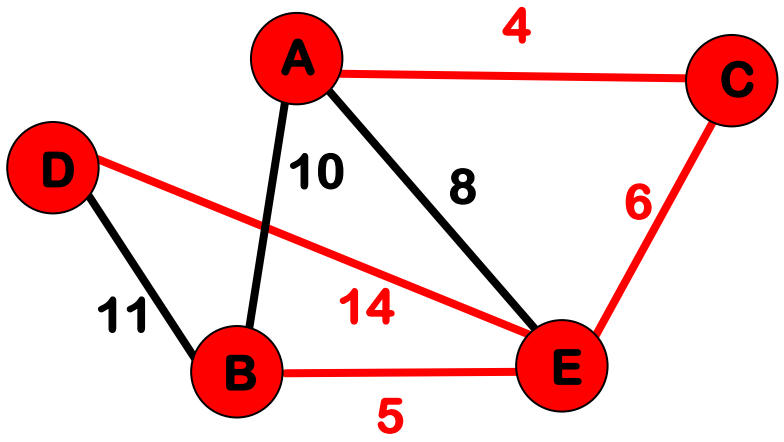
n nodes → n-1 edges



WEIGHT OF A SPANNING TREE

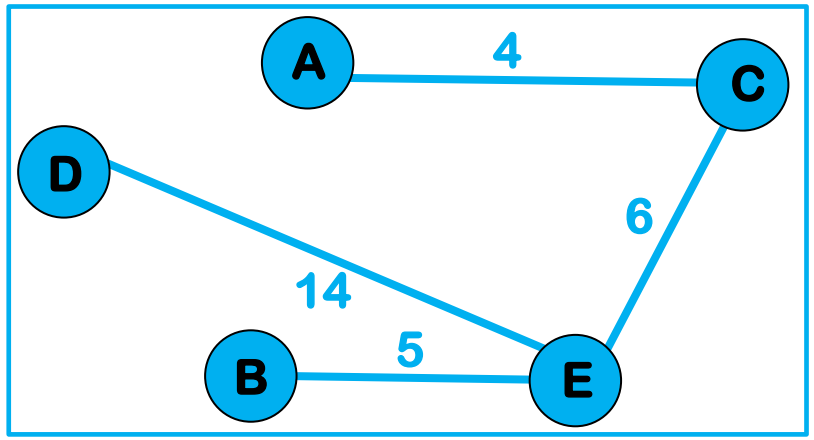


G=(V,E)

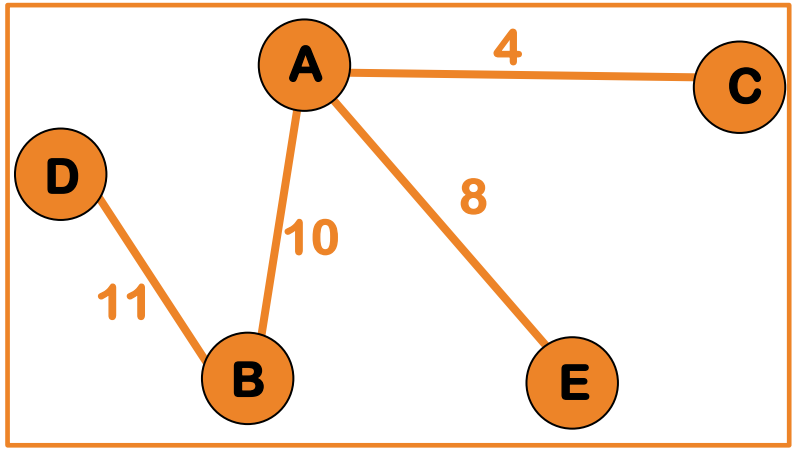


Weight=5+14+4+6=29

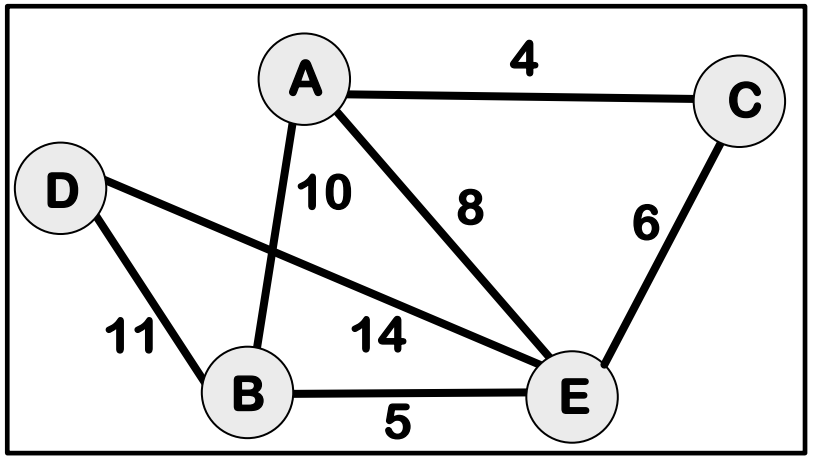
MINIMUM SPANNING TREE

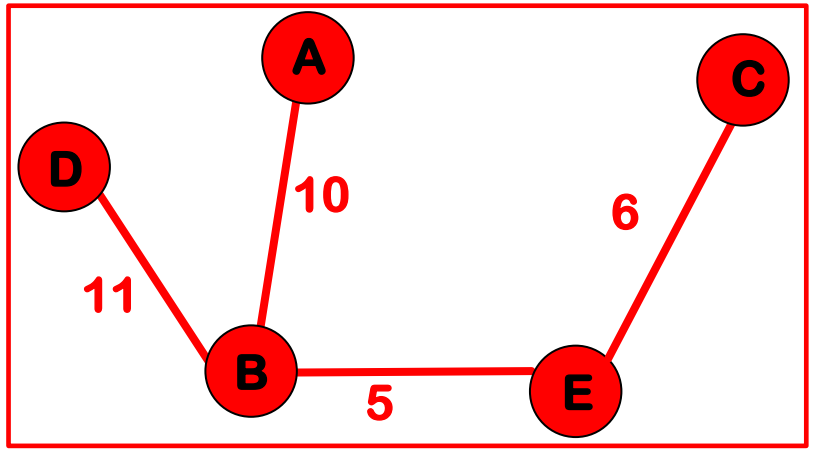


4+6+5+14=29

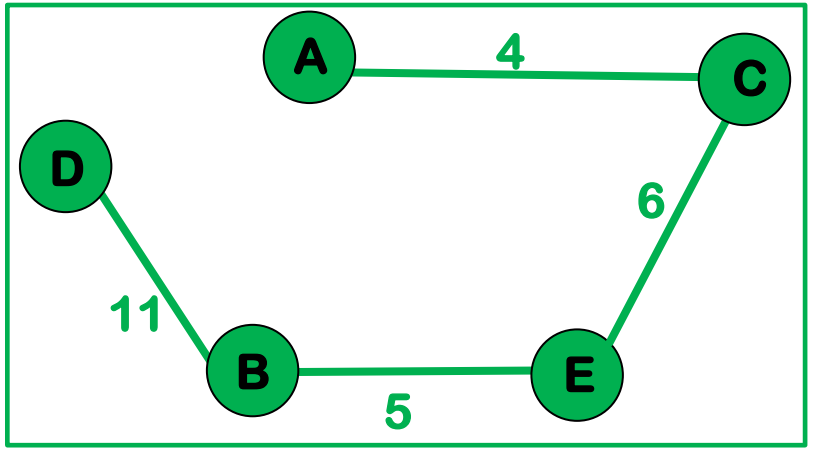


11+10+4+8=33



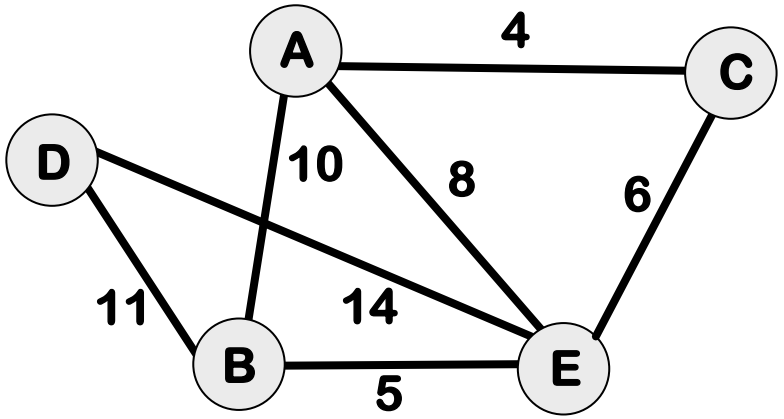


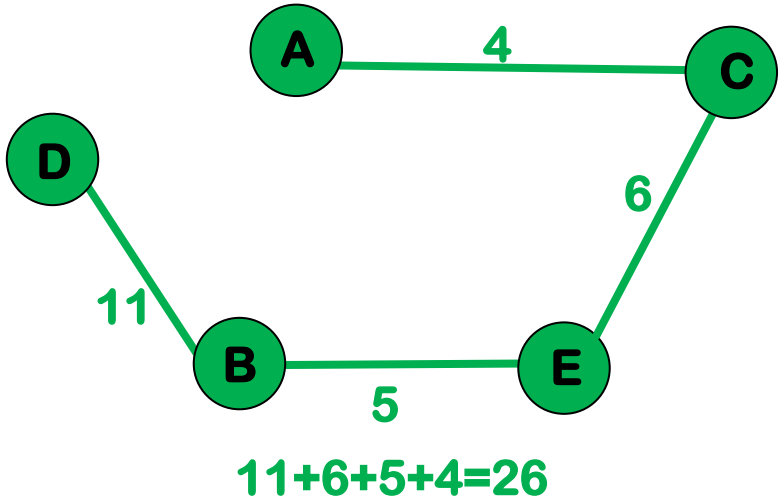
11+10+5+6=42



11+6+5+4=26

MINIMUM SPANNING TREE

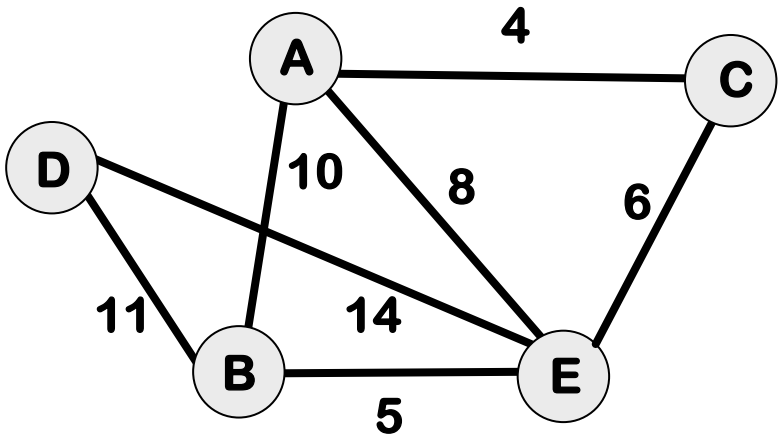




MINIMUM SPANNING TREE

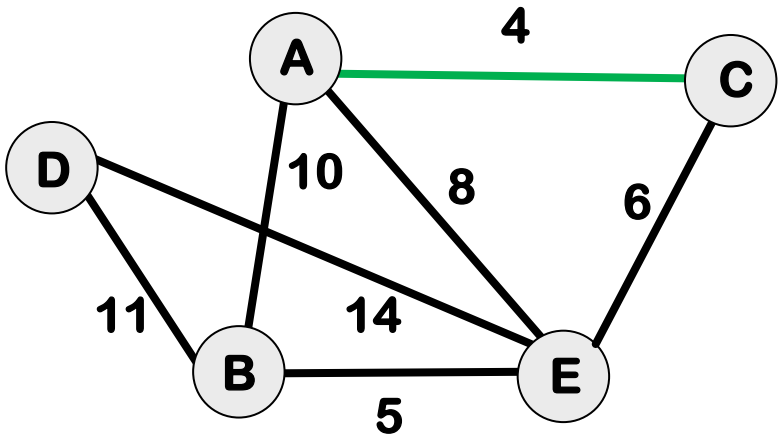
```
MST(E)
      E.sort
      F=∅
      WHILE F!=MST DO
            e=min_length_edge
            IF  F+{e} has no cycle THEN
                 F.add(e)
            E.del(e)
       RETURN F
```

MINIMUM SPANNING TREE



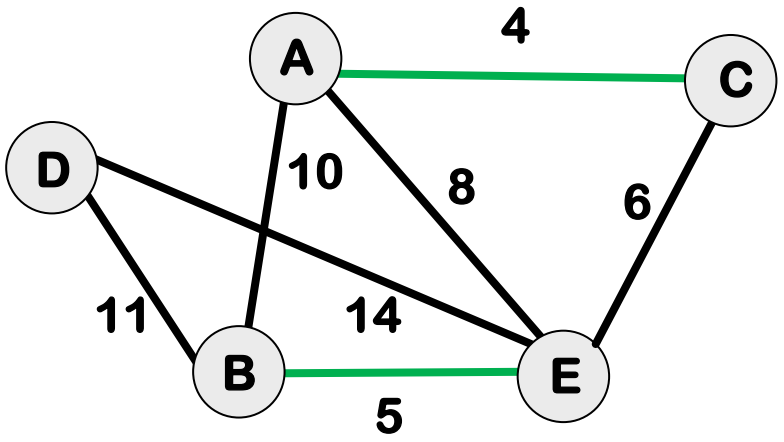
E	F
AC	
BE	
CE	
AE	
AB	
BD	
DE	
Input	Output

MINIMUM SPANNING TREE



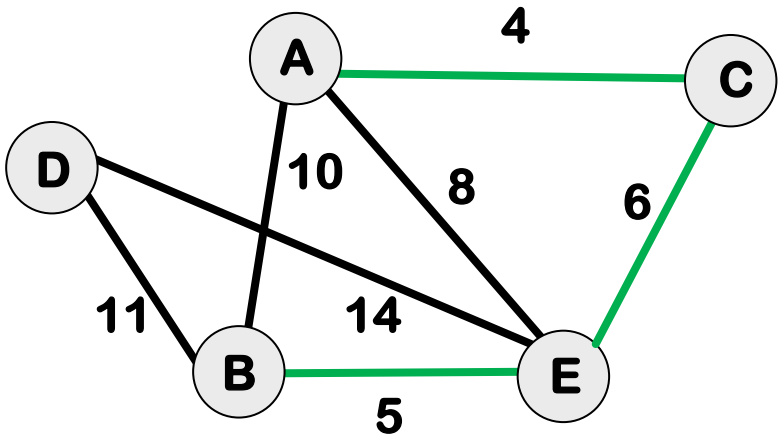
E	F
AC	AC
BE	
CE	
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



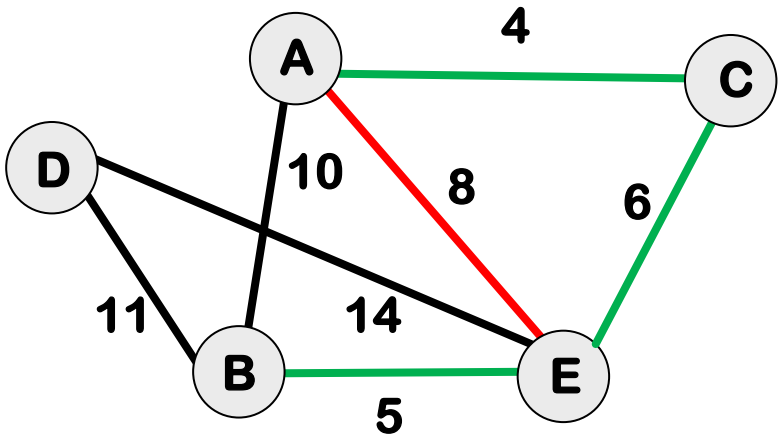
E	F
AC	AC
BE	BE
CE	
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



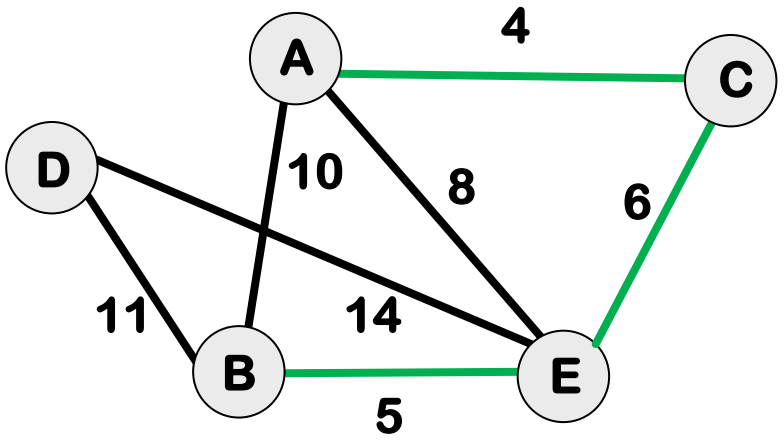
E	F
AC	AC
BE	BE
CE	CE
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



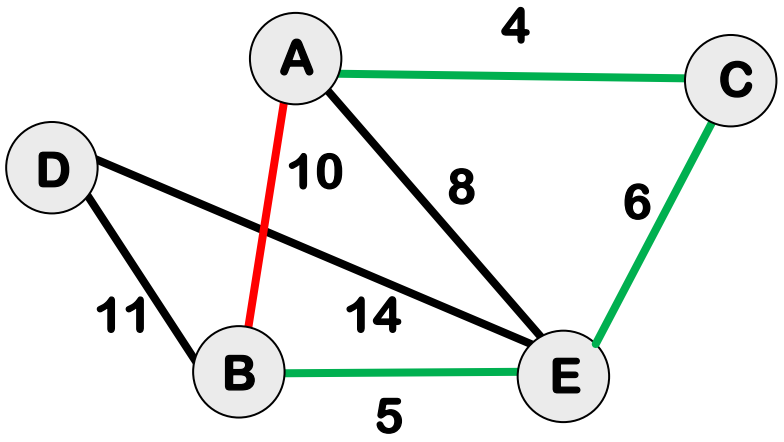
E	F
AC	AC
BE	BE
CE	CE
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



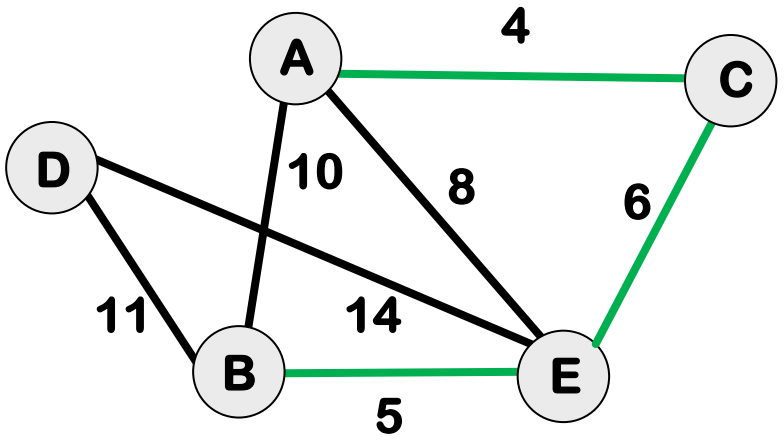
E	F
AC	AC
BE	BE
CE	CE
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



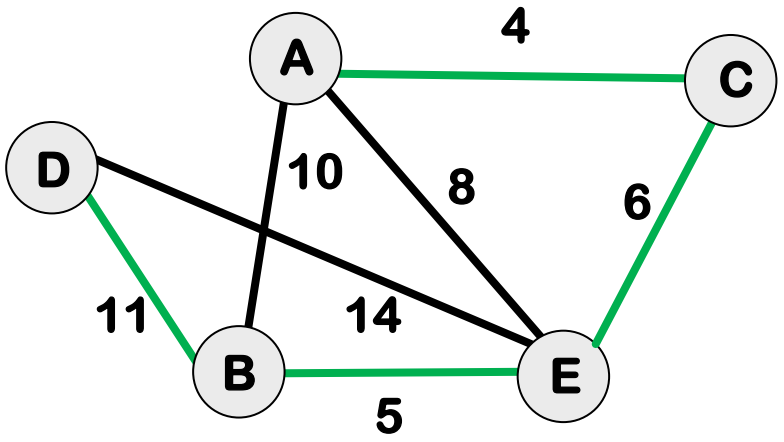
E	F
AC	AC
BE	BE
CE	CE
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



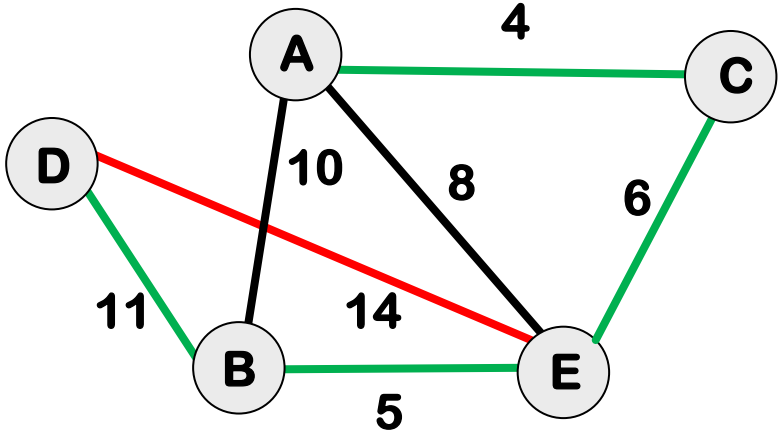
E	F
AC	AC
BE	BE
CE	CE
AE	
AB	
BD	
DE	

MINIMUM SPANNING TREE



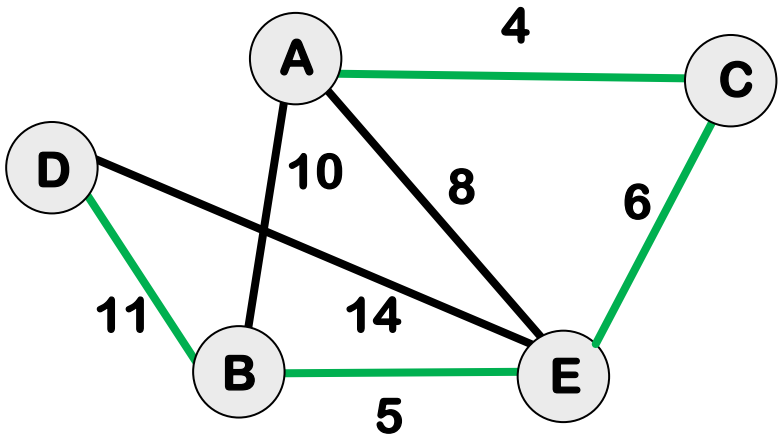
E	F
AC	AC
BE	BE
CE	CE
AE	BD
AB	
BD	
DE	

MINIMUM SPANNING TREE



E	F
AC	AC
BE	BE
CE	CE
AE	BD
AB	
BD	
DE	

MINIMUM SPANNING TREE



E	F
AC	AC
BE	BE
CE	CE
AE	BD
AB	
BD	
DE	

KRUSKAL'S ALGORITHM

```
MST(E)
      E.sort
      F=∅
      WHILE F!=MST DO
            e=min_length_edge
            IF  F+{e} has no cycle THEN
                 F.add(e)
            E.del(e)
       RETURN F
```

KRUSKAL'S ALGORITHM

```
MST(E)
      E.sort
      F=∅
      WHILE F!=MST DO
            e=min_length_edge
            IF  F+{e} has no cycle THEN
                 F.add(e)
            E.del(e)
       RETURN F
```

KRUSKAL'S ALGORITHM

```
MST(E)
      E.sort
      F=∅
      WHILE len(F)!=n-1 DO
            e=min_length_edge
            IF  F+{e} has no cycle THEN
                 F.add(e)
            E.del(e)
       RETURN F
```

KRUSKAL'S ALGORITHM

```
MST(E)
      E.sort
      F=∅
      WHILE len(F)!=n-1 DO
            e=min_length_edge
            IF  F+{e} has no cycle THEN
                  F.add(e)
            E.del(e)
       RETURN F
```

Time Complexity?

KRUSKAL'S ALGORITHM

```
MST(E)
      E.sort
      F=∅
      WHILE len(F)!=n-1 DO
            e=min_length_edge
            IF  F+{e} has no cycle THEN
                  F.add(e)
            E.del(e)
       RETURN F
```

$O(|E| \times ?)$

KRUSKAL'S ALGORITHM: DETECT A CYCLE

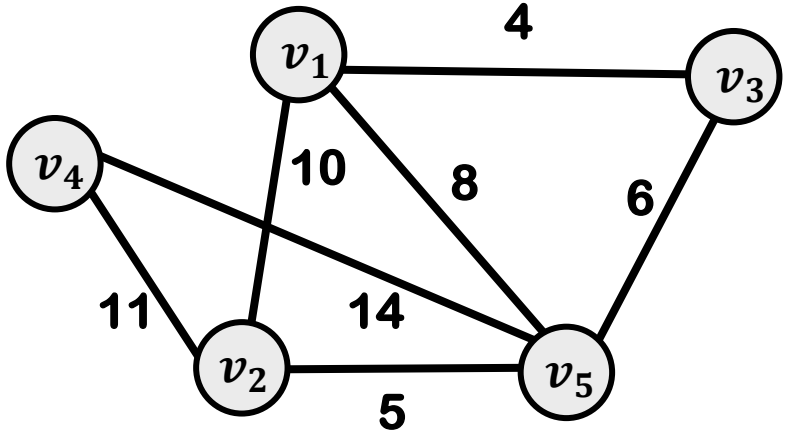
$m = |E|$

Method	Time complexity
DFS/BFS	$O(mn)$
Union-Find – Array Implementation	$O(mn)$
Union-Find – Tree Implementation	$O(mn)$
Weighted Union-Find	$O(m \log n)$
Weighted Union-Find + Path Compression	$O(m\alpha(n))$

∗ α is the **inverse Ackermann function**, which grows *extremely slowly* - practically constant time.

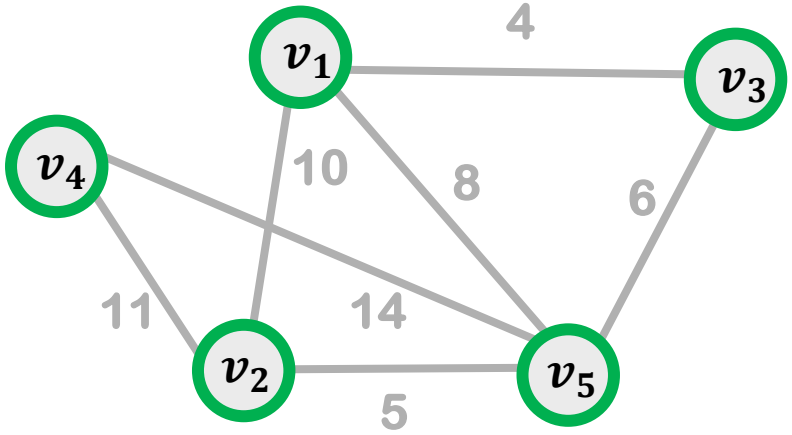
MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.



MST: KRUSKAL

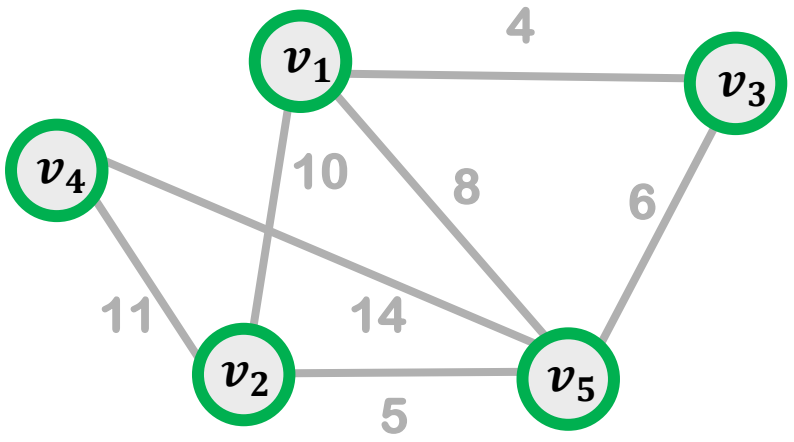
we partition the nodes into disjoint sets based on their connectivity in MST.



$\{v_1\}$ $\{v_2\}$ $\{v_3\}$ $\{v_4\}$ $\{v_5\}$

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

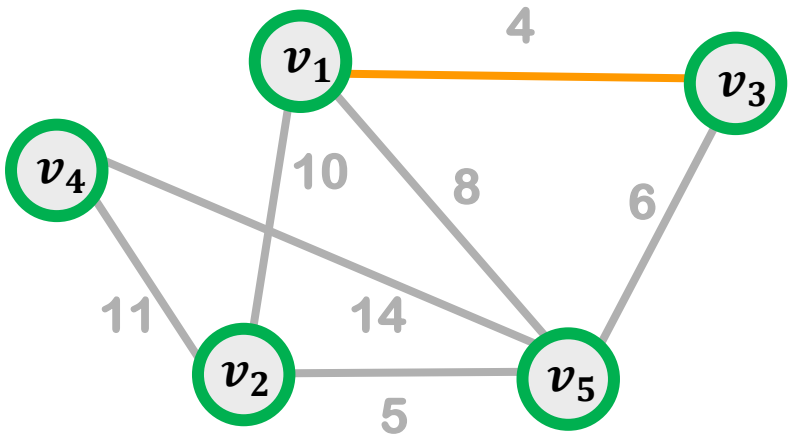


$\{v_1\}$ $\{v_2\}$ $\{v_3\}$ $\{v_4\}$ $\{v_5\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

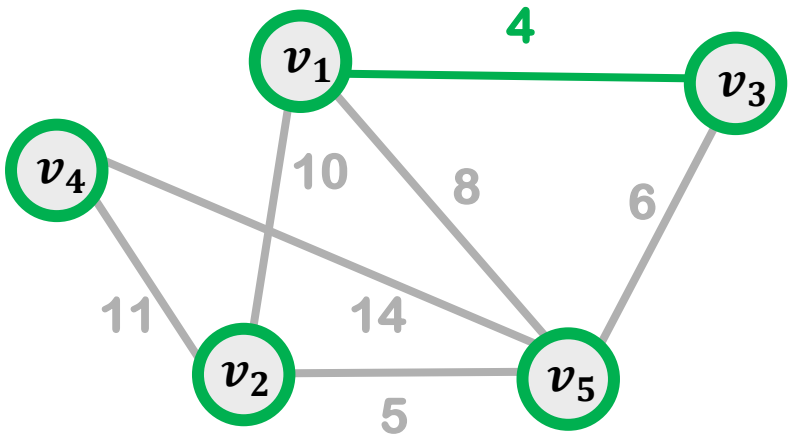


$\{v_1\}$ $\{v_2\}$ $\{v_3\}$ $\{v_4\}$ $\{v_5\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

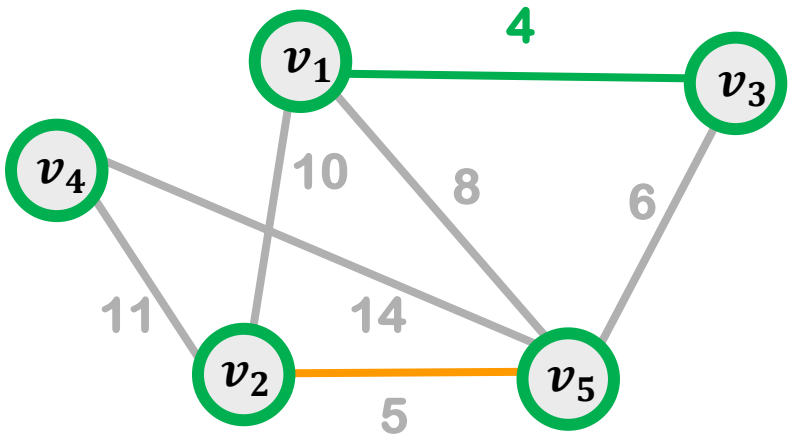


$\{v_1, v_3\}$ $\{v_2\}$ $\{v_4\}$ $\{v_5\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

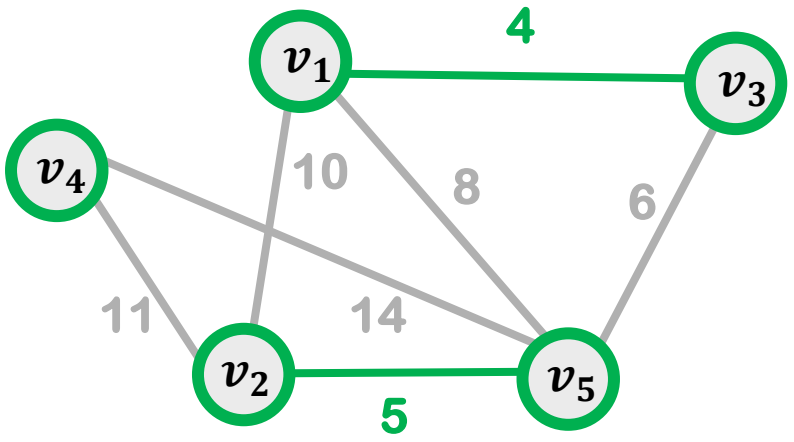


$\{v_1, v_3\}$ $\{v_2\}$ $\{v_4\}$ $\{v_5\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

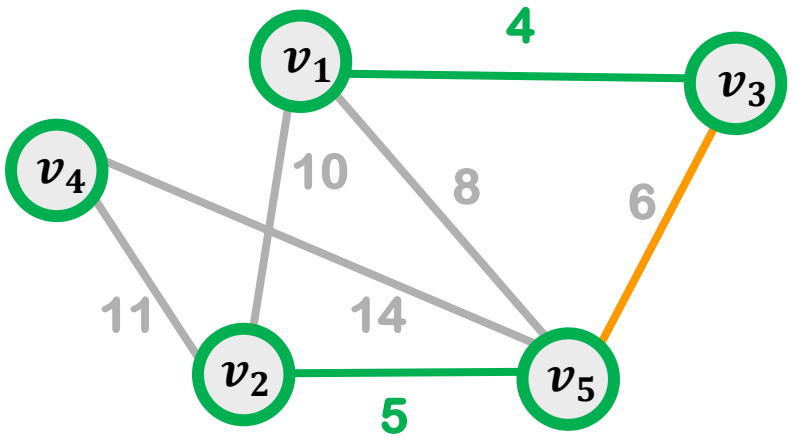


$\{v_1, v_3\}$ $\{v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

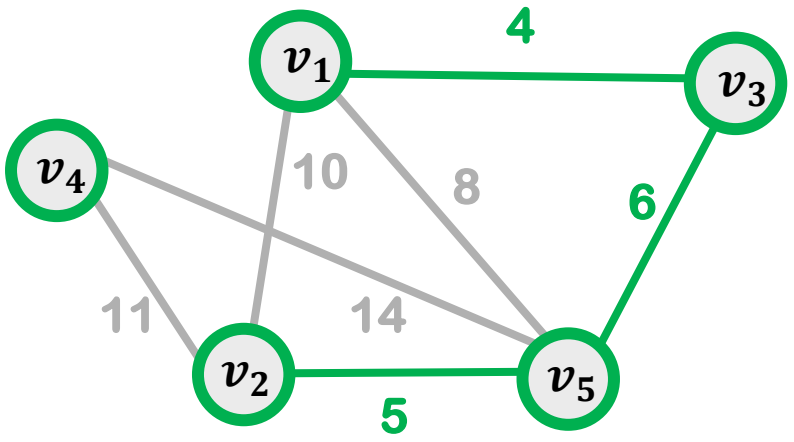


$\{v_1, v_3\}$ $\{v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

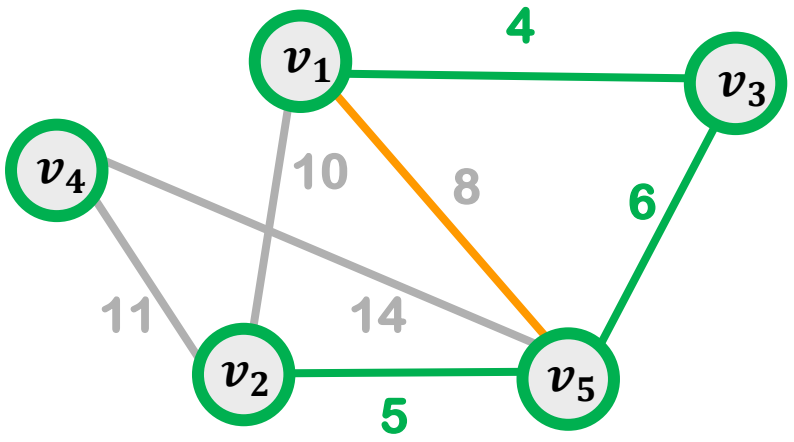


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

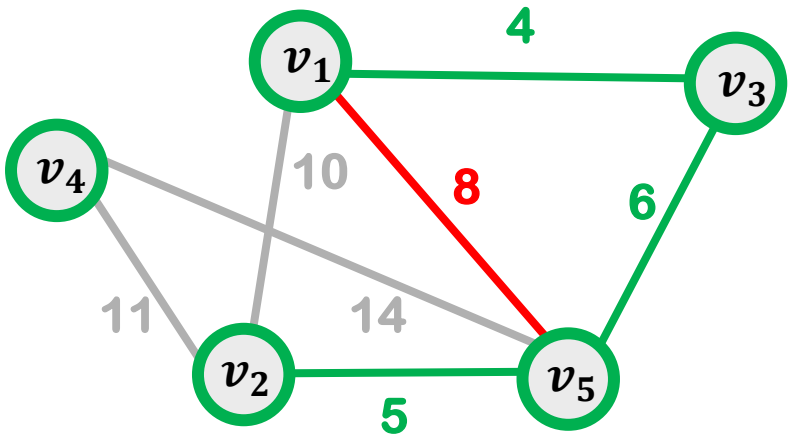


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

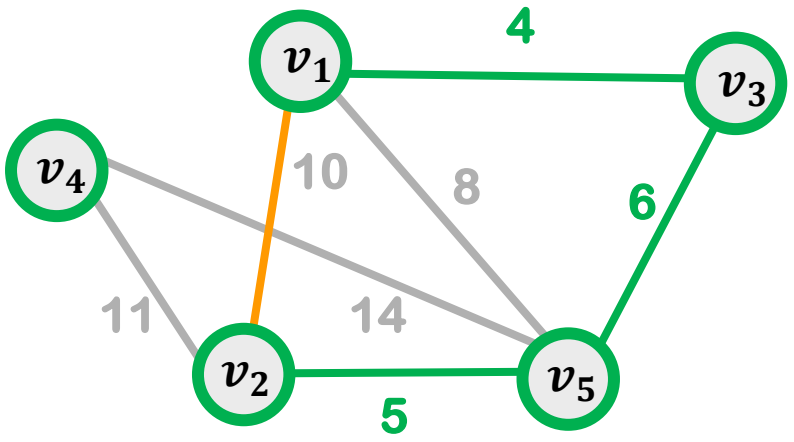


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

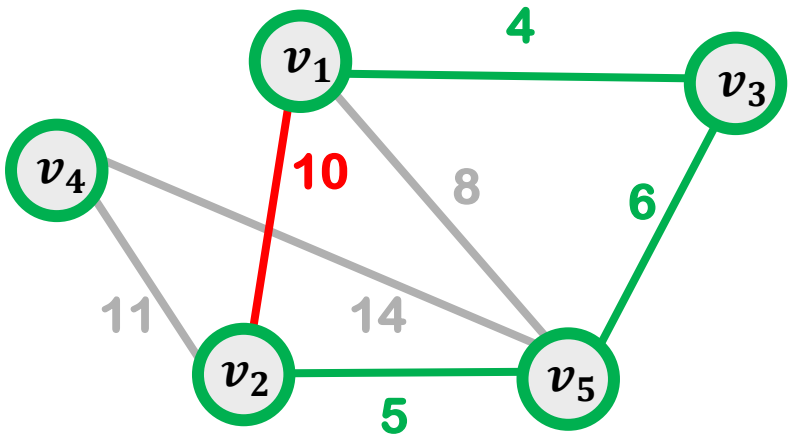


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

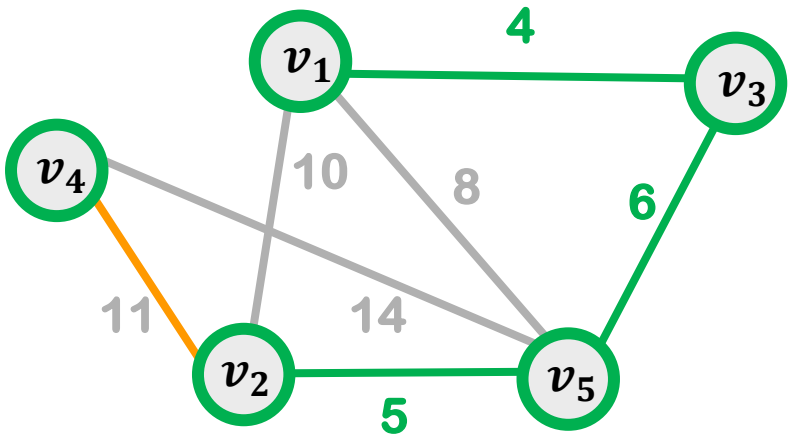


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.

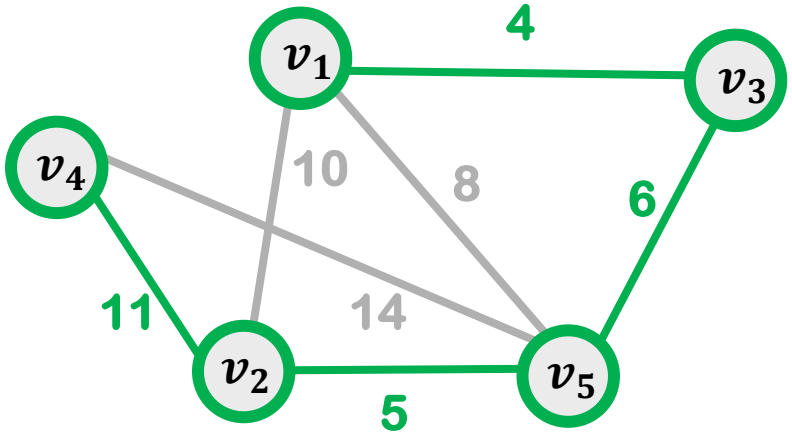


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

MST: KRUSKAL

we partition the nodes into disjoint sets based on their connectivity in MST.



$$\{v_1, v_3, v_2, v_5, v_4\}$$

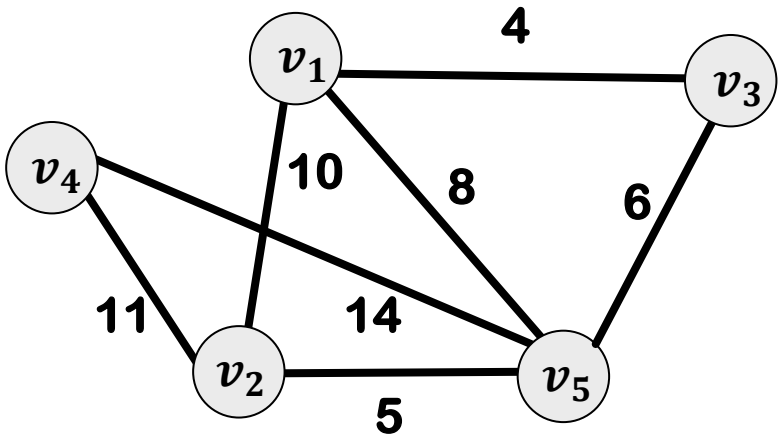
If e connects nodes in the same set, skip it; otherwise, accept it and merge the sets.

KRUSKAL'S ALGORITHM: UNION-FIND

```
MST(E)
      E.sort
      F=∅
      Initialize n sets, each contains one vertex of G
      WHILE len(F)!=n-1 DO
            e=min_length_edge
            IF e connects nodes in disjoint sets THEN
                  F.add(e)
                  merge two sets
             E.del(e)
       RETURN F
```

$O(|E| \times ?)$

MST: KRUSKAL



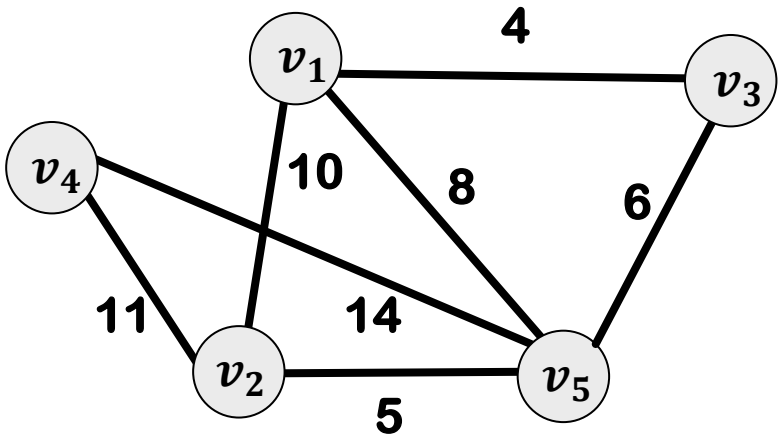
1	2	3	4	5
$\{v_1\}$	$\{v_2\}$	$\{v_3\}$	$\{v_4\}$	$\{v_5\}$

vertices	v1	v2	v3	v4	v5
Index of Set					

S

S[k]= index of the set containing v_k

MST: KRUSKAL

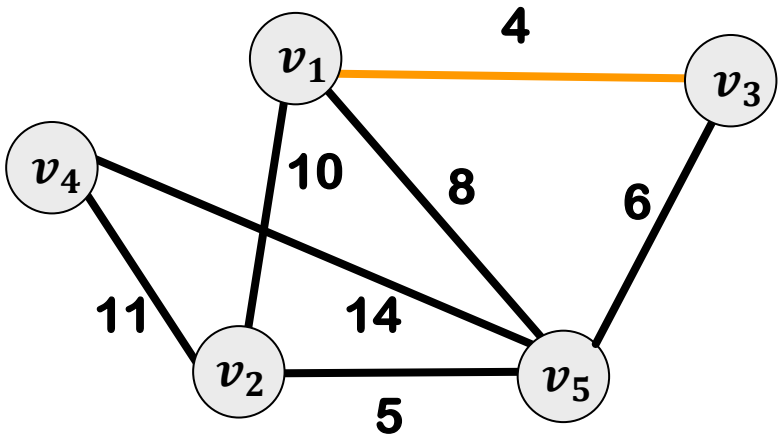


1	2	3	4	5
$\{v_1\}$	$\{v_2\}$	$\{v_3\}$	$\{v_4\}$	$\{v_5\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	2	3	4	5

S

MST: KRUSKAL

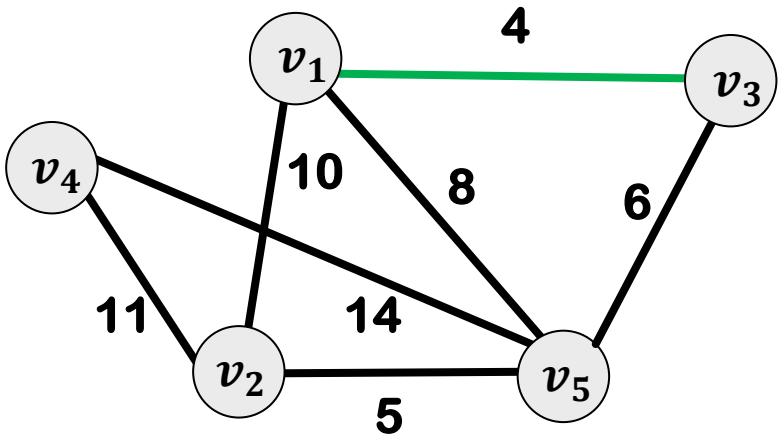


$\{v_1\}$ $\{v_2\}$ $\{v_3\}$ $\{v_4\}$ $\{v_5\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	2	3	4	5

S

MST: KRUSKAL

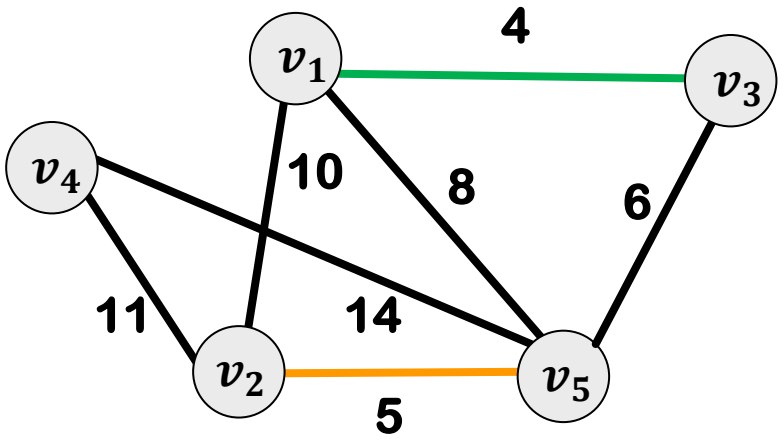


$\{v_1, v_3\}$ $\{v_2\}$ $\{v_4\}$ $\{v_5\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	2	1	4	5

S

MST: KRUSKAL

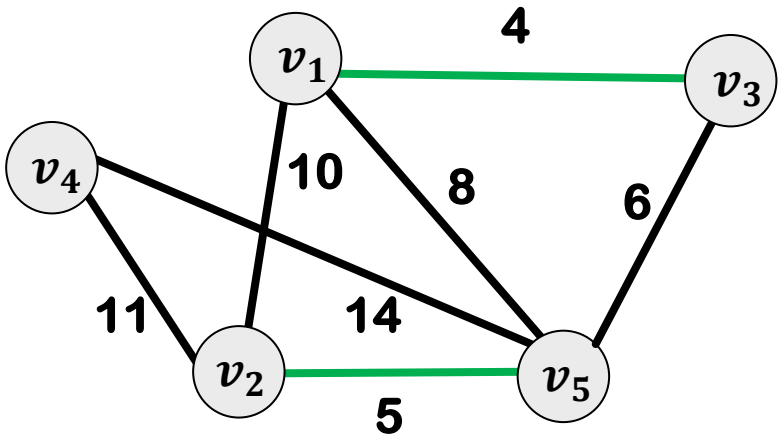


$\{v_1, v_3\}$ $\{v_2\}$ $\{v_4\}$ $\{v_5\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	2	1	4	5

S

MST: KRUSKAL

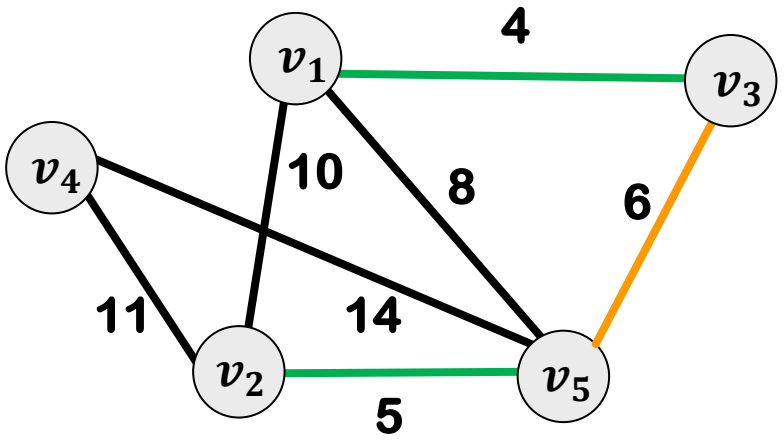


$\{v_1, v_3\}$ $\{v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	2	1	4	2

S

MST: KRUSKAL

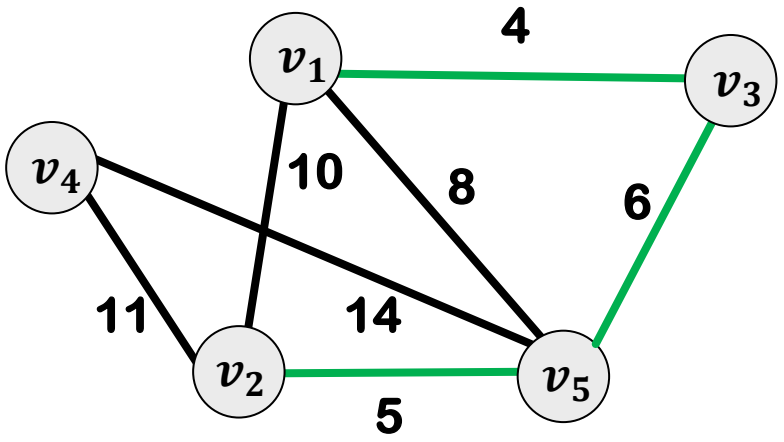


$\{v_1, v_3\}$ $\{v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	2	1	4	2

S

MST: KRUSKAL

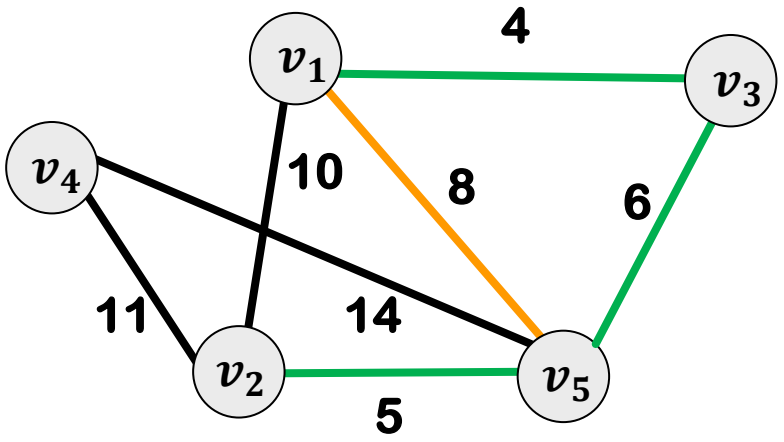


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	4	1

S

MST: KRUSKAL

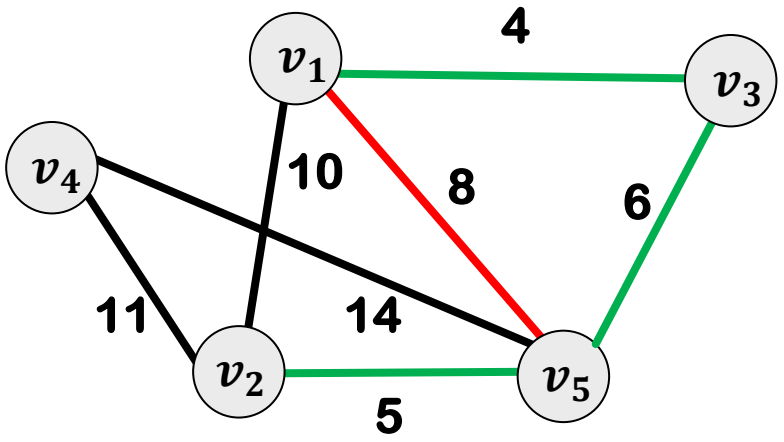


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	4	1

S

MST: KRUSKAL

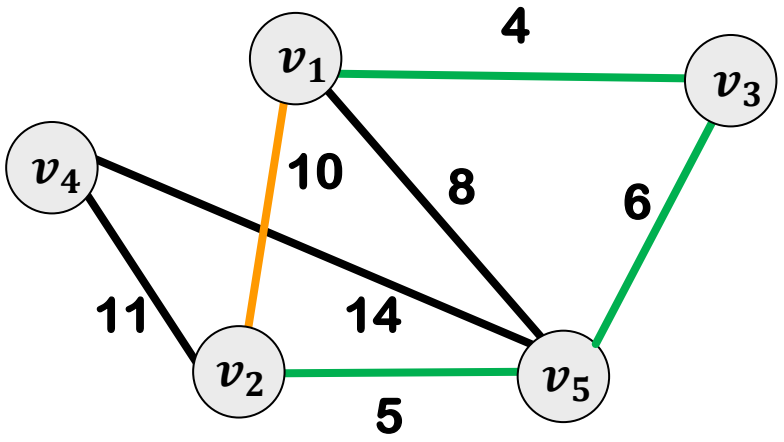


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	4	1

S

MST: KRUSKAL

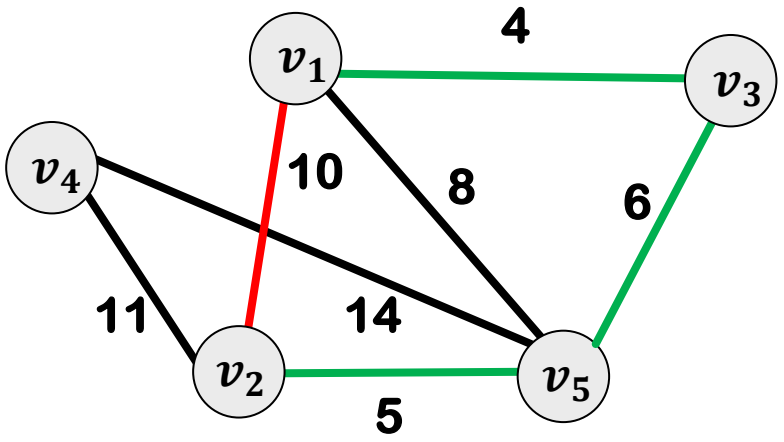


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	4	1

S

MST: KRUSKAL

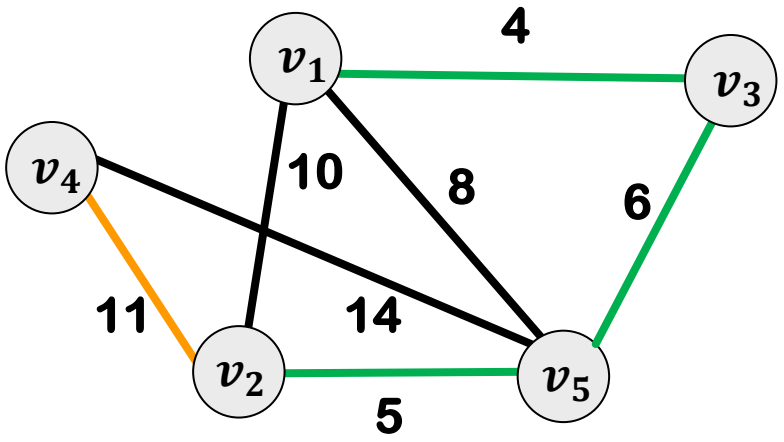


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	4	1

S

MST: KRUSKAL

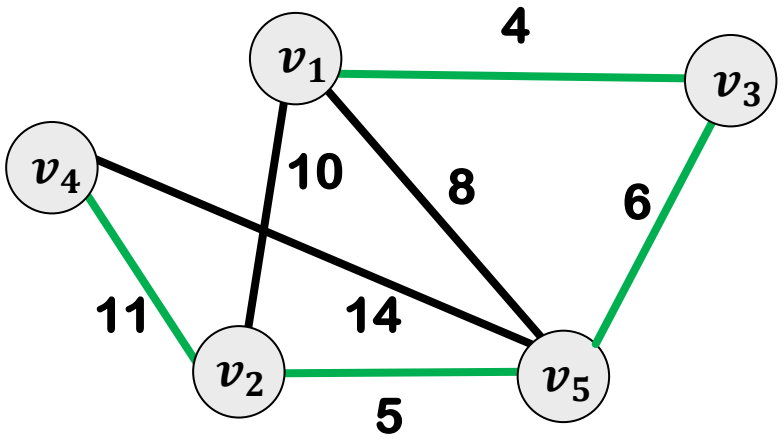


$\{v_1, v_3, v_2, v_5\}$ $\{v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	4	1

S

MST: KRUSKAL



$\{v_1, v_3, v_2, v_5, v_4\}$

vertices	v1	v2	v3	v4	v5
Index of Set	1	1	1	1	1

S

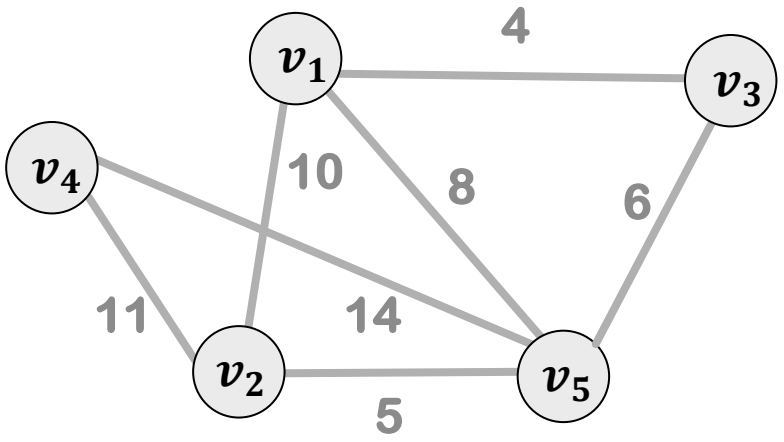
KRUSKAL'S ALGORITHM: UNION-FIND

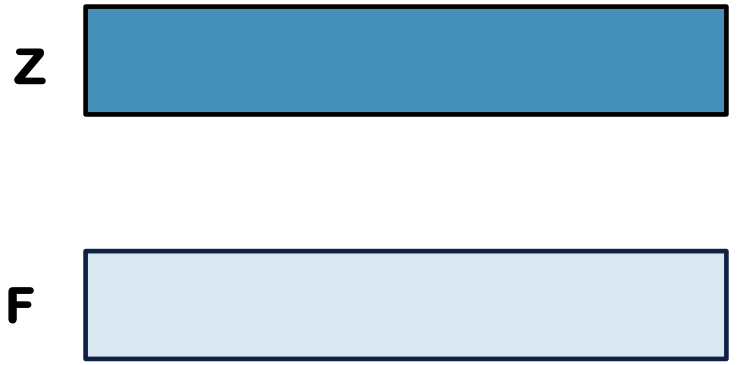
```
MST(E)
      E.sort
      F=∅
      FOR k=1 to n DO   S[k]=k
      WHILE len(F)!=n-1 DO
              e=(v_i,v_j)=min_length_edge
              IF S[i]!=S[j] THEN
                    F.add(e)
                    Min=min(S[i],S[j])
                    Max=max(S[i],S[j])
                    FOR k=1 to n DO
                          IF S[k]==Max THEN S[k]=Min
              E.del(e)

         RETURN F
```

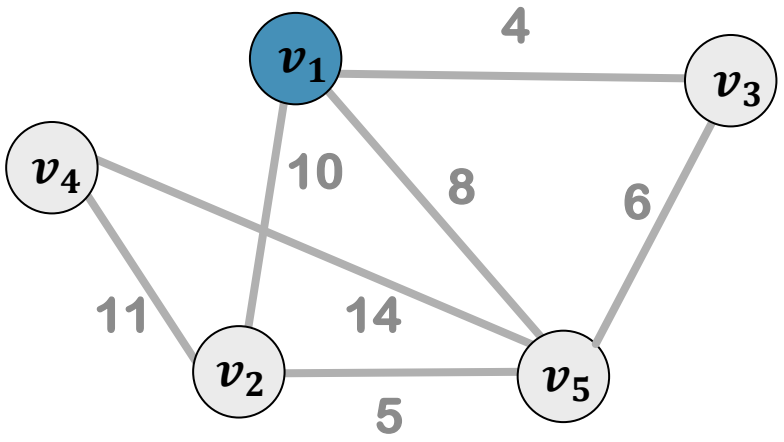
$$O(n|E|) = O(n^3)$$

MST: PRIM





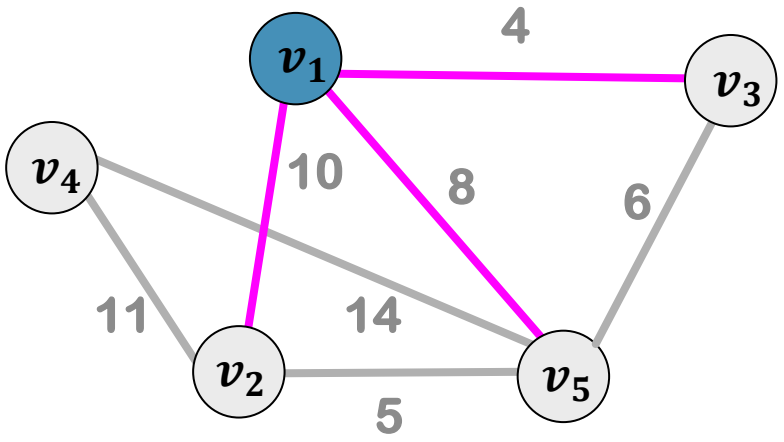
MST: PRIM

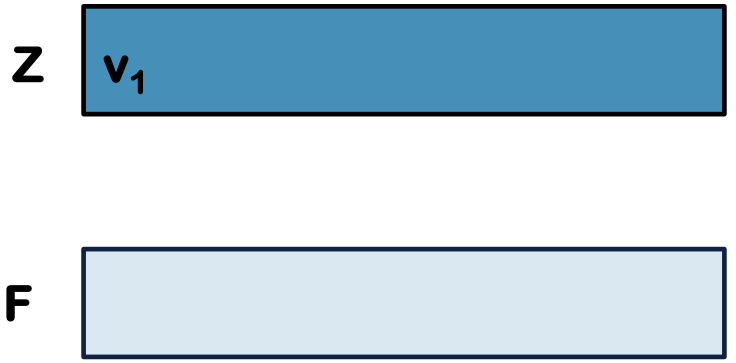


Z v_1

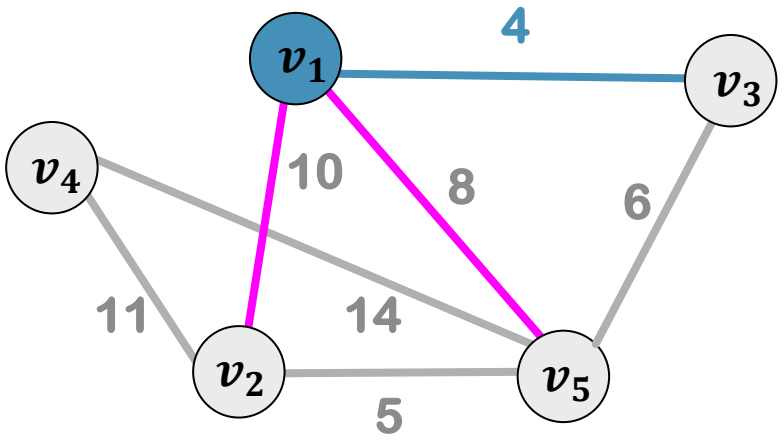
F

MST: PRIM





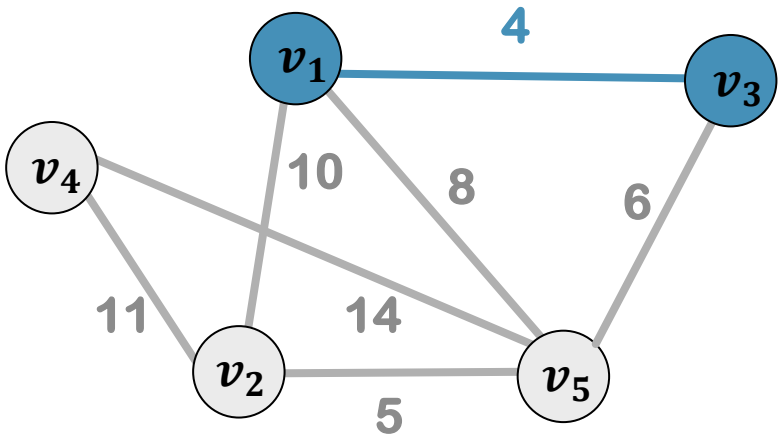
MST: PRIM



Z v_1

F v_1v_3

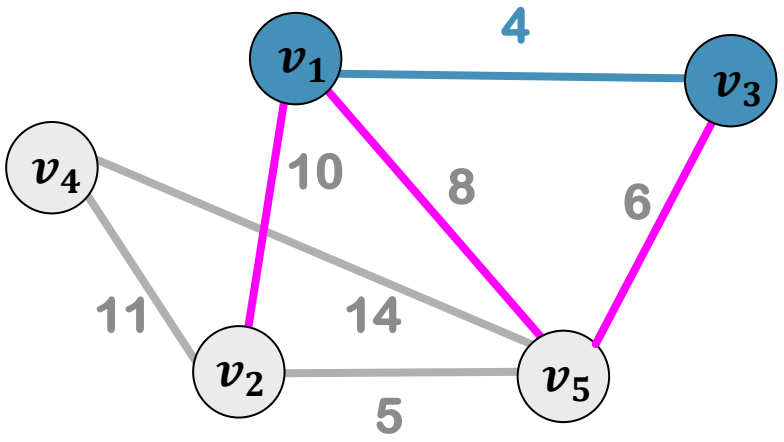
MST: PRIM



Z v_1 , v_3

F v_1v_3

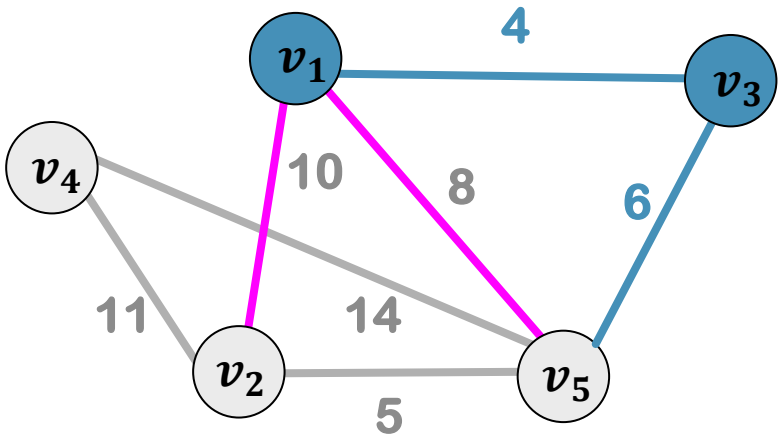
MST: PRIM



Z v_1 , v_3

F v_1v_3

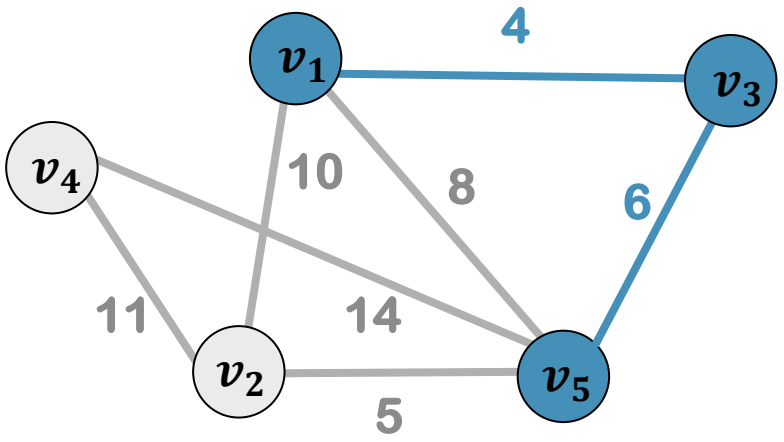
MST: PRIM



Z v_1 , v_3

F v_1v_3 , v_3v_5

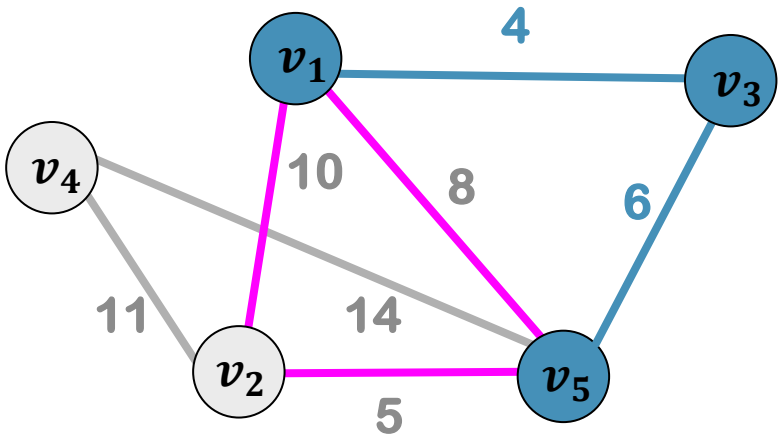
MST: PRIM



Z v_1, v_3, v_5

F v_1v_3, v_3v_5

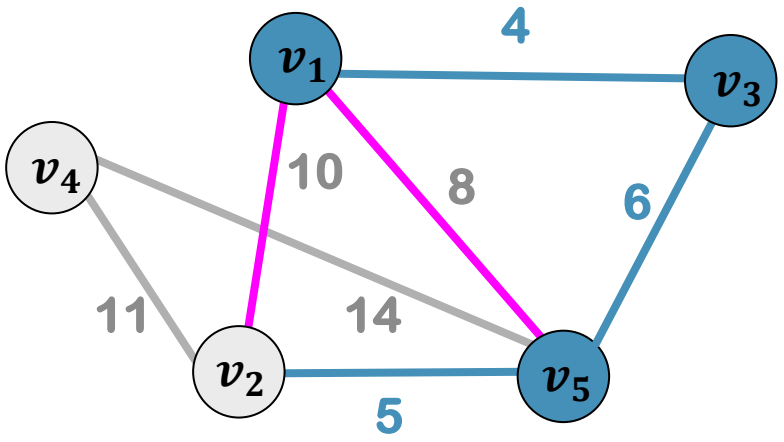
MST: PRIM



Z v_1, v_3, v_5

F v_1v_3, v_3v_5

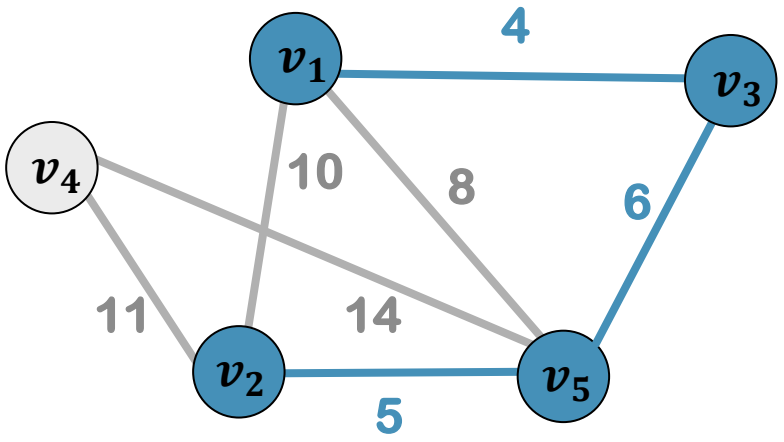
MST: PRIM



Z | v_1 , v_3 , v_5

F | v_1v_3 , v_3v_5 , v_2v_5

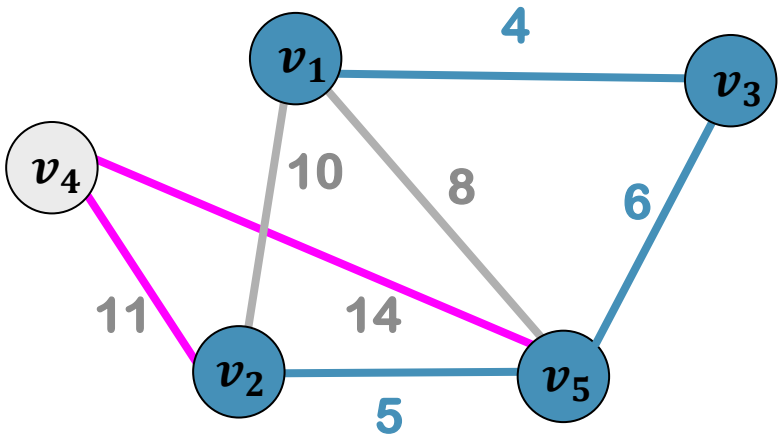
MST: PRIM



Z v_1, v_3, v_5, v_2

F v_1v_3, v_3v_5, v_2v_5

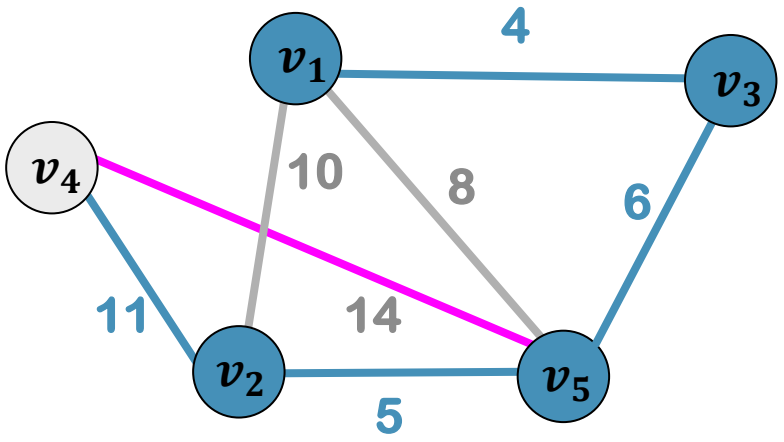
MST: PRIM



Z v_1, v_3, v_5, v_2

F v_1v_3, v_3v_5, v_2v_5

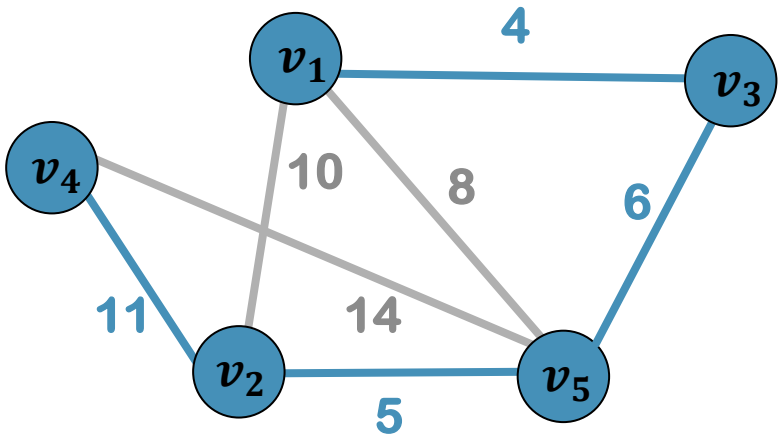
MST: PRIM



Z v_1, v_3, v_5, v_2

F $v_1v_3, v_3v_5, v_2v_5, v_2v_4$

MST: PRIM



Z v_1, v_3, v_5, v_2, v_4

F $v_1v_3, v_3v_5, v_2v_5, v_2v_4$

MINIMUM SPANNING TREE

```
Prim()
   F=∅
   Z={v1}
   WHILE Z!=V DO
       find edge(a,b) of minimum length: a in Z and b in V-Z
       F.add((a,b))
       Z.add(b)

    RETURN F
```

MINIMUM SPANNING TREE

```
Prim()
   F=∅
   Z={v1}
   WHILE Z!=V DO
       find edge(a,b) of minimum length: a in Z and b in V-Z
       F.add((a,b))
       Z.add(b)

    RETURN F
```

Time Complexity?

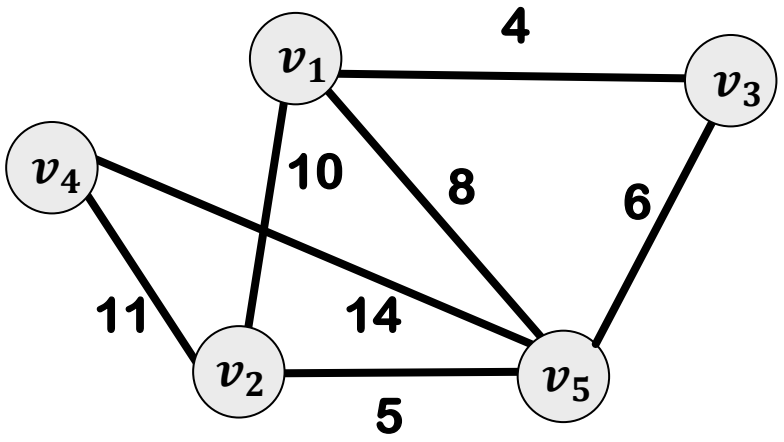
MINIMUM SPANNING TREE

```
Prim()
   F=∅
   Z={v1}
   WHILE Z!=V DO
       find edge(a,b) of minimum length: a in Z and b in V-Z
       F.add((a,b))
       Z.add(b)

    RETURN F
```

$O(n \times ?)$

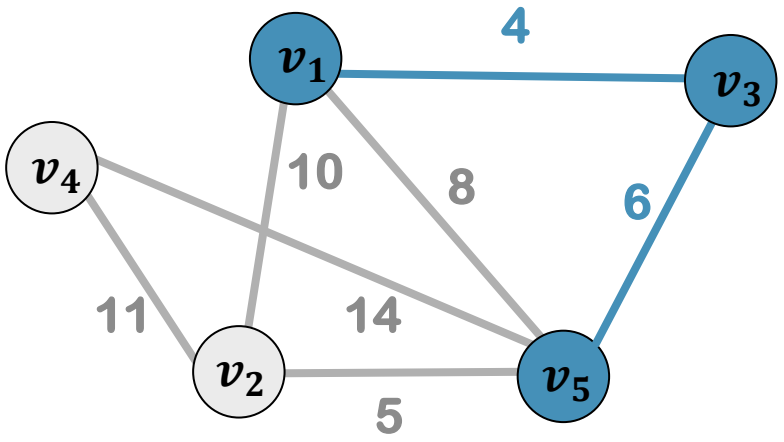
PRIM: NOTATIONS



	v_1	v_2	v_3	v_4	v_5
v_1	0	10	4	∞	8
v_2	10	0	∞	11	5
v_3	4	∞	0	∞	6
v_4	∞	11	∞	0	14
v_5	8	5	6	14	0

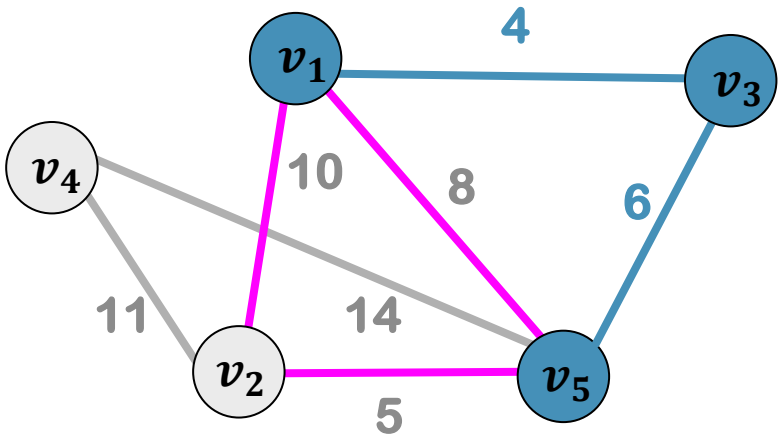
Adjacency Matrix: W

MST: PRIM



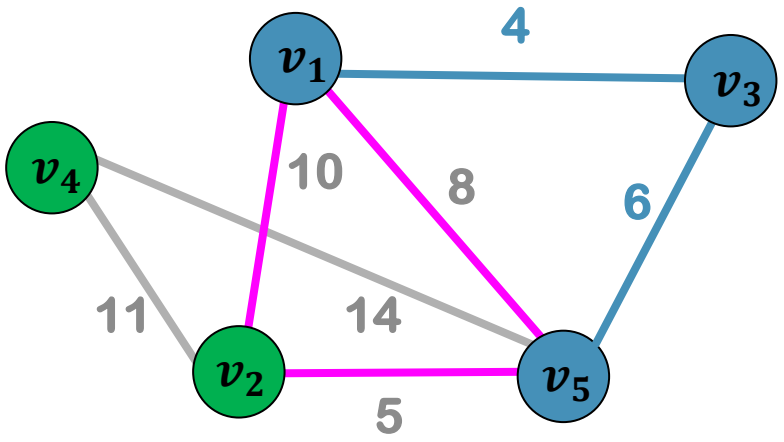
Z | v_1 , v_3 , v_5

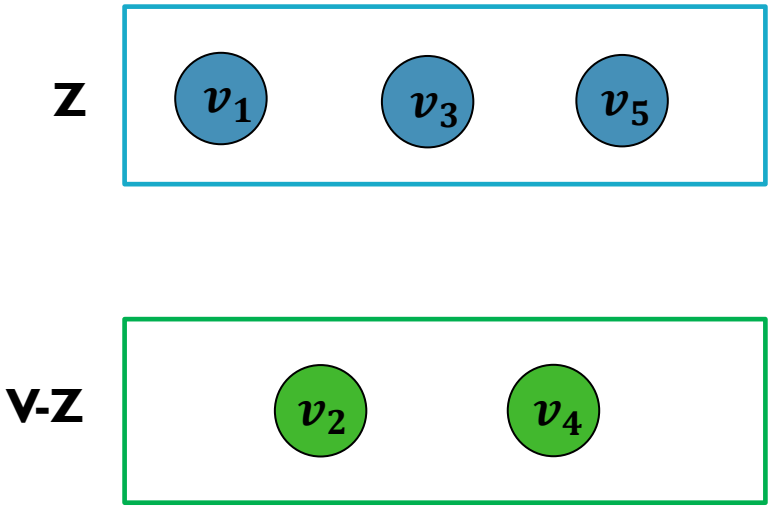
MST: PRIM



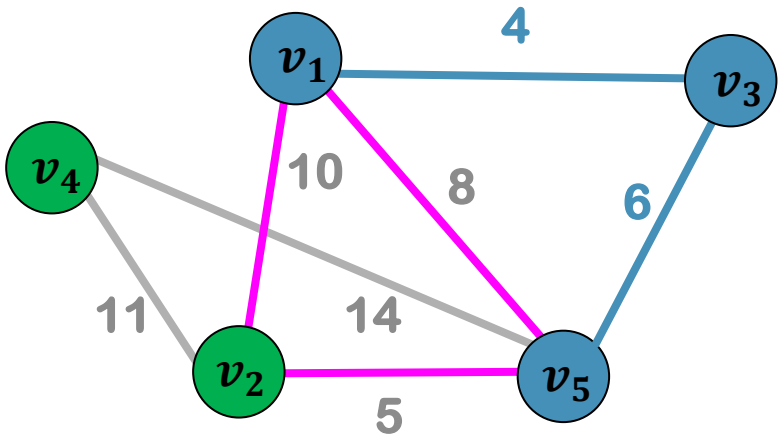
Z | v_1 , v_3 , v_5

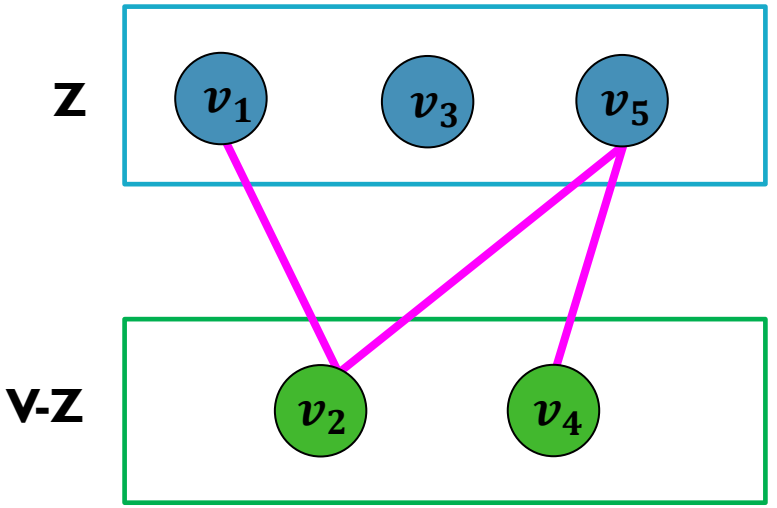
MST: PRIM



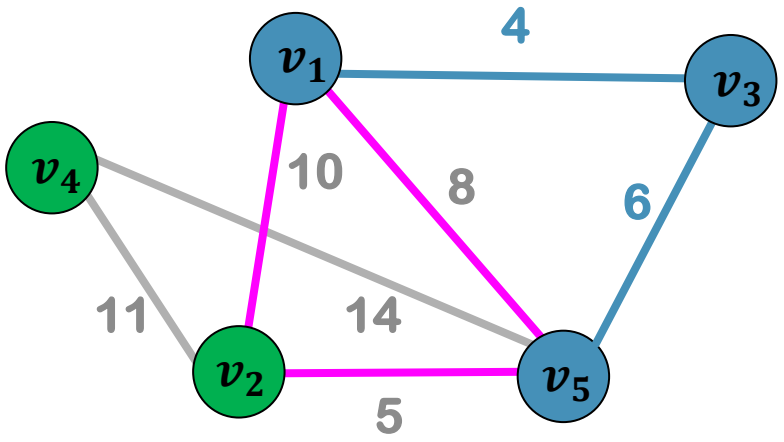


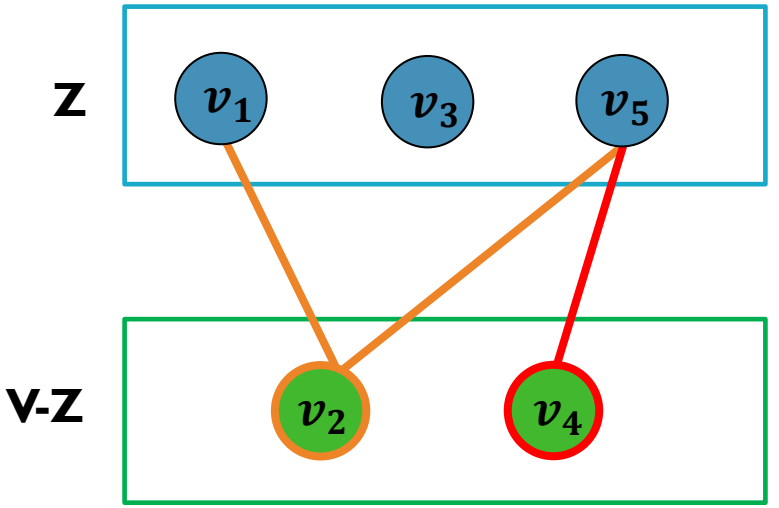
MST: PRIM





MST: PRIM

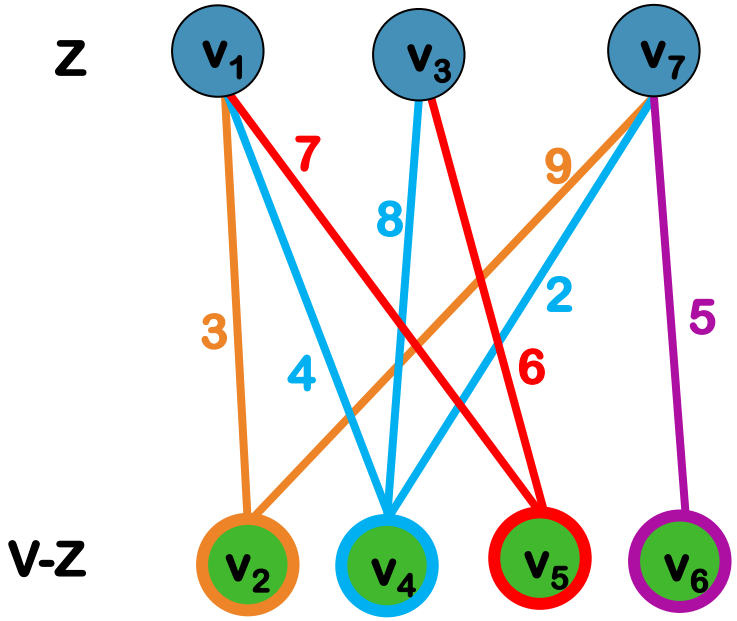




Min {v_1v_2 ,v_5v_2 ,v_5v_4 }

Min {Min {v_2v_1 ,v_2v_5 },v_4v_5 }

PRIM: NOTATIONS



PRIM: NOTATIONS

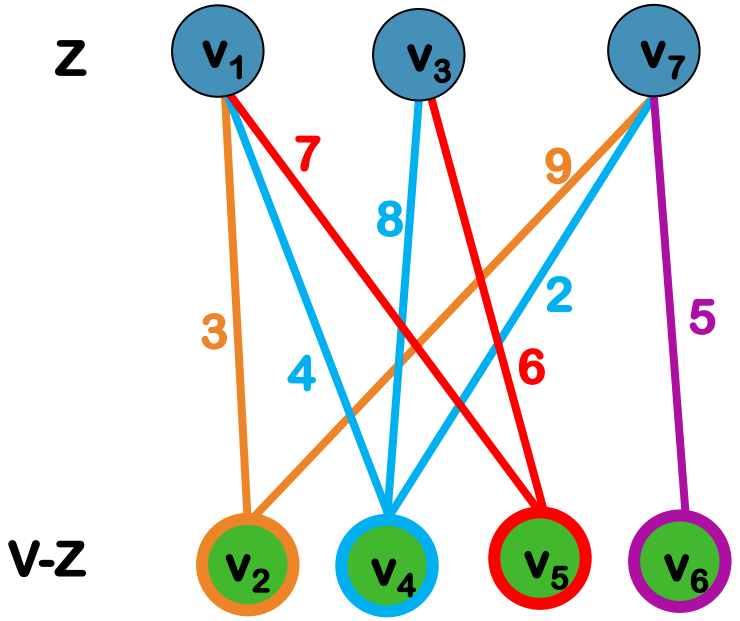
$\text{Min}\{v_2v_1, v_2v_7\} = v_2v_1$

$\text{Min}\{v_4v_1, v_4v_3, v_4v_7\} = v_4v_7$

$\text{Min}\{v_5v_1, v_5v_3\} = v_5v_3$

$\text{Min}\{v_6v_7\} = v_6v_7$

} v_4v_7



PRIM: NOTATIONS

nearest[i]= the other endpoint of the shortest edge in group v_i

distance[i]= length of the shortest edge in group v_i

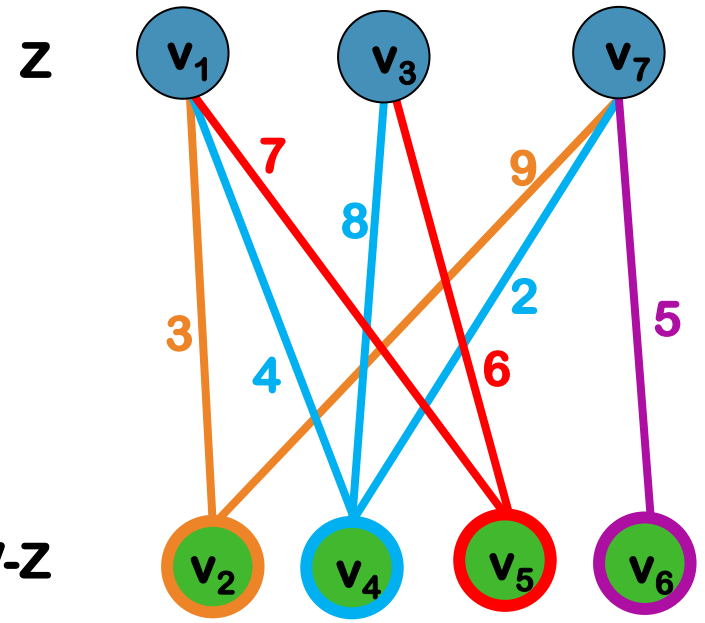
$\text{Min}\{v_2v_1, v_2v_7\} = v_2v_1$

$\text{Min}\{v_4v_1, v_4v_3, v_4v_7\} = v_4v_7$

$\text{Min}\{v_5v_1, v_5v_3\} = v_5v_3$

$\text{Min}\{v_6v_7\} = v_6v_7$

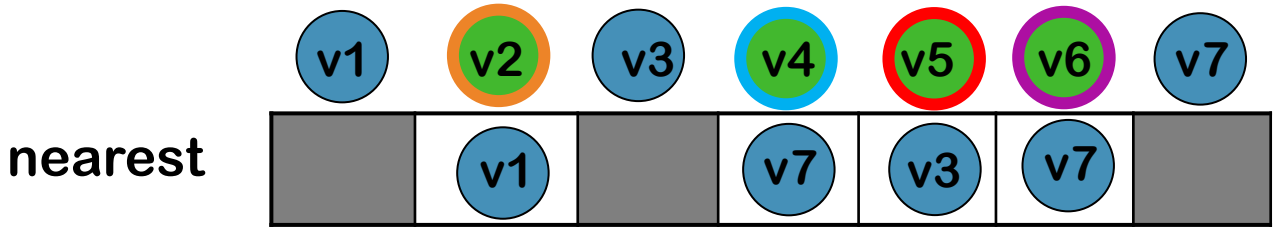
} v_4v_7

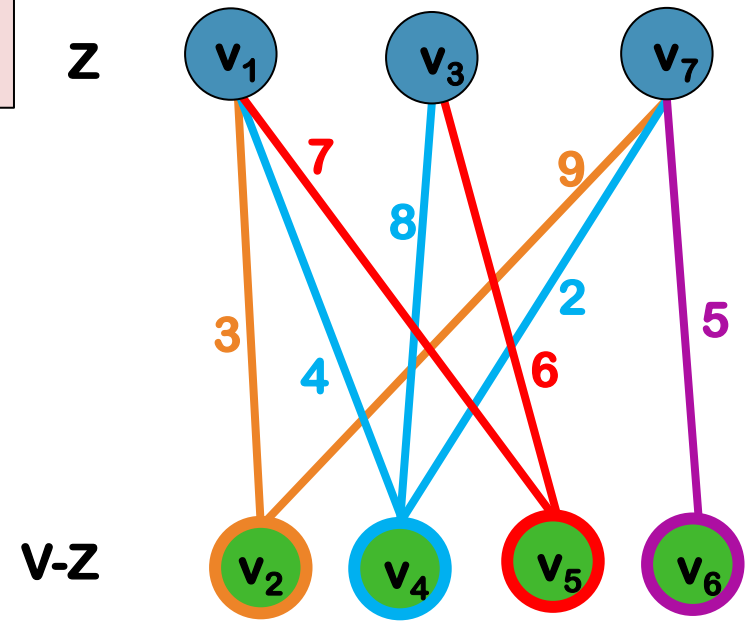


PRIM: NOTATIONS

nearest[i]= the other endpoint of the shortest edge in group v_i

distance[i]= length of the shortest edge in group v_i





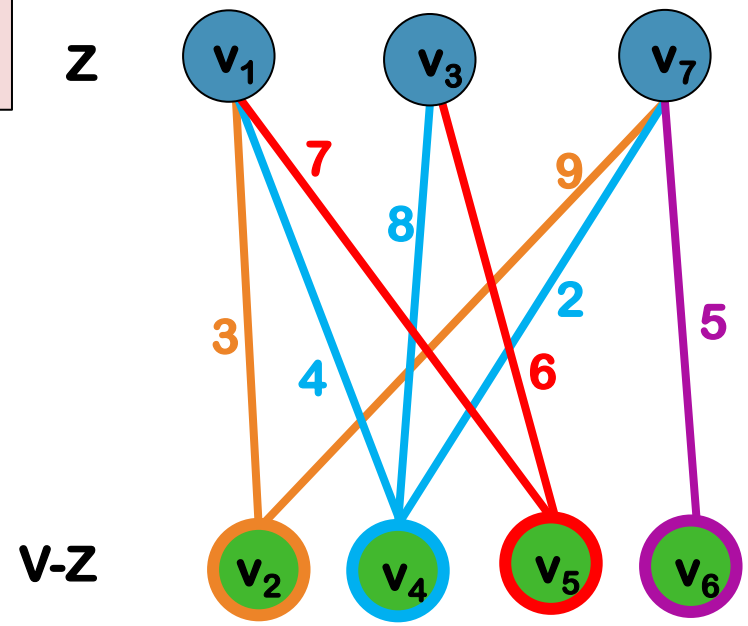
PRIM: NOTATIONS

nearest[i]= the other endpoint of the shortest edge in group v_i

distance[i]= length of the shortest edge in group v_i

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

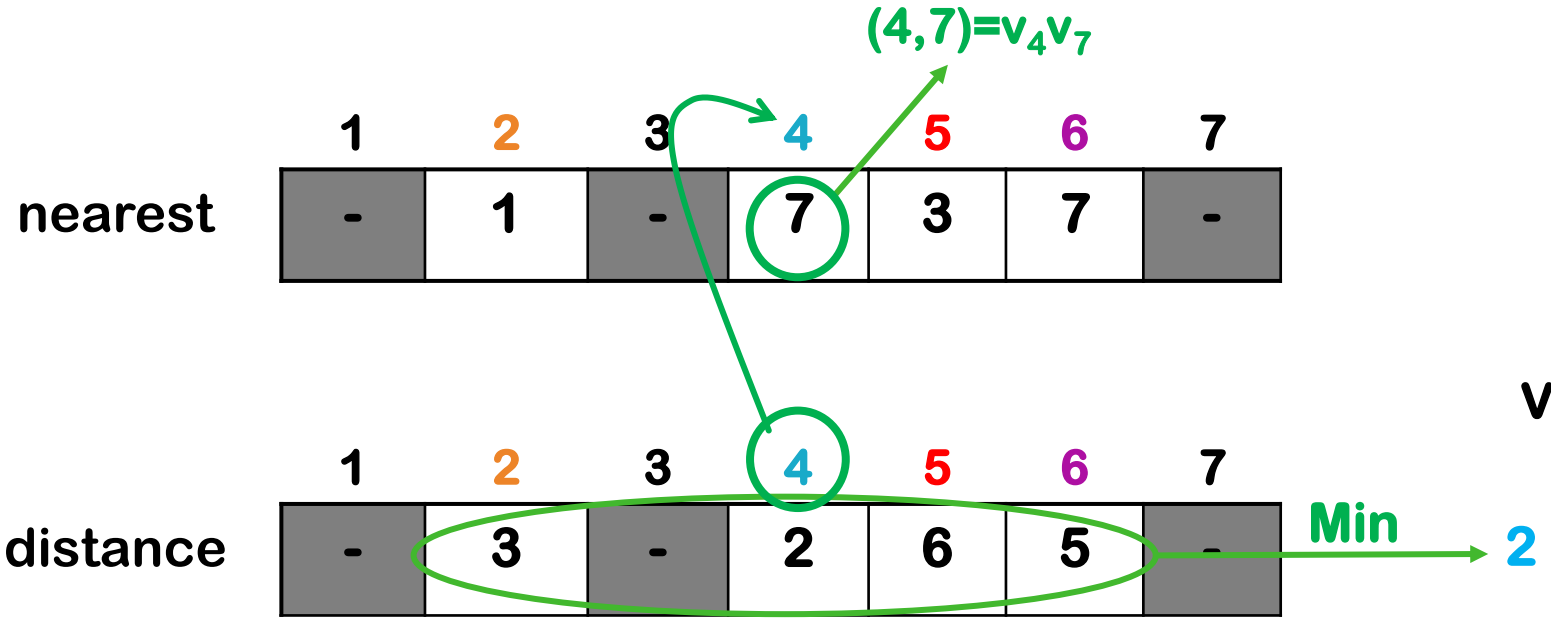
	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-

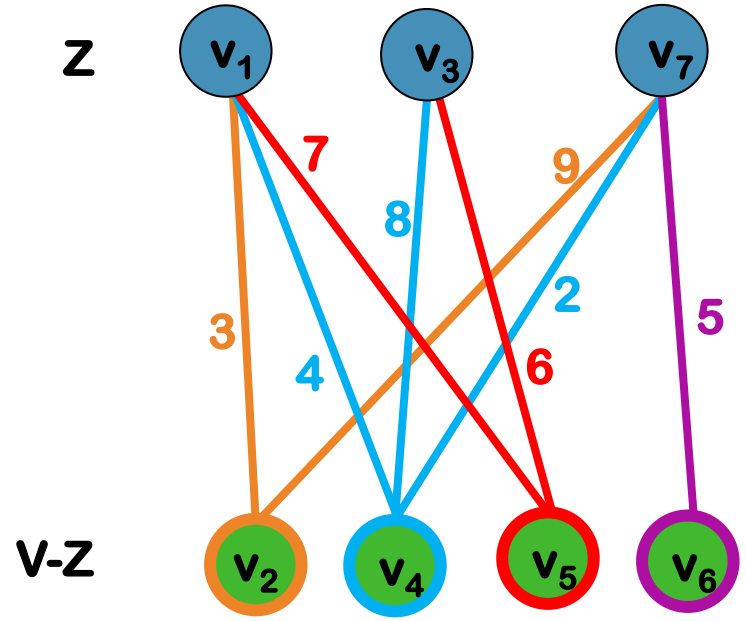


PRIM: NOTATIONS

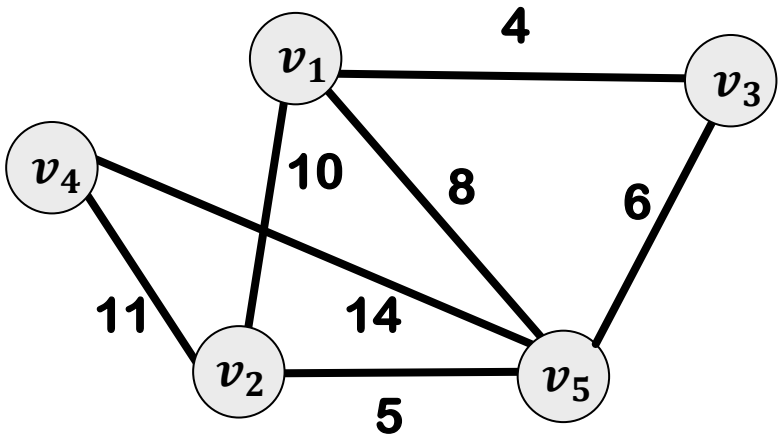
nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]





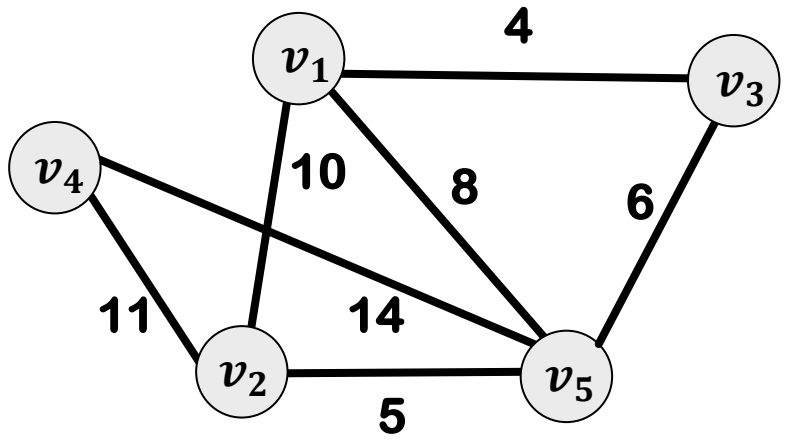
PRIM: HOW TO USE NEAREST AND DISTANCE



	v_1	v_2	v_3	v_4	v_5
v_1	0	10	4	∞	8
v_2	10	0	∞	11	5
v_3	4	∞	0	∞	6
v_4	∞	11	∞	0	14
v_5	8	5	6	14	0

Adjacency Matrix: W

MST: PRIM



nearest

1	2	3	4	5
-	-	-	-	-

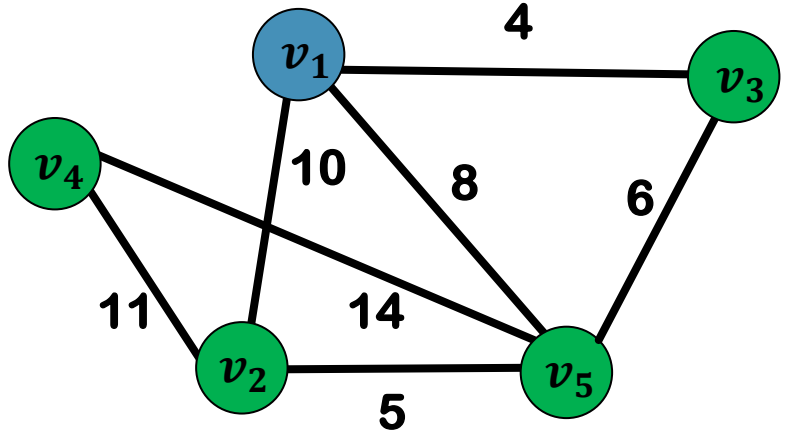
distance

1	2	3	4	5
-	-	-	-	-

nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]

MST: PRIM



nearest

1	2	3	4	5
-	-	-	-	-

distance

1	2	3	4	5
-	-	-	-	-

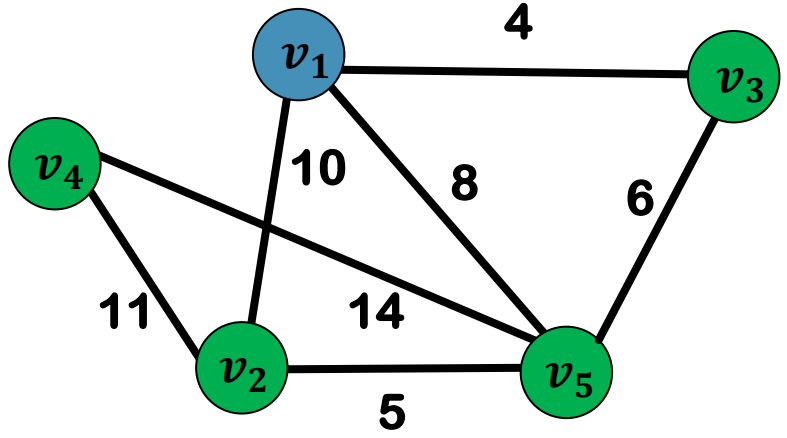
Z v_1

F

nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]

MST: PRIM



	1	2	3	4	5
nearest	-	1	1	1	1

	1	2	3	4	5
distance	-	-	-	-	-

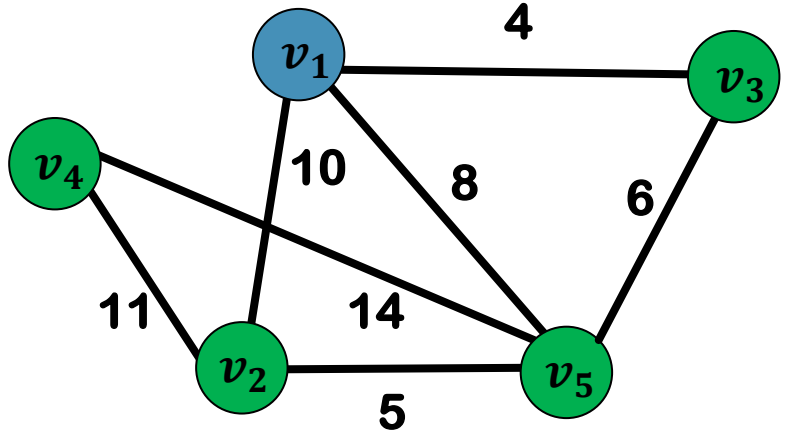
Z v_1

F

nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]

MST: PRIM



	1	2	3	4	5
nearest	-	1	1	1	1

	1	2	3	4	5
distance	-	10	4	∞	8

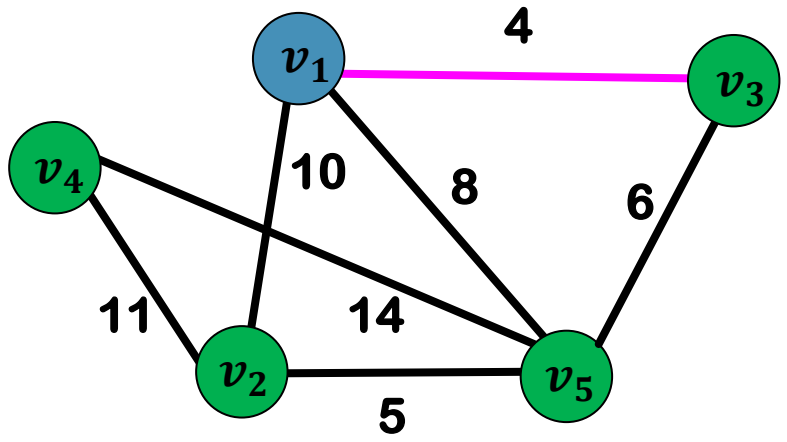
nearest[i]= index of the nearest node in Z to v_i

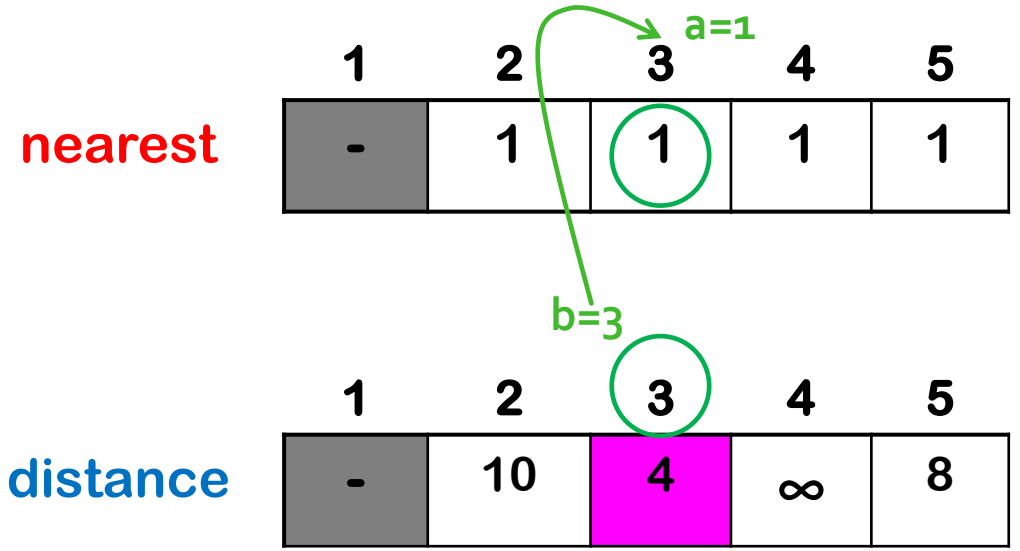
distance[i]= W[nearest[i]][v_i]

Z v_1

F

MST: PRIM

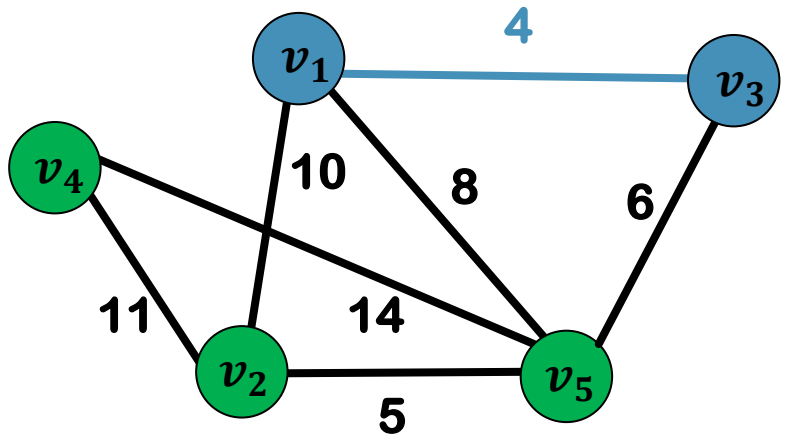


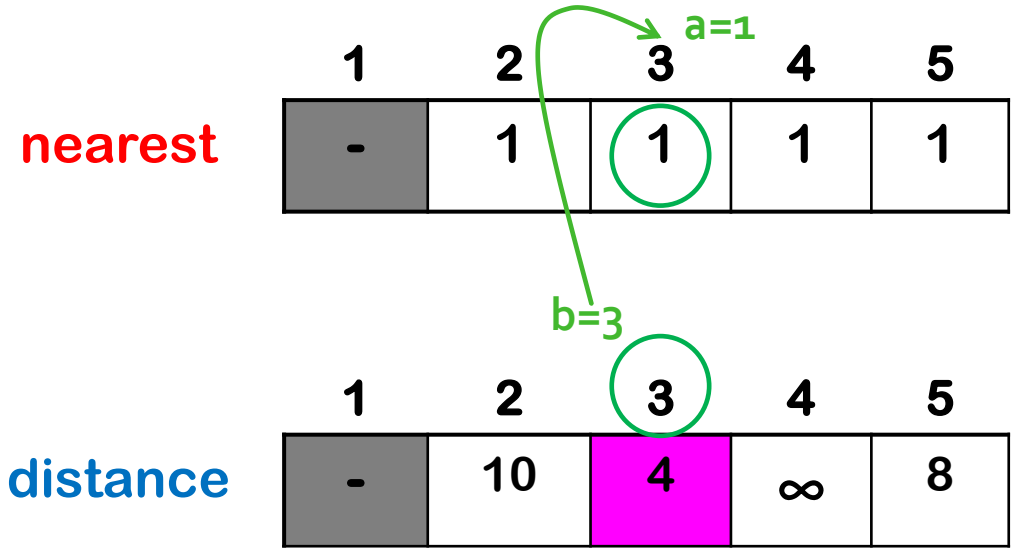


nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]

MST: PRIM





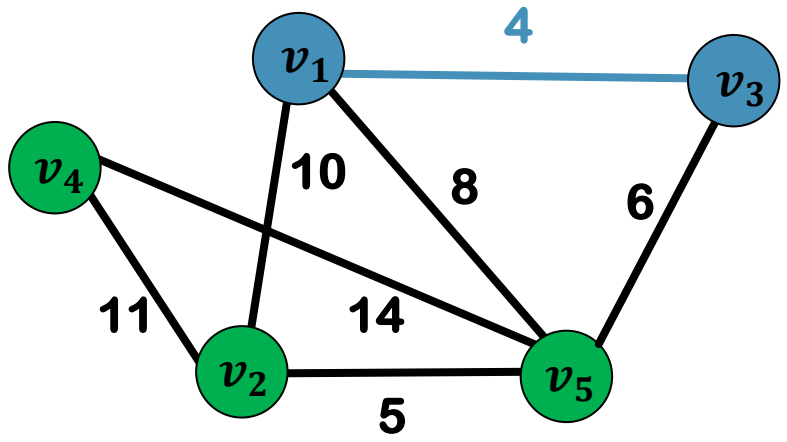
nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]

Z v_1 , v_3

F (1,3)

MST: PRIM



	1	2	3	4	5
nearest	-	1	1	1	1

	1	2	3	4	5
distance	-	10	4	∞	8

UPDATE!

Z	v_1 , v_3
F	(1,3)

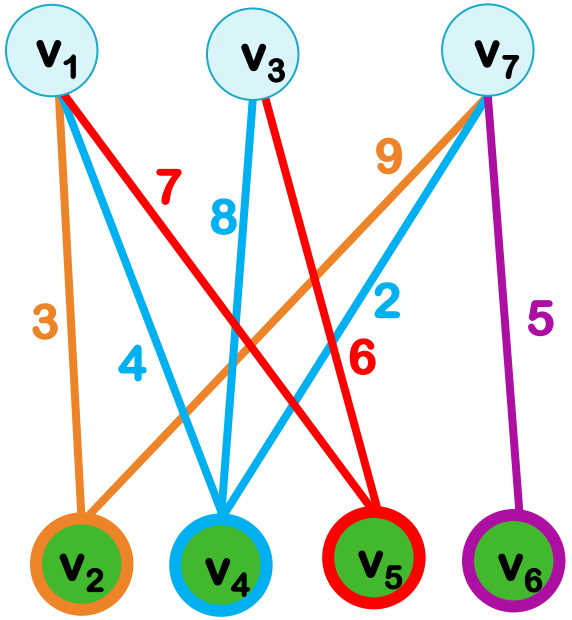
nearest[i]= index of the nearest node in Z to v_i

distance[i]= W[nearest[i]][v_i]

PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

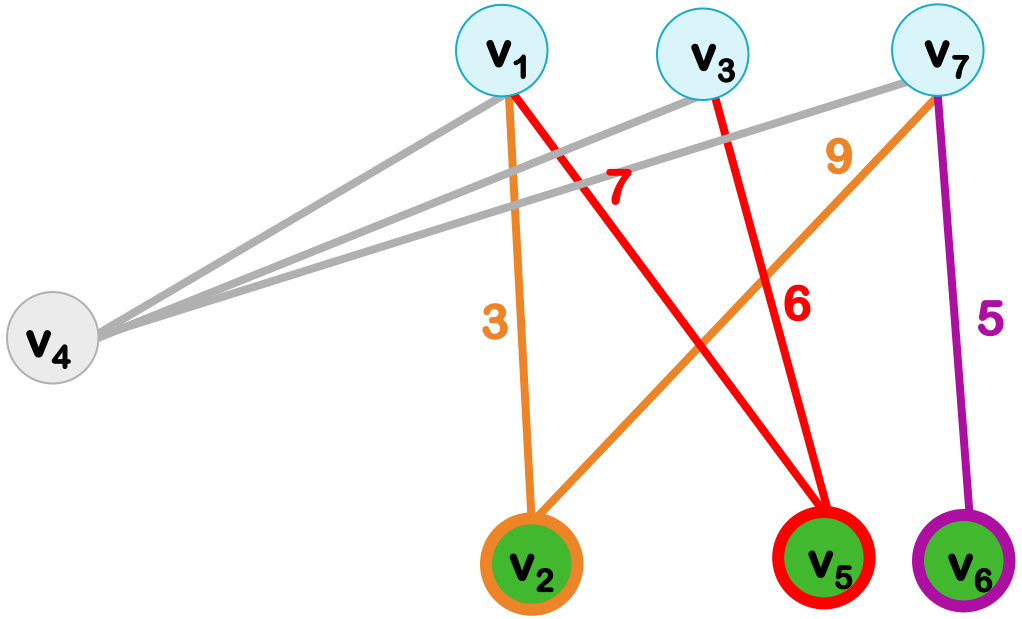
	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

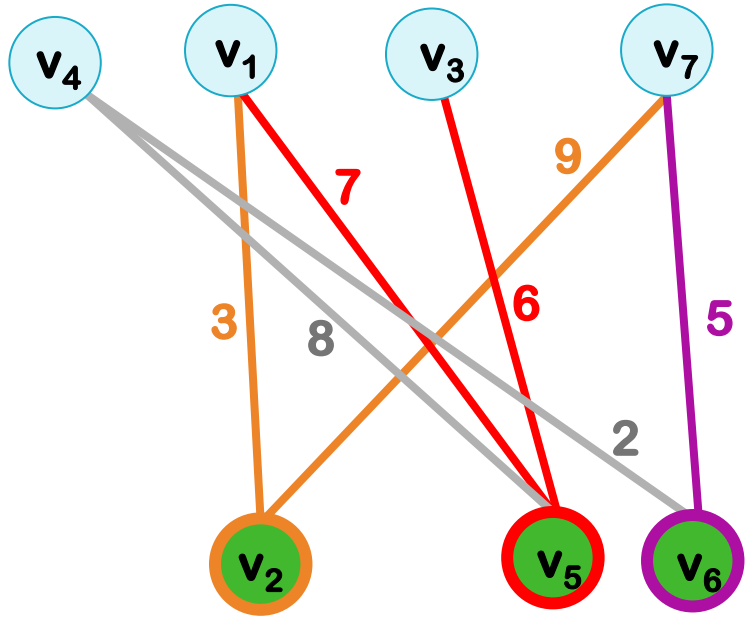
	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

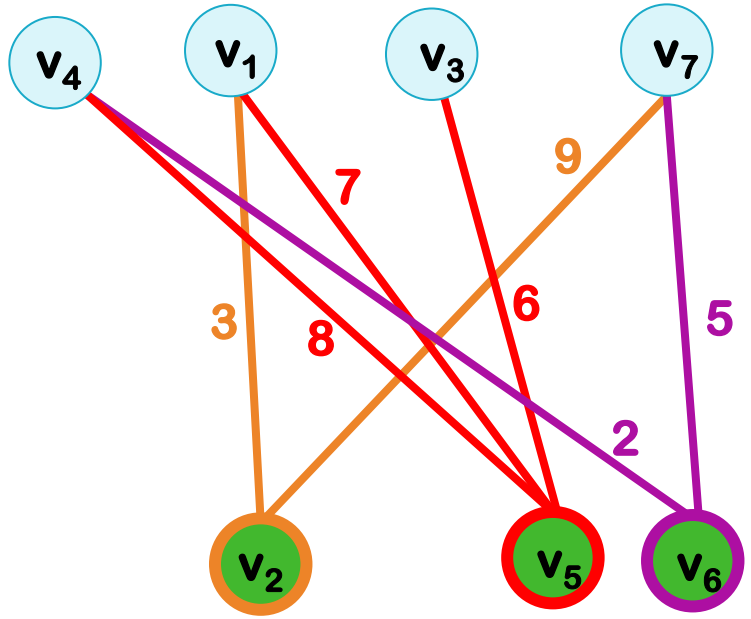
	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

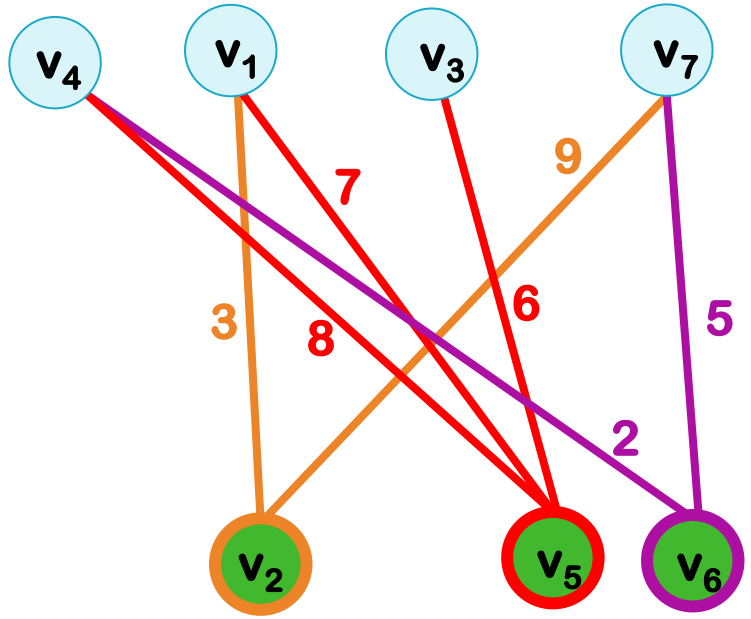
	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

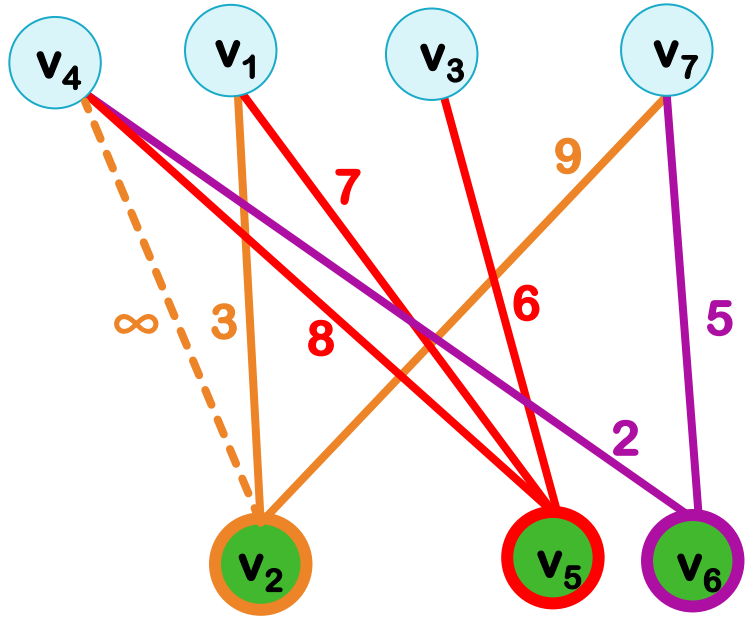
	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

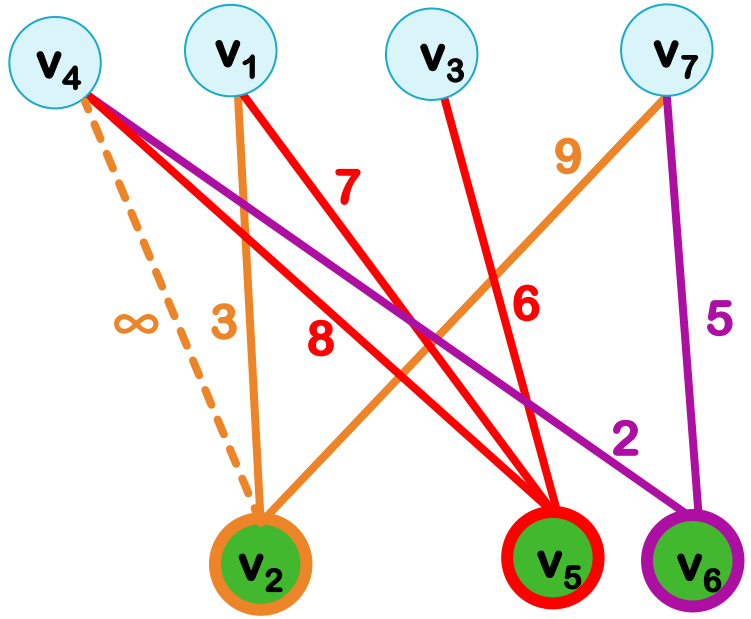
	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

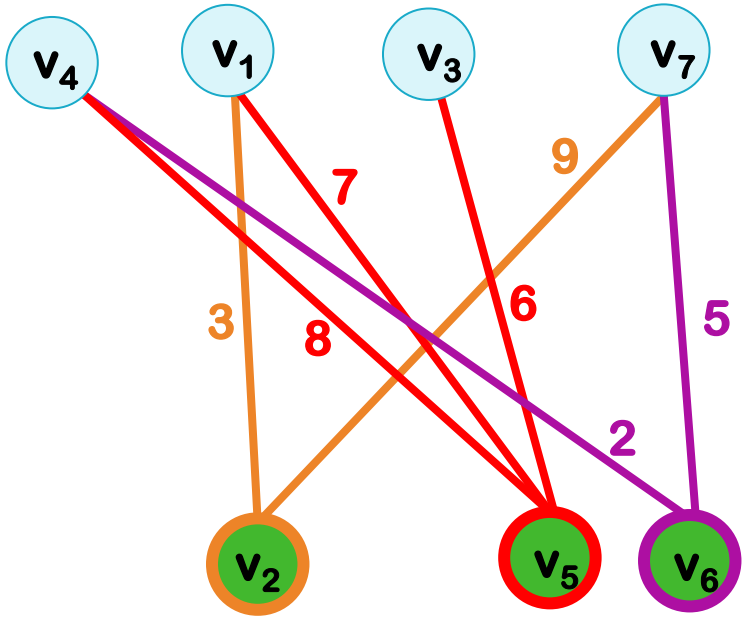
	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-

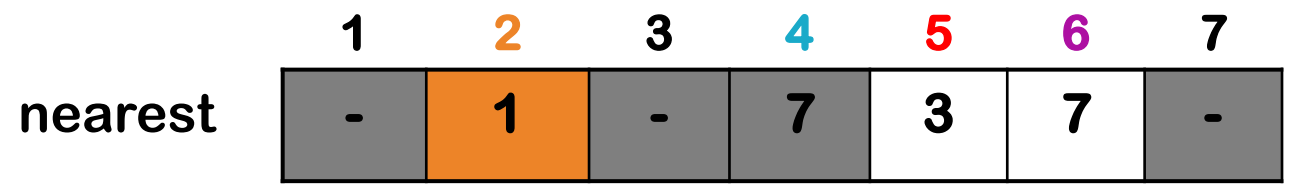


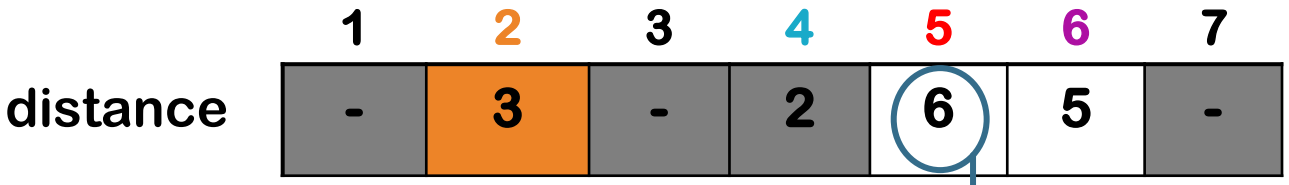
PRIM: UPDATING NEAREST[]

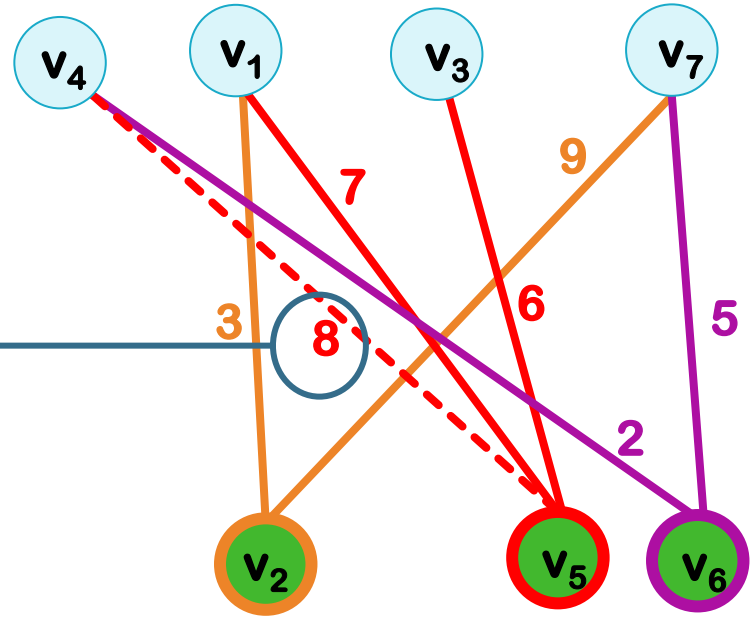
	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-
distance	-	3	-	2	6	5	-

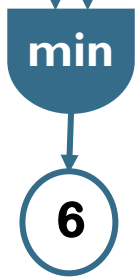


PRIM: UPDATING NEAREST[]







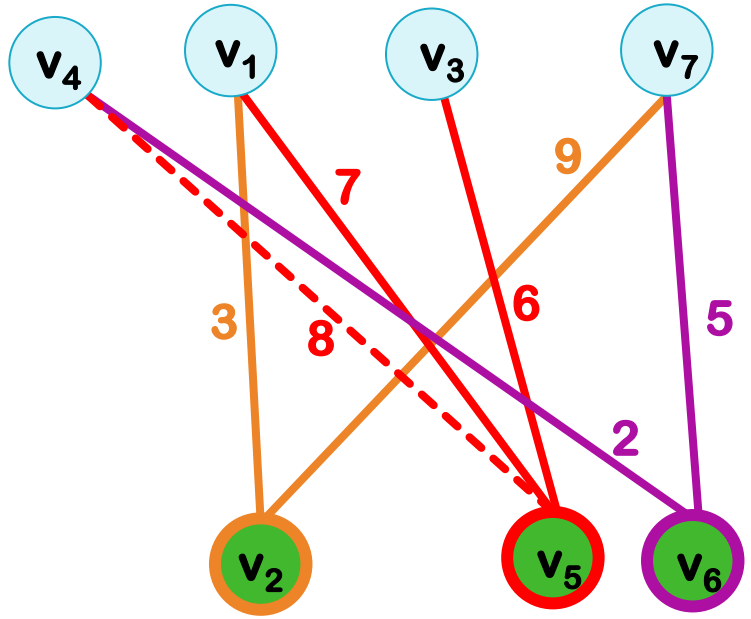


Min{distance[5],W[5][4]}

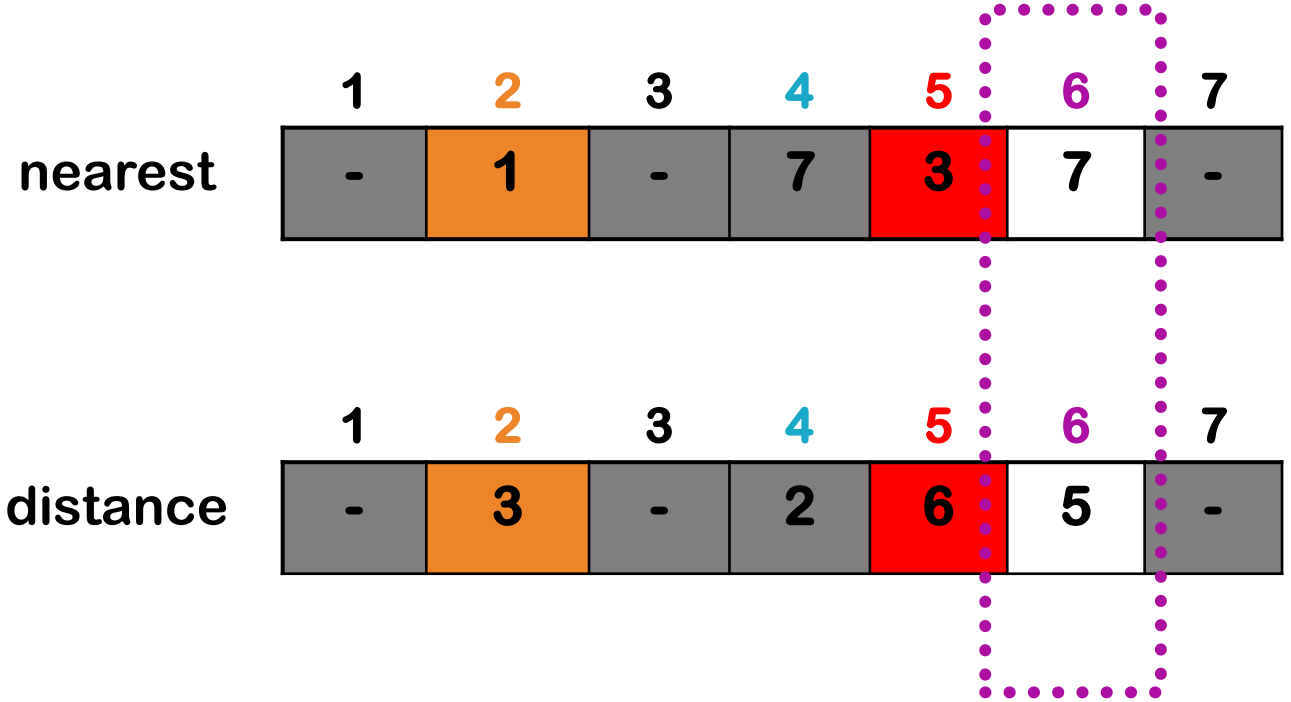
PRIM: UPDATING NEAREST[]

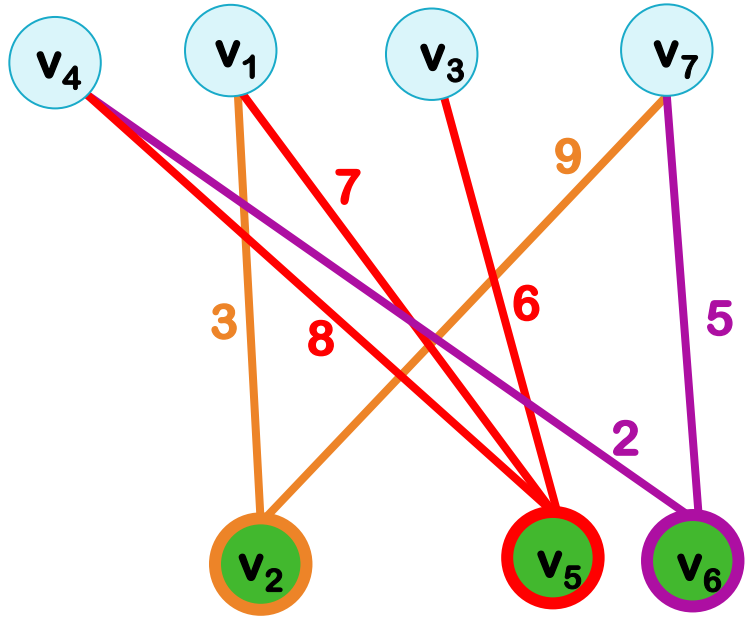
	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-



PRIM: UPDATING NEAREST[]

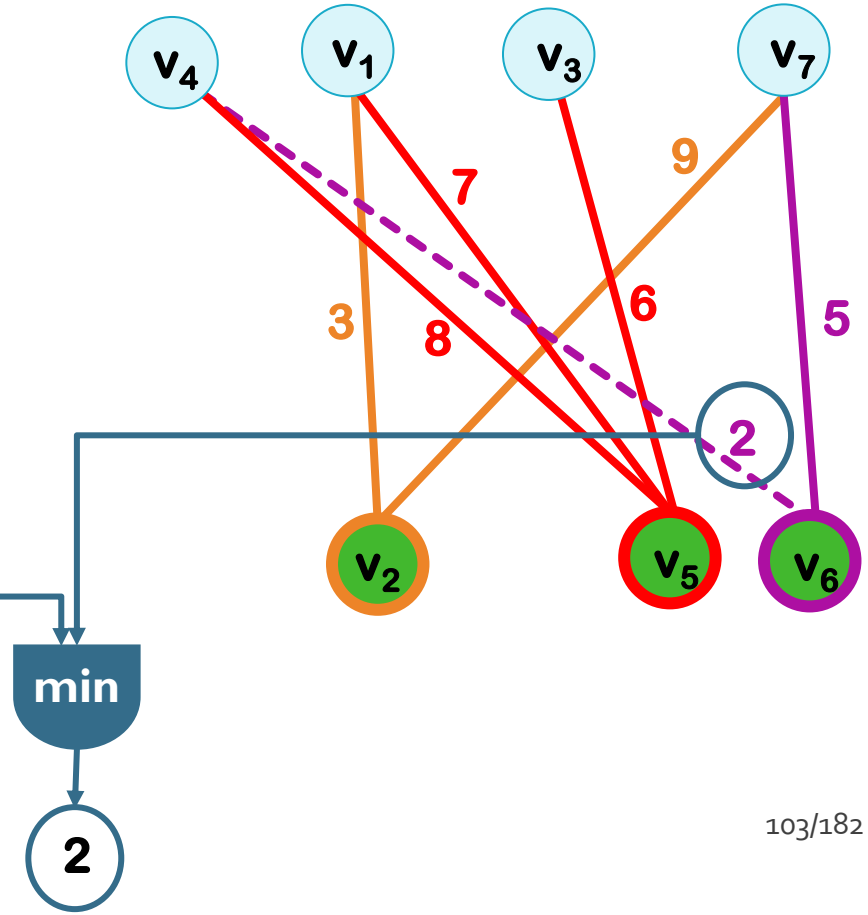




PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	5	-

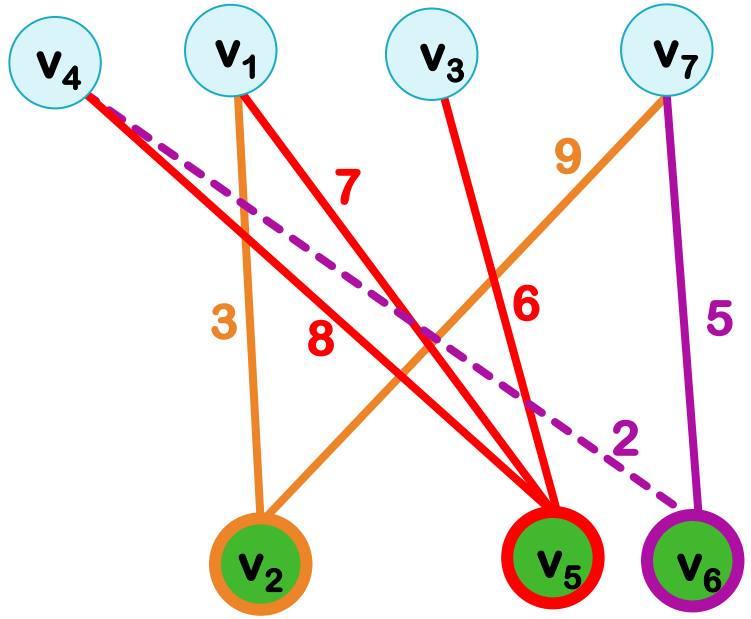


Min{distance[6],W[6][4]}

PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	7	-

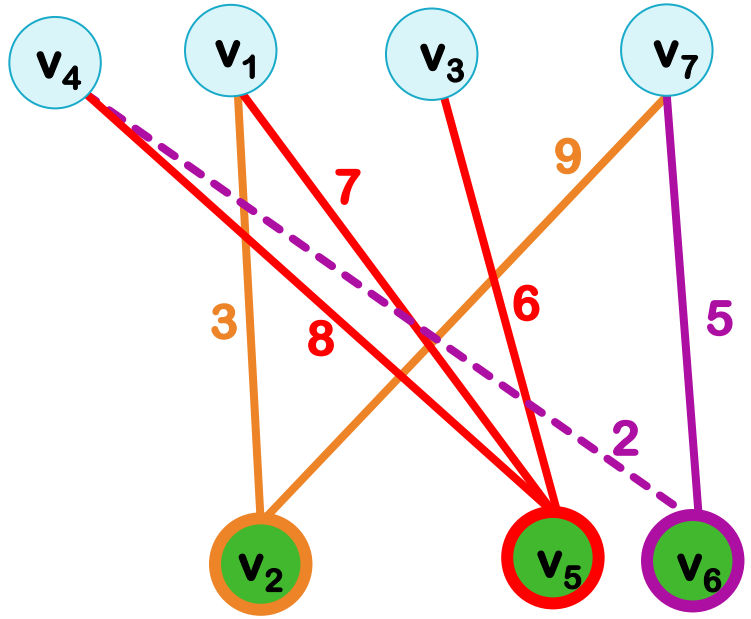
	1	2	3	4	5	6	7
distance	-	3	-	2	6	2	-



PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	4	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	2	-

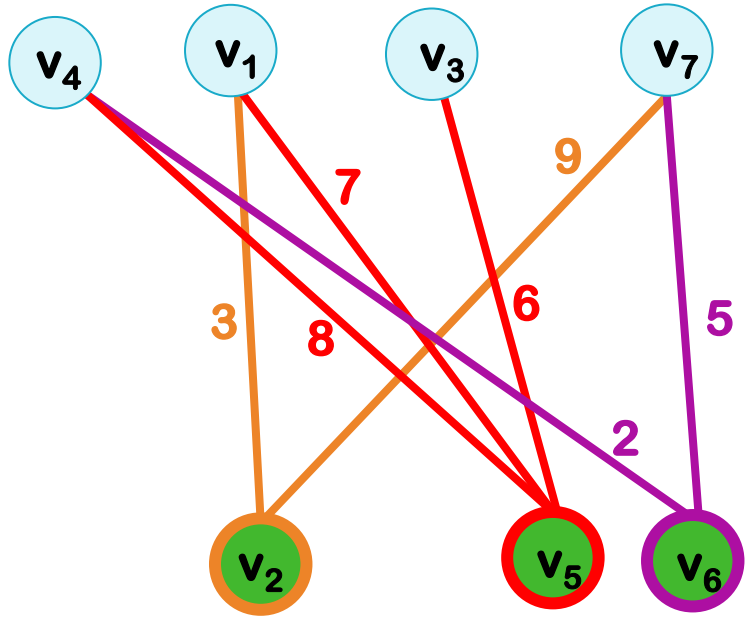


nearest[6]=4

PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	4	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	2	-

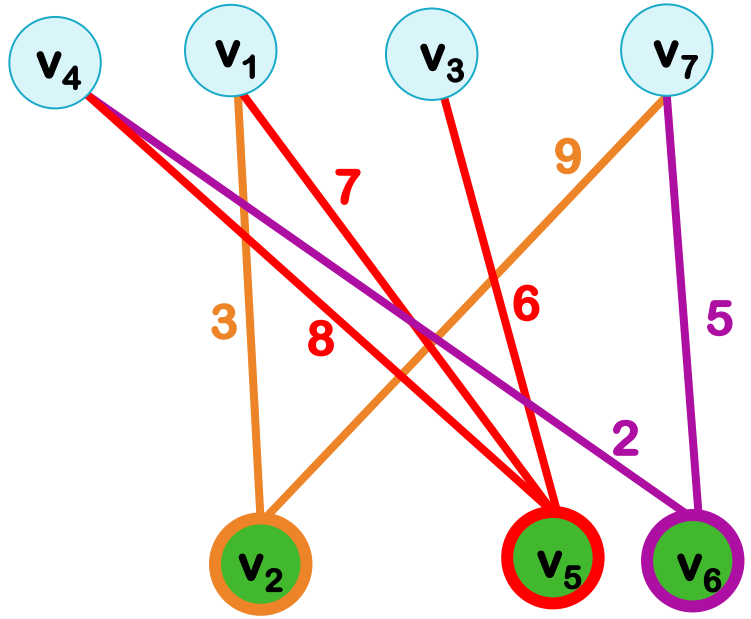


UPDATE: distance[i]=Min{distance[i],W[i][b]}

PRIM: UPDATING NEAREST[]

	1	2	3	4	5	6	7
nearest	-	1	-	7	3	4	-

	1	2	3	4	5	6	7
distance	-	3	-	2	6	2	-



UPDATE: IF W[i][b]< distance[i] THEN distance[i]=W[i][b]

PRIM'S ALGORITHM

```
Prim(W)
 F=∅
 FOR i=2 TO n DO // v1 is in Z
       nearest[i]=1  // Z has only one element (v1)
       distance[i]=W[1][i]
 FOR j=1 TO n-1 DO
        b=index of minimum element in distance
        a=nearest[b]
        F.add((a,b))
        distance.delete(b)
        nearest.delete(b)
         FOR EACH i in distance DO
            IF W[i][b]<distance[i] THEN
                     distance[i]=W[i][b]
                     nearest[i]=b
 RETURN F
```

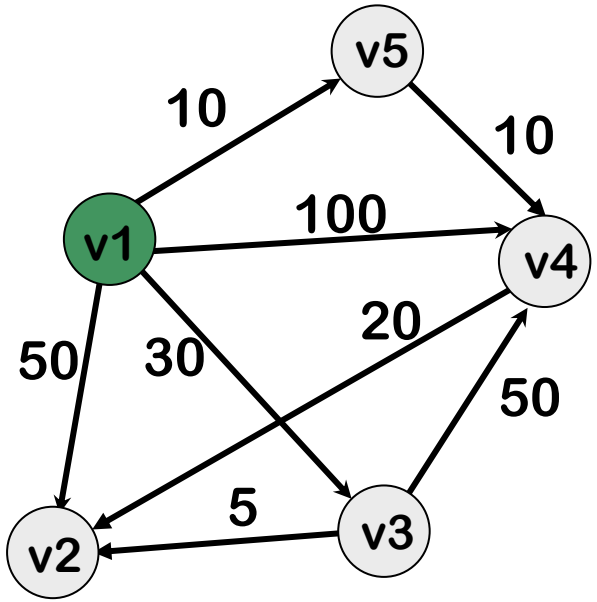
$O(n^2)$

PRIM'S ALGORITHM: WRAP UP

Implementation	Data Structure Used	Time Complexity	Notes
Naive implementation	Adjacency matrix + array	$O(V^2)$	Suitable for dense graphs.
Using a binary heap (priority queue)	Min-heap + adjacency list	$O(E \log V)$	Efficient for sparse graphs.
Using Fibonacci heap	Fibonacci heap + adjacency list	$O(E + V \log V)$	Theoretically optimal but complex to implement.

DIJKSTRA'S ALGORITHM

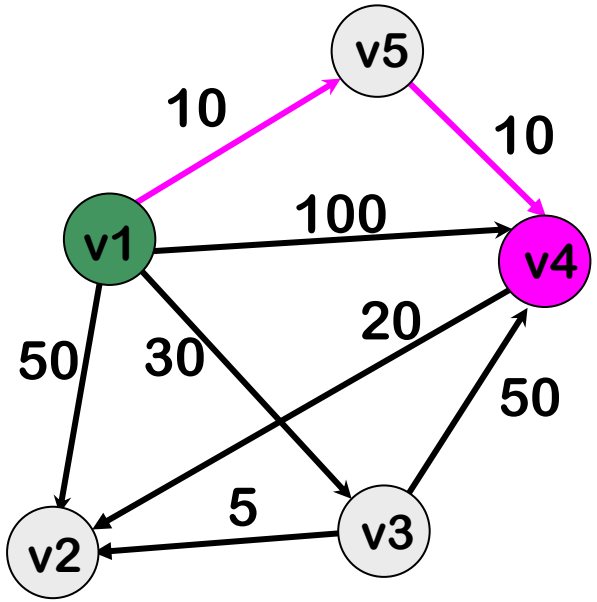
- Given a weighted directed graph, find the shortest path from v_1 to the other nodes.



G=(V,E)

DIJKSTRA'S ALGORITHM

- Given a weighted directed graph, find the shortest path from v_1 to the other nodes.

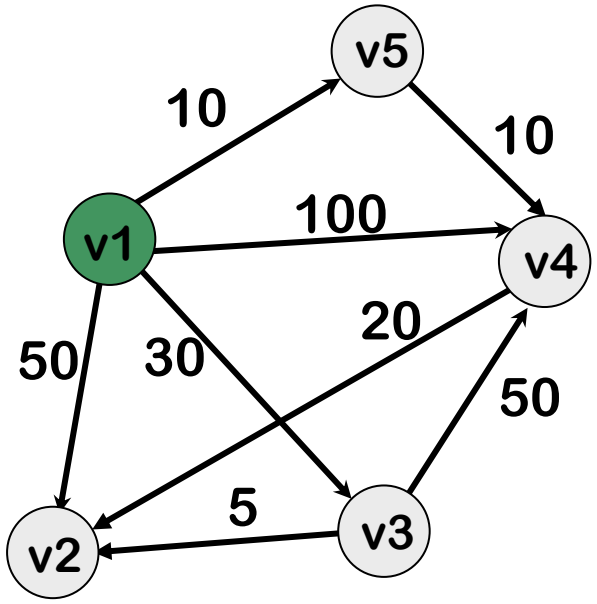


G=(V,E)

$$SP(v_1, v_4) = (v_1, v_5, v_4)$$

DIJKSTRA'S ALGORITHM

- Given a weighted directed graph, find the shortest path from v_1 to the other nodes.



G=(V,E)

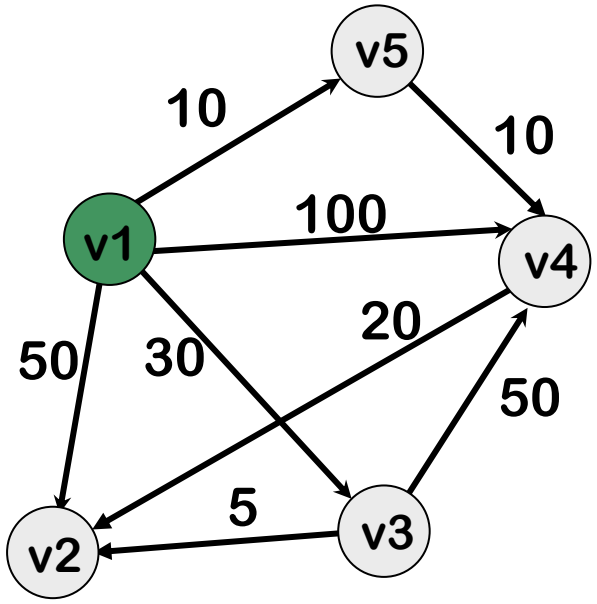
Output

four shortest paths:

$SP(v_1, v_2), SP(v_1, v_3), SP(v_1, v_4), SP(v_1, v_5)$

DIJKSTRA'S ALGORITHM

- Given a weighted directed graph, find the shortest path from v_1 to the other nodes.

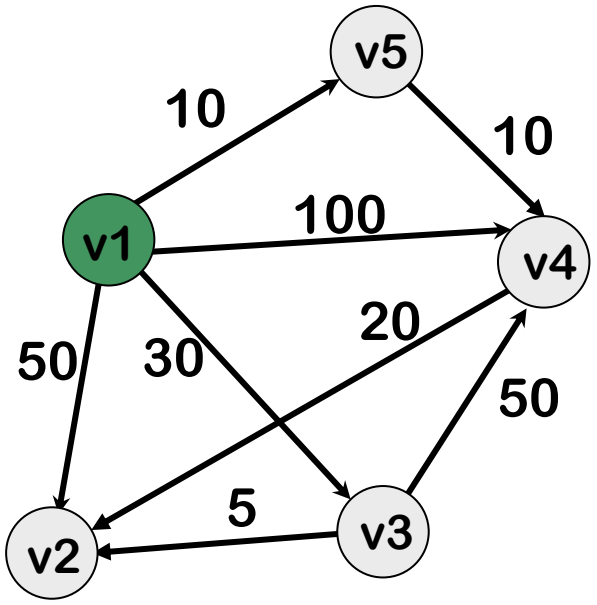


G=(V,E)

Output

Lengths of these four shortest paths:
$|SP(v_1, v_2)|, |SP(v_1, v_3)|, |SP(v_1, v_4)|, |SP(v_1, v_5)|$

GRAPH: REPRESENTATION



G

	v_1	v_2	v_3	v_4	v_5
v_1	0	50	30	100	10
v_2	∞	0	∞	∞	∞
v_3	∞	5	0	50	∞
v_4	∞	20	∞	0	∞
v_5	∞	∞	∞	10	0

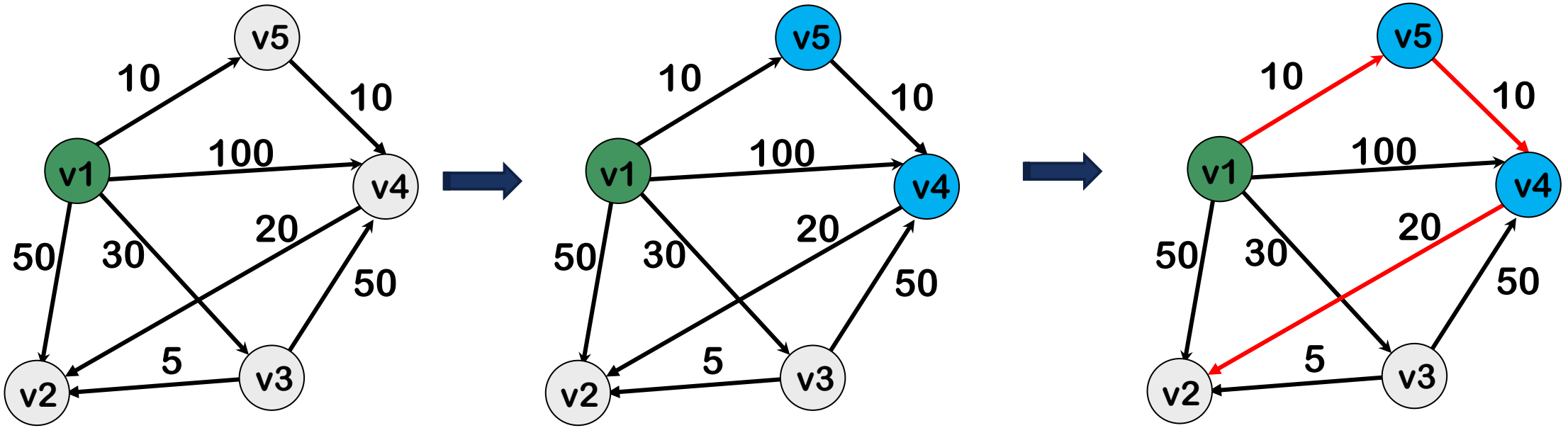
W

	v_1	v_2	v_3	v_4	v_5
v_1	0	35	30	20	10

D

NOTATION

$SP_S(v_1,v_i)$= shortest path from v_1 to v_i which passes through some vertices in S.



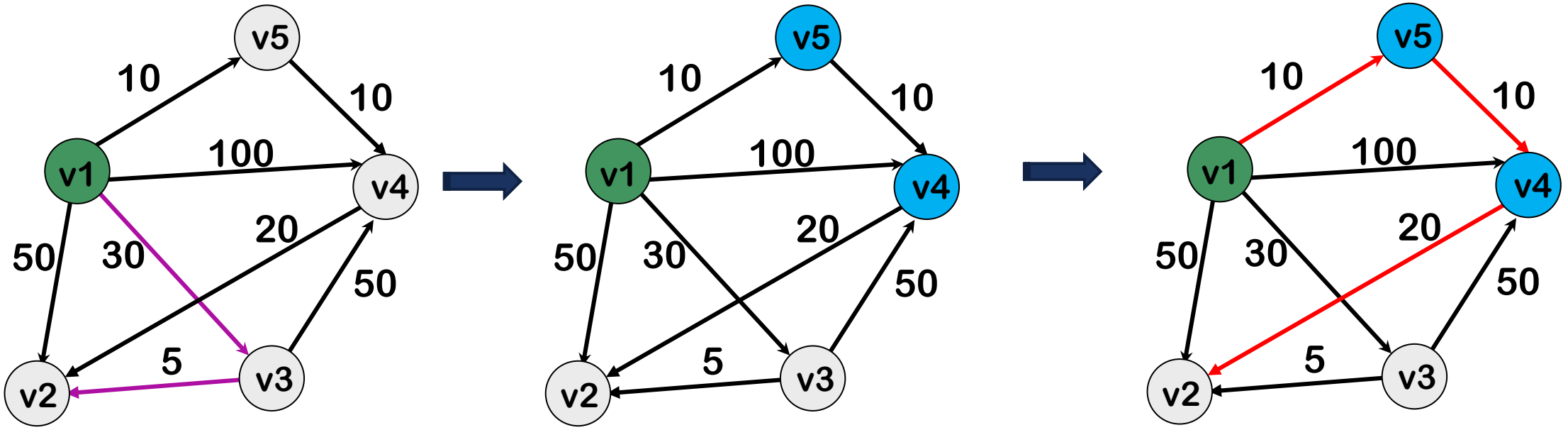
$S=\{v_4, v_5\}$

$SP_S(v_1,v_2)$=?

$SP_S(v_1, v_2)$

NOTATION

$SP_S(v_1,v_i)$= shortest path from v_1 to v_i which passes through some vertices in **S**.



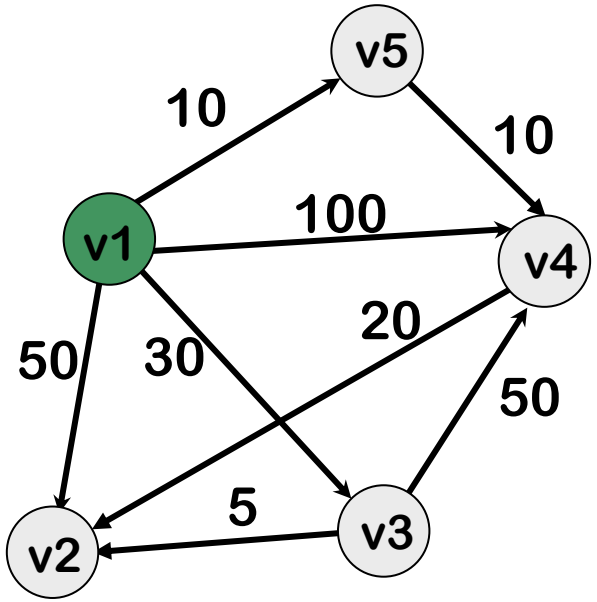
$SP(v_1, v_2)$

$S=\{v_4, v_5\}$

$SP_S(v_1,v_2)$=?

$SP_S(v_1, v_2)$

IDEA



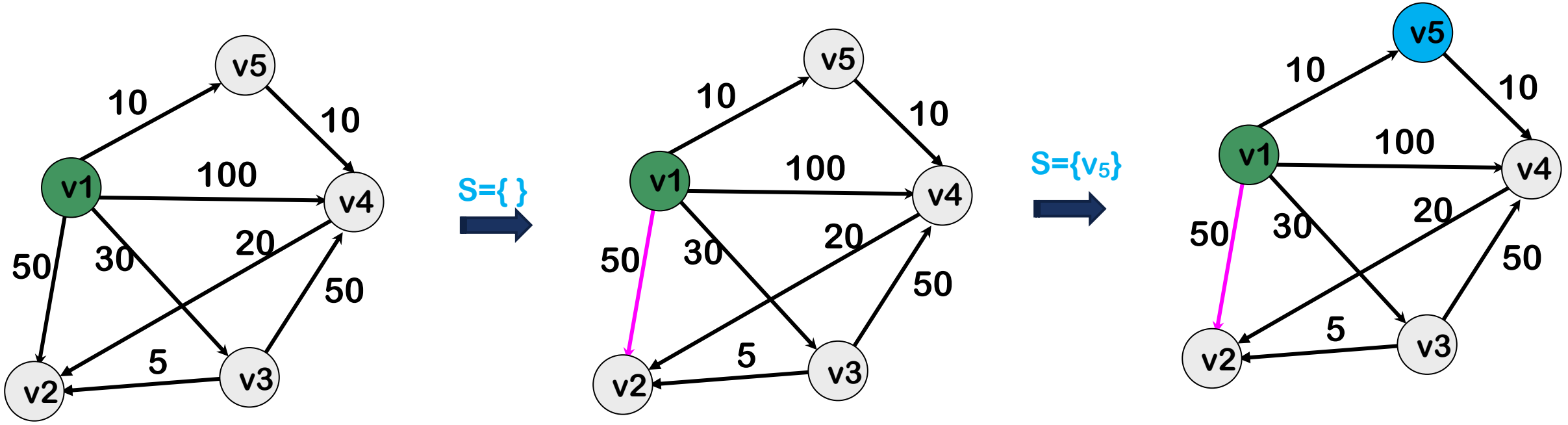
$SP_{\{\}}(v_1,v_2) \rightarrow SP_{\{v\}}(v_1,v_2) \rightarrow SP_{\{v,u\}}(v_1,v_2) \rightarrow$

$SP_{\{u,v,z\}}(v_1,v_2) \rightarrow SP_{\{v2,v3,v4,v5\}}(v_1,v_2) = SP(v_1,v_2)$

$SP(v_1,v_2)=?$

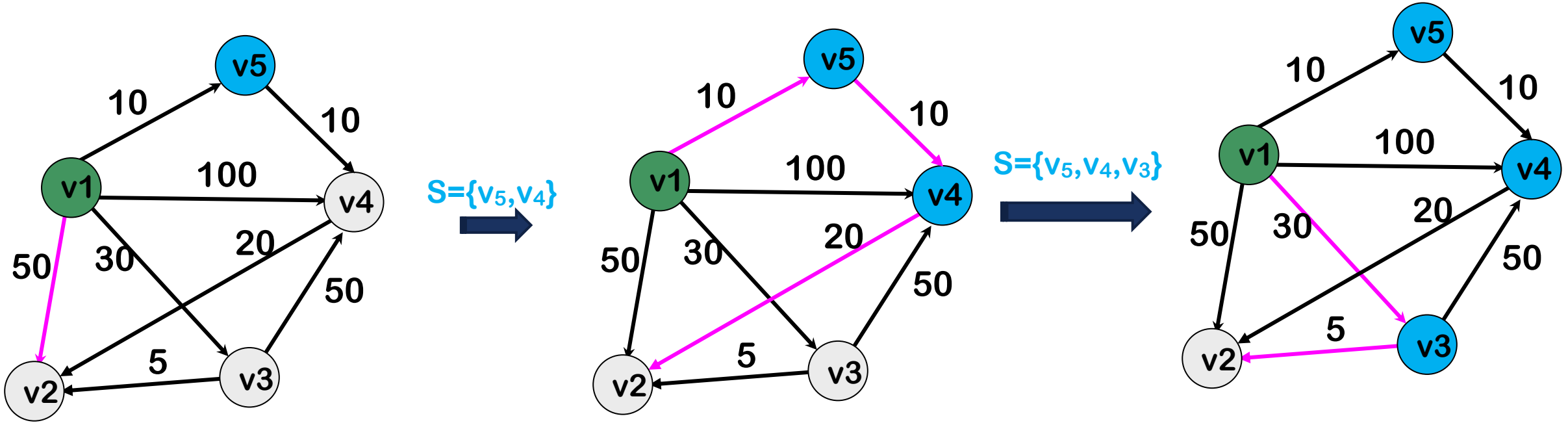
IDEA

SP(v_1,v_2)=?


v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3
S={ }
v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3
S={v5}
v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3

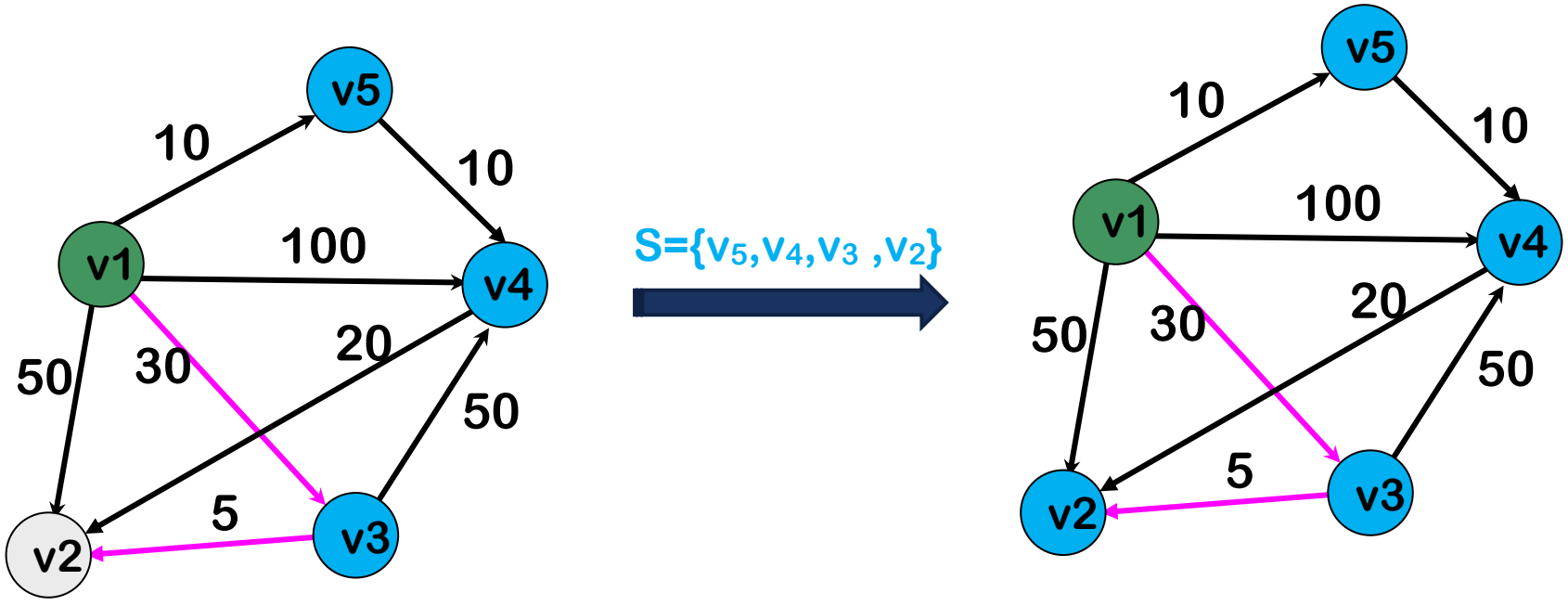
IDEA

SP(v_1,v_2)=?

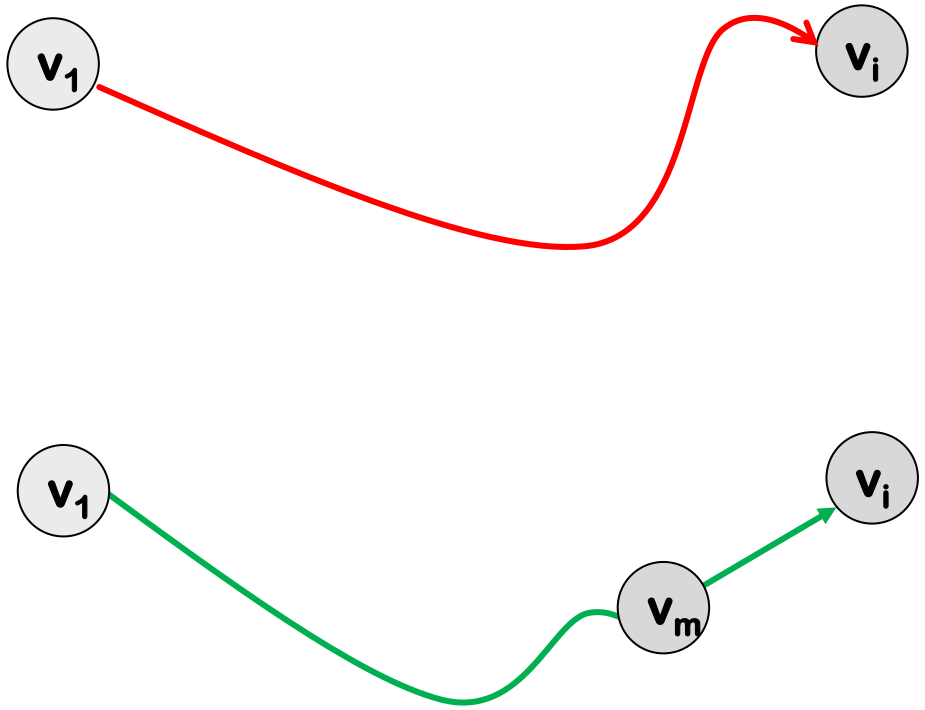

v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3
S={v5,v4}
v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3
S={v5,v4,v3}
v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3

IDEA

$SP(v_1,v_2)=?$

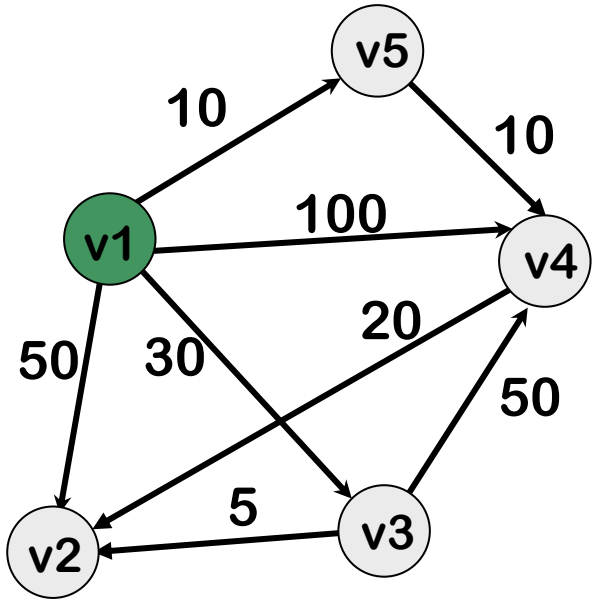

v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3
S={v5,v4,v3 ,v2}
v5
10
10
v1
100
v4
50
30
20
50
5
v2
v3

STRATEGY!



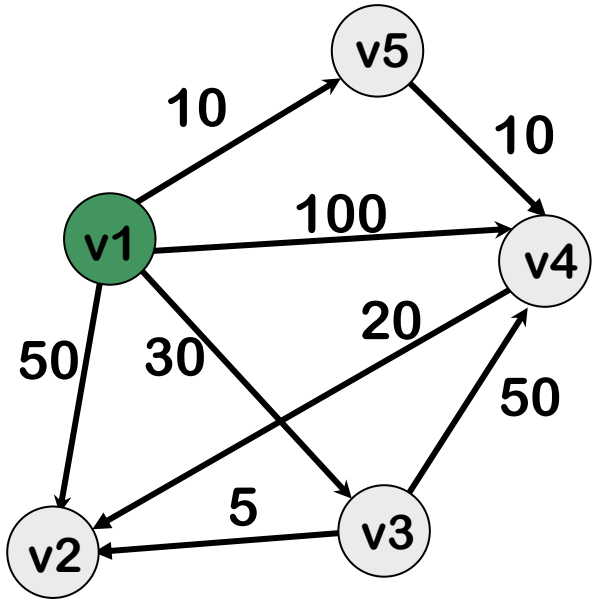
DIJKSTRA: EXAMPLE

S={ }



DIJKSTRA: EXAMPLE

$S=\{\ \}$



$|SP_S(v_1, v_2)|=50$

$|SP_S(v_1, v_3)|=30$

$|SP_S(v_1, v_4)|=100$

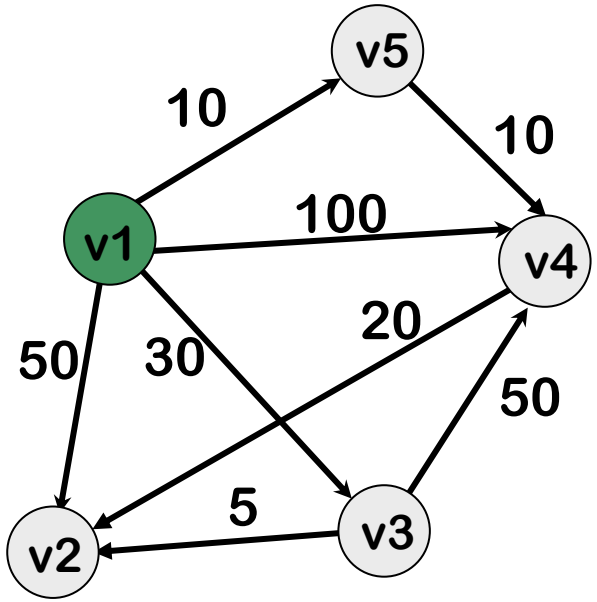
$|SP_S(v_1, v_5)|=10$

$|SP(v_1, v_5)|=10$

This path cannot become shorter!!

DIJKSTRA: EXAMPLE

$S=\{v_5\}$



$|SP_S(v_1, v_2)|=50$

$|SP_S(v_1, v_3)|=30$

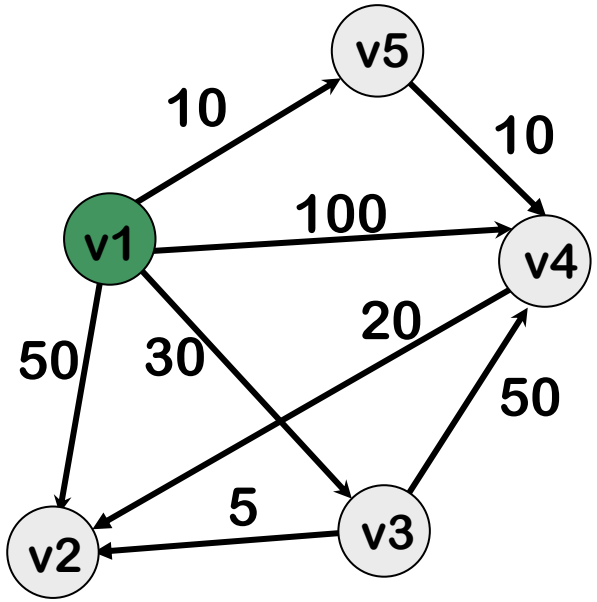
$|SP_S(v_1, v_4)|=100$

$|SP_S(v_1, v_5)|=10$

$S=\{v_5\}$

DIJKSTRA: EXAMPLE

$S=\{v_5\}$



|SPS (v1 , v2)|=50

|SPS (v1 , v3)|=30

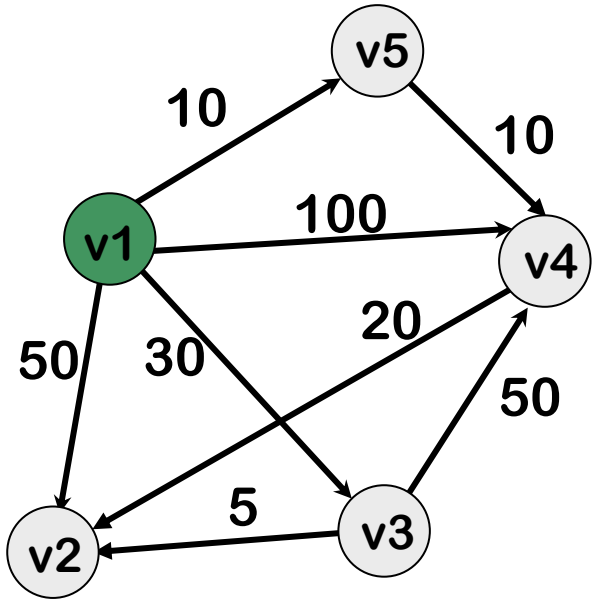
|SPS (v1 , v4)|=20

$|SP(v_1 , v_4)|=20$

This path cannot become shorter!!

DIJKSTRA: EXAMPLE

$S=\{v_5, v_4\}$



|SPS (v1 , v2)|=50

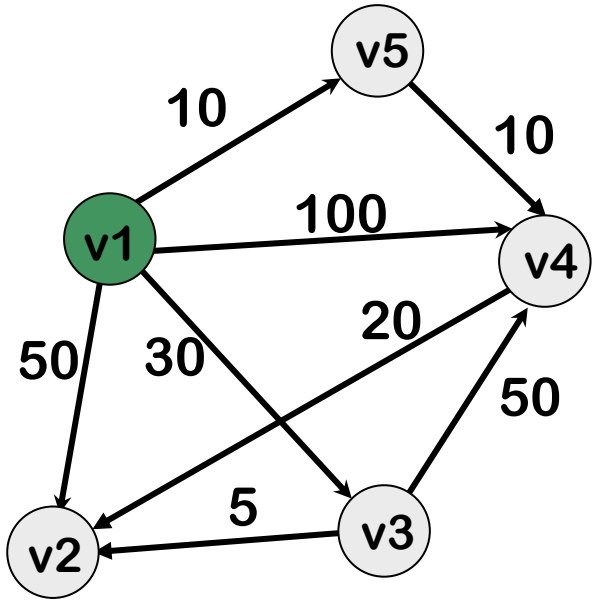
|SPS (v1 , v3)|=30

|SPS (v1 , v4)|=20

$S=\{v_5, v_4\}$

DIJKSTRA: EXAMPLE

$S=\{v_5, v_4\}$



$|SP_S(v_1, v_2)|=40$

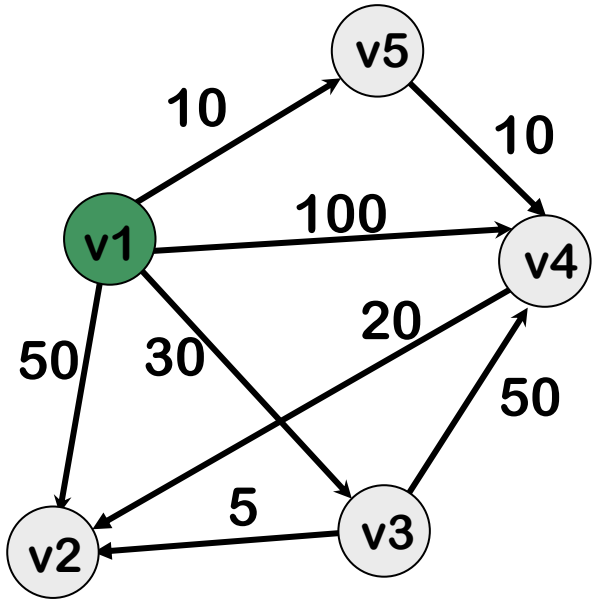
$|SP_S(v_1, v_3)|=30$

$|SP(v_1, v_3)|=30$

This path cannot become shorter!!

DIJKSTRA: EXAMPLE

$S=\{v_5, v_4, v_3\}$



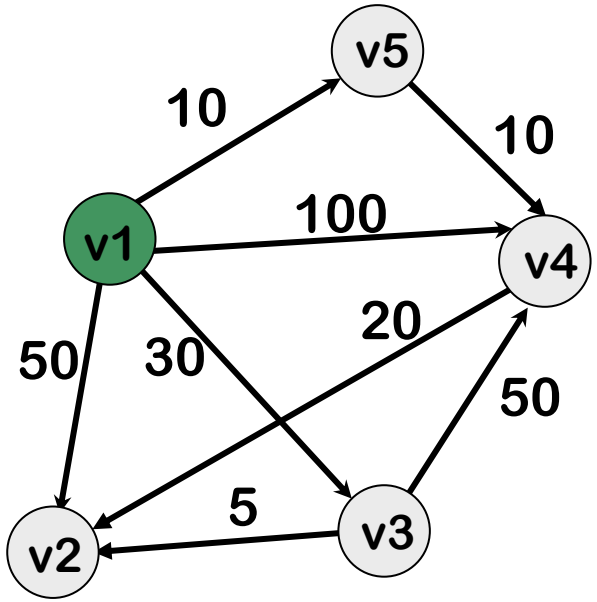
$|SP_S(v_1, v_2)|=40$

$|SP_S(v_1, v_3)|=30$

$S=\{v_5, v_4, v_3\}$

DIJKSTRA: EXAMPLE

$S=\{v_5, v_4, v_3\}$



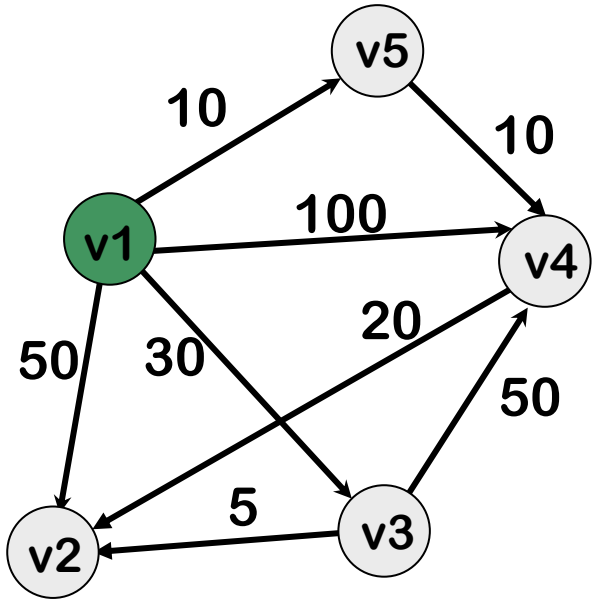
$|SP_S(v_1, v_2)|=35$

$|SP(v_1, v_2)|=35$

This path cannot become shorter!!

DIJKSTRA: EXAMPLE

$S=\{v_5, v_4, v_3, v_2\}$

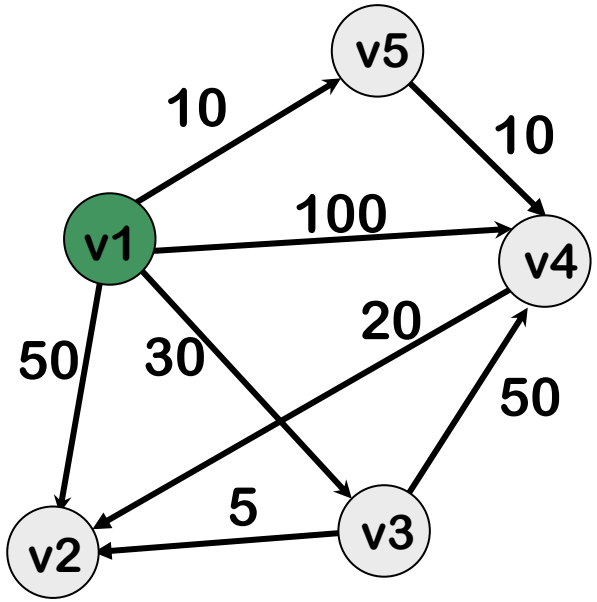


$|SP_S(v_1, v_2)|=35$

$S=\{v_5, v_4, v_3, v_2\}$

DIJKSTRA: IMPLEMENTATION

$S=\{\ \}$



$|SP_S(v_1, v_2)|=50$

$|SP_S(v_1, v_3)|=30$

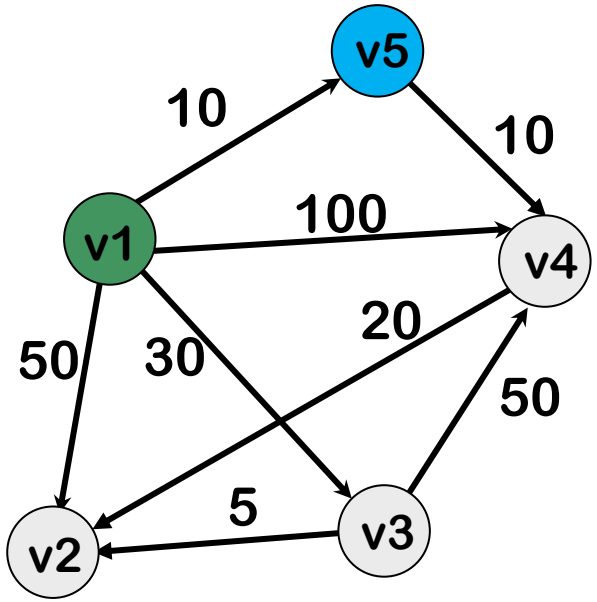
$|SP_S(v_1, v_4)|=100$

$|SP_S(v_1, v_5)|=10$

	v_2	v_3	v_4	~~v_5~~
D	50	30	100	10

DIJKSTRA: IMPLEMENTATION

$S=\{v_5\}$



$|SP_S(v_1, v_2)|=?$

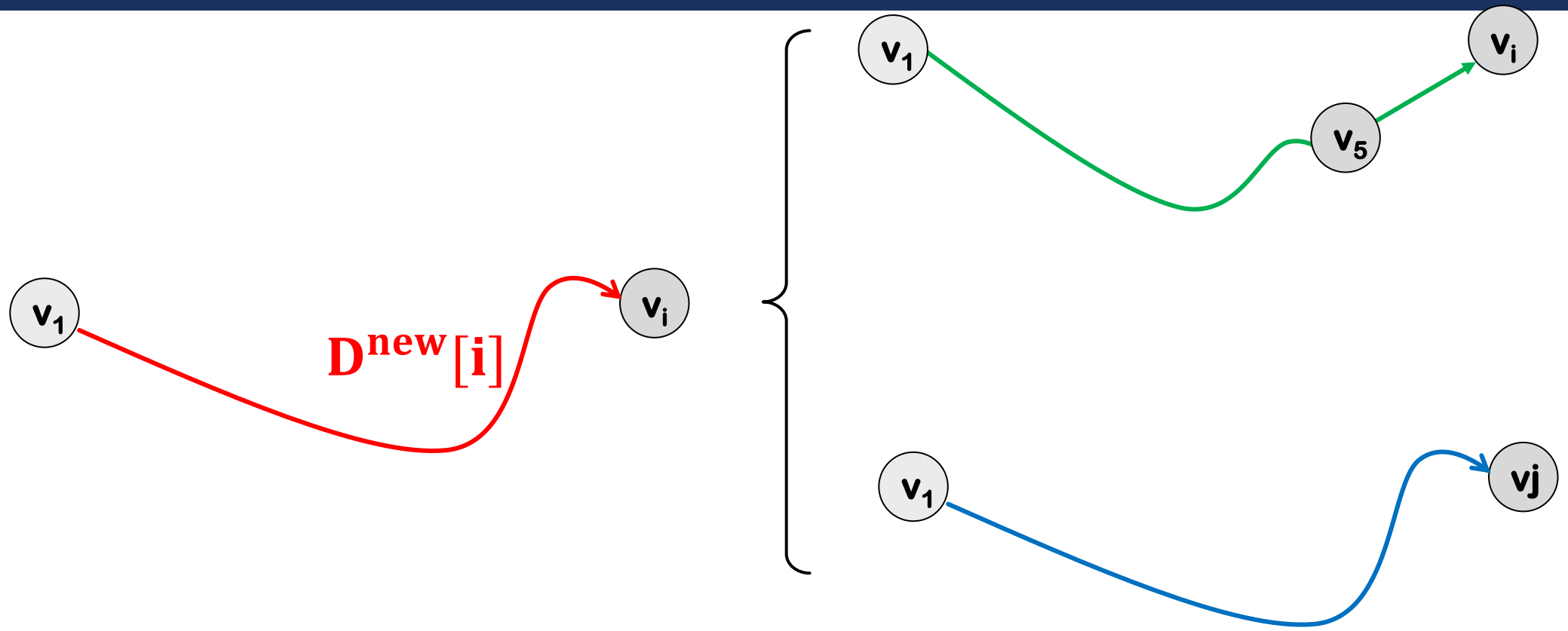
$|SP_S(v_1, v_3)|=?$

$|SP_S(v_1, v_4)|=?$

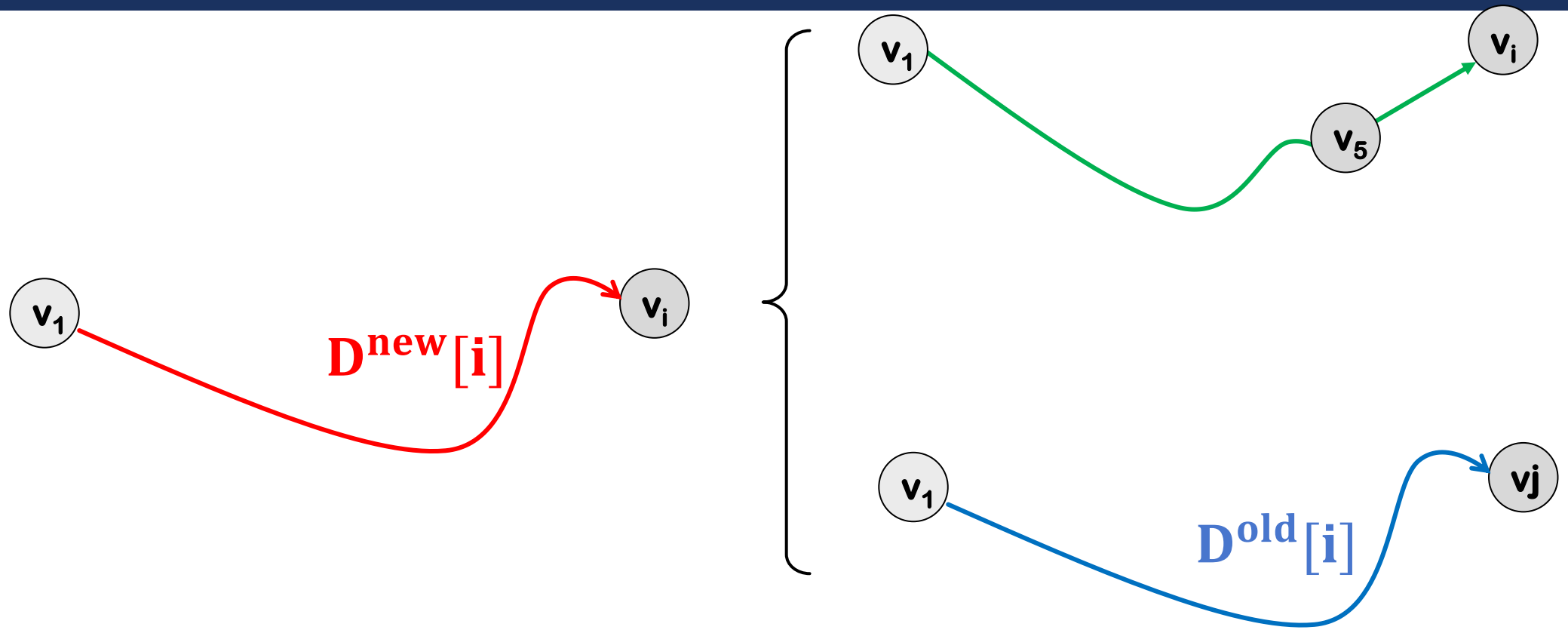
	v_2	v_3	v_4	-
D	?	?	?	-

UPDATE!

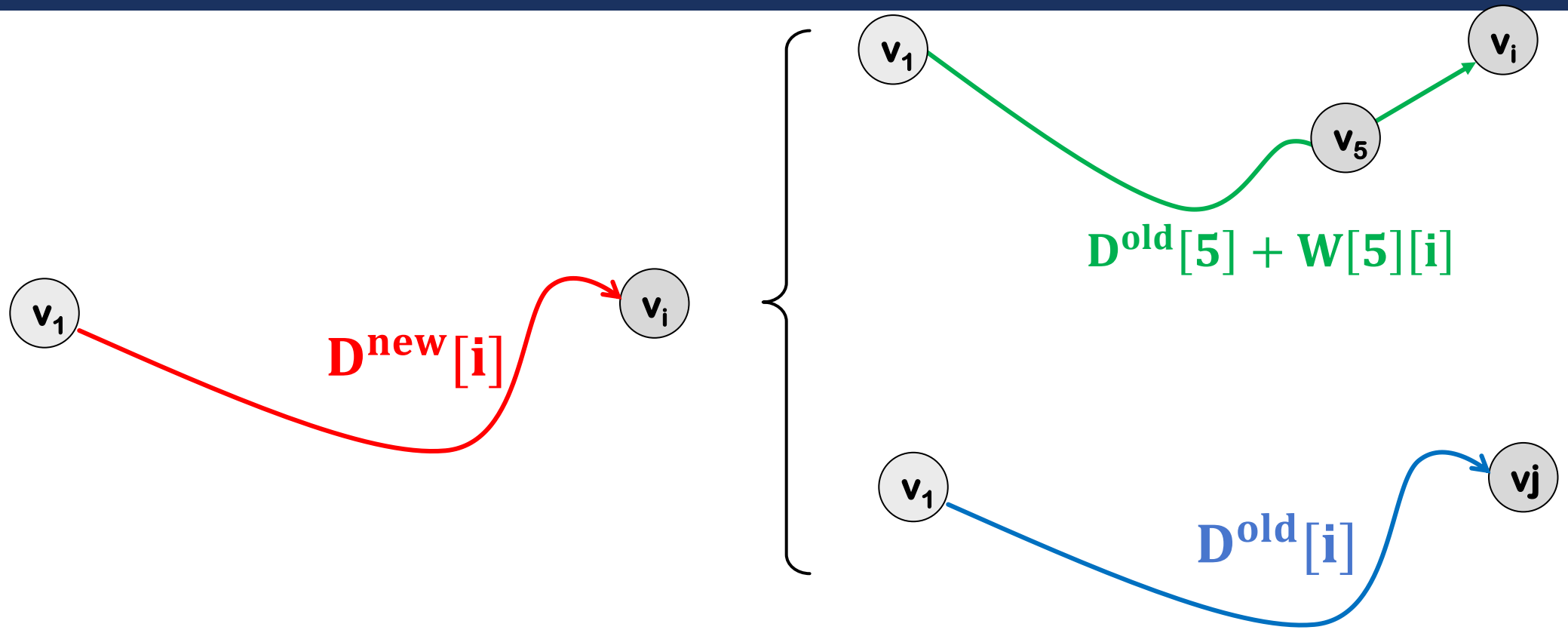
CALCULATING D[ ]



CALCULATING D[ ]

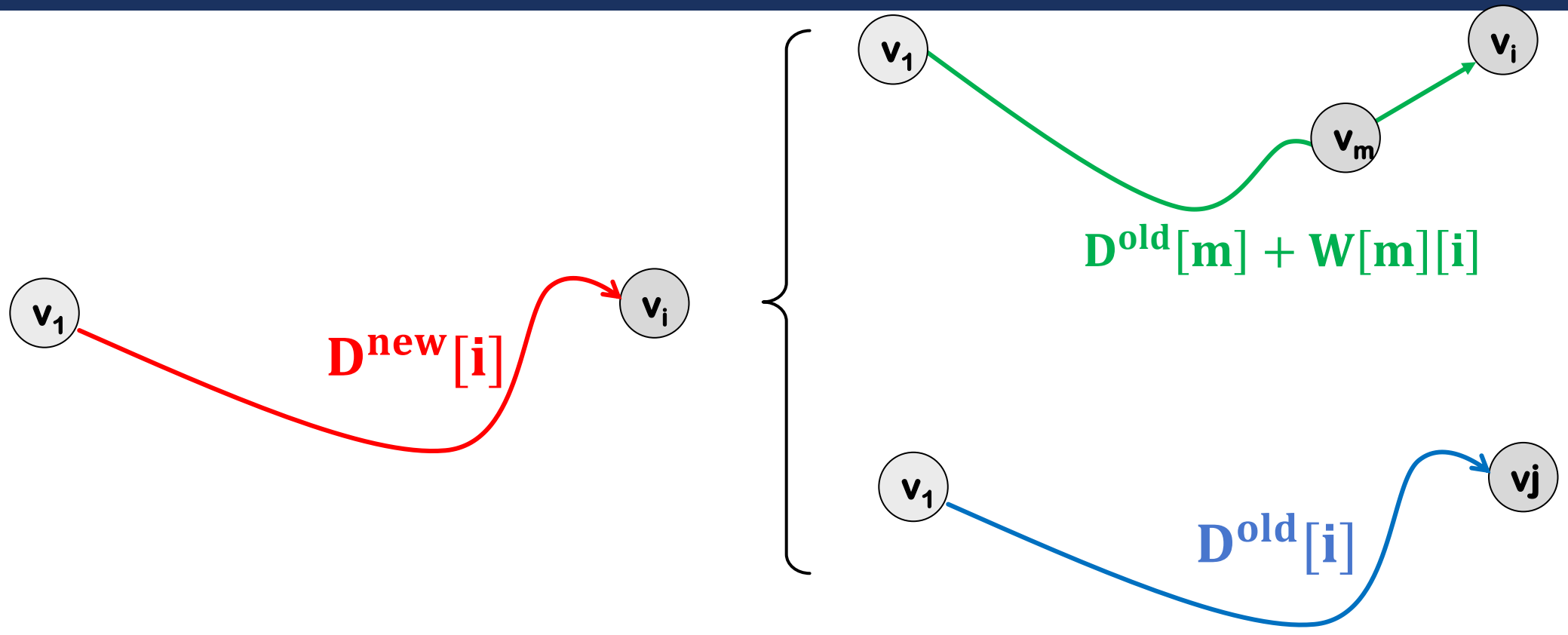


CALCULATING D[ ]



$$D^{new}[i] = \min\{D^{old}[5] + W[5][i], D^{old}[i]\}$$

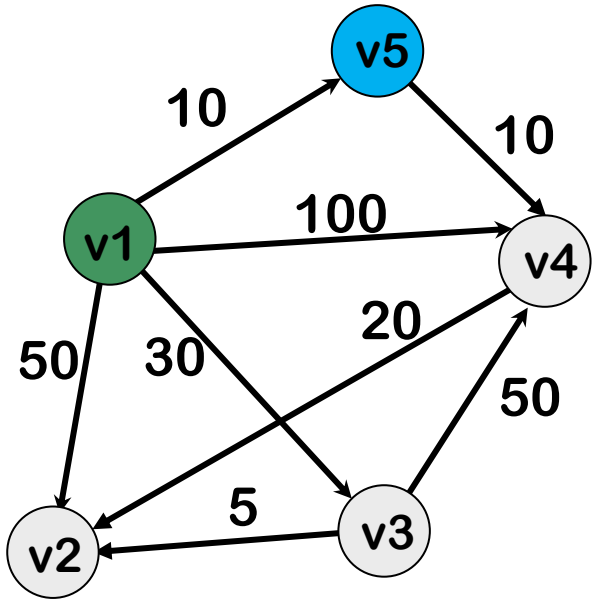
CALCULATING D[ ]



$$D^{new}[i] = \min\{D^{old}[m] + W[m][i], D^{old}[i]\}$$

DIJKSTRA: UPDATING D

$S=\{v_5\}$



D[2]=?

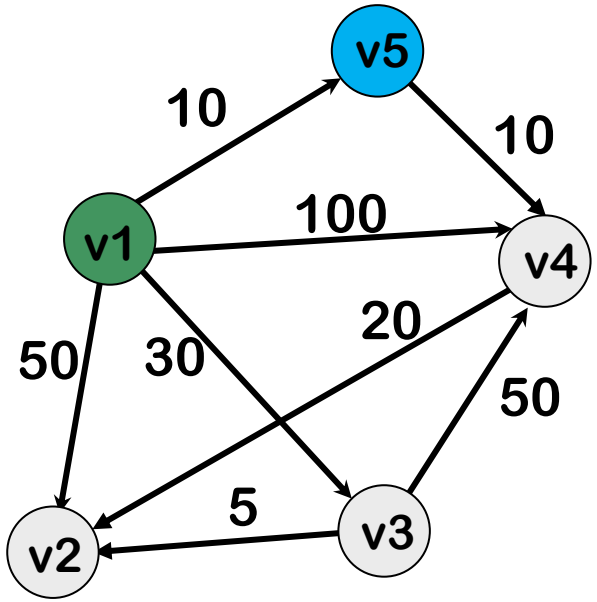
D[3]=?

D[4]=?

UPDATE!

	v_2	v_3	v_4	-
D	?	?	?	-

DIJKSTRA: UPDATING D



m=5

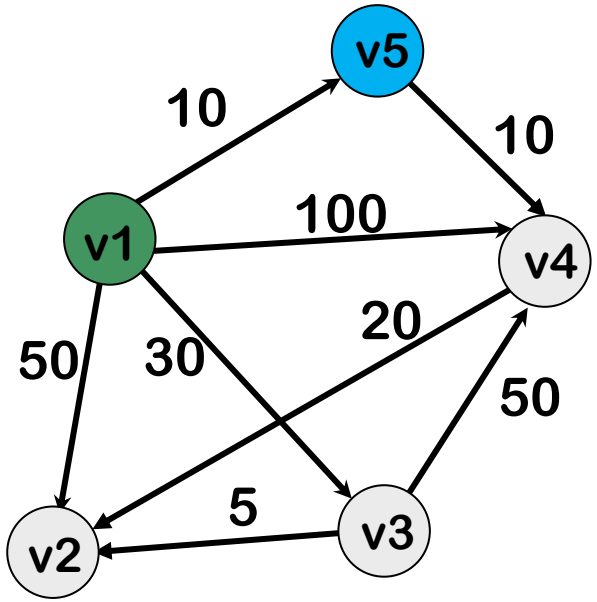
D[2]=min{D[2],D[5]+W[5][2]}=50

D[3]=min{D[3],D[5]+W[5][3]}=30

D[4]=min{D[4],D[5]+W[5][4]}=20

	v_2	v_3	v_4	-
D	?	?	?	-

DIJKSTRA: UPDATING D



m=5

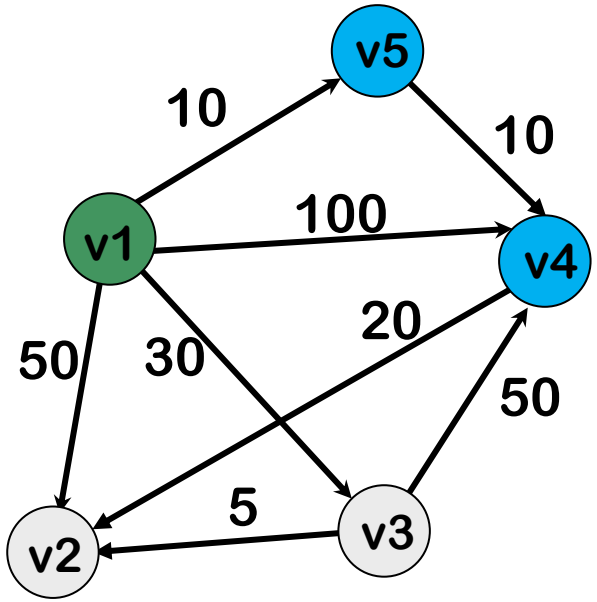
D[2]=min{D[2],D[5]+W[5][2]}=50

D[3]=min{D[3],D[5]+W[5][3]}=30

D[4]=min{D[4],D[5]+W[5][4]}=20

	v_2	v_3	~~v_4~~	-
D	50	30	20	-

DIJKSTRA: UPDATING D



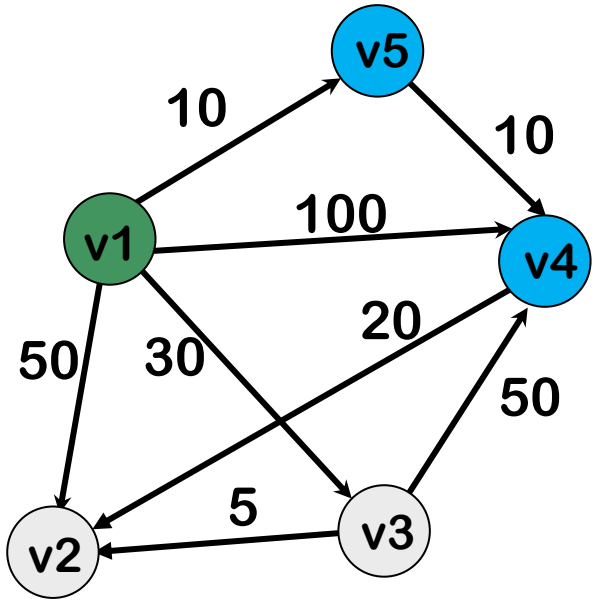
m=4

D[2]=?

D[3]=?

	v_2	v_3	-	-
D	**50**	**30**	**-**	**-**

DIJKSTRA: UPDATING D



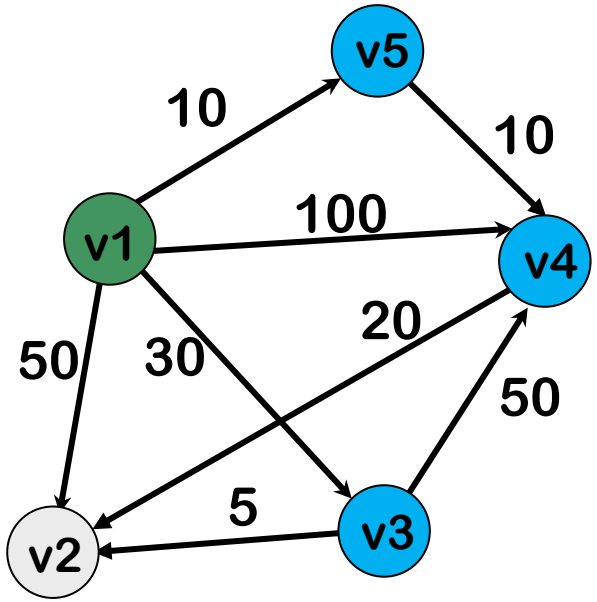
m=4

D[2]=min{D[2],D[4]+W[4][2]}=40

D[3]=min{D[3],D[4]+W[4][3]}=30

	v_2	~~v_3~~	-	-
D	40	30	-	-

DIJKSTRA: UPDATING D

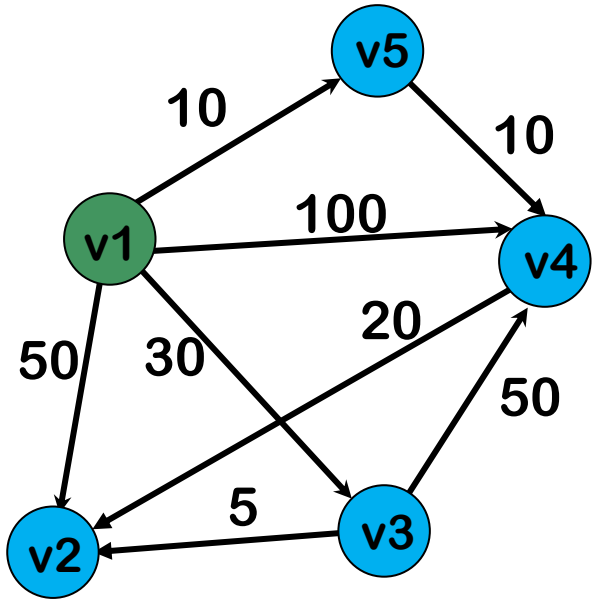


m=3

D[2]=min{D[2],D[3]+W[3][2]}=35

	v2	-	-	-
D	35	-	-	-

DIJKSTRA: UPDATING D



m=2

	-	-	-	-
D	-	-	-	-

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
           D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
           D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
           D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
           D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
            D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

$$T(n) = O(n) + (n-1)\big(O(n) + O(n)\big) = O(n^2)$$

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
            D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
            D[j]=min(D[m]+W[m][j],D[j])
        PRINT D[m]
        D.delete(m)
```

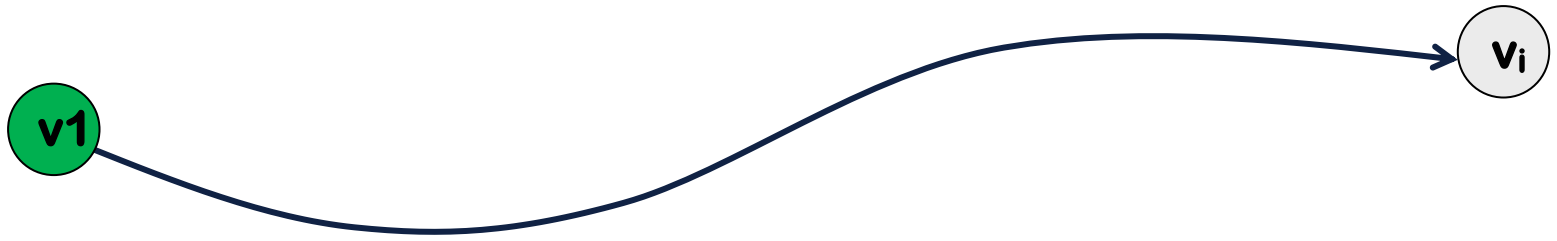
DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
            IF D[m]+W[m][j]<D[j] THEN
                D[j]=D[m]+W[m][j]
        PRINT D[m]
        D.delete(m)
```

DIJKSTRA: PATH MATRIX

For i=2,3,..,n: P[i]=index of the vertex right before i on the shortest path from 1 to i:

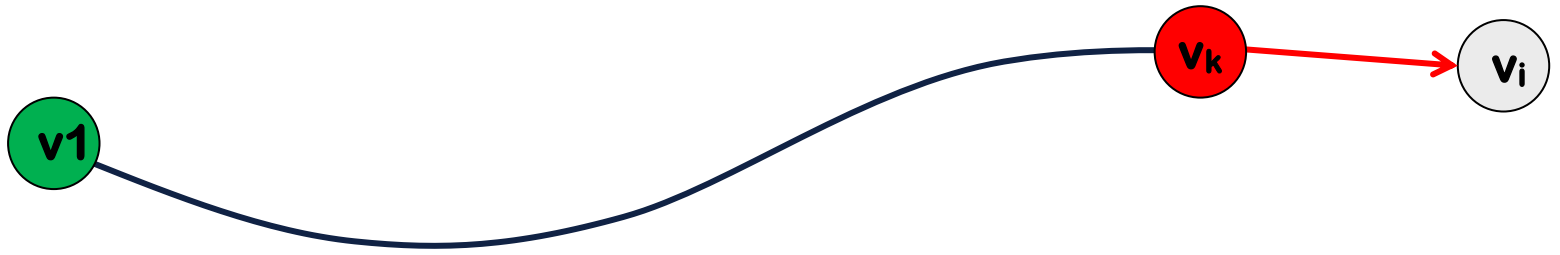
P[i]=k: $SP(v_1,v_i)=(1,\ldots,k,i)$



DIJKSTRA: PATH MATRIX

For i=2,3,..,n: P[i]=index of the vertex right before i on the shortest path from 1 to i:

P[i]=k: $SP(v_1,v_i)=(1,\ldots,k,i)$



DIJKSTRA: PATH MATRIX

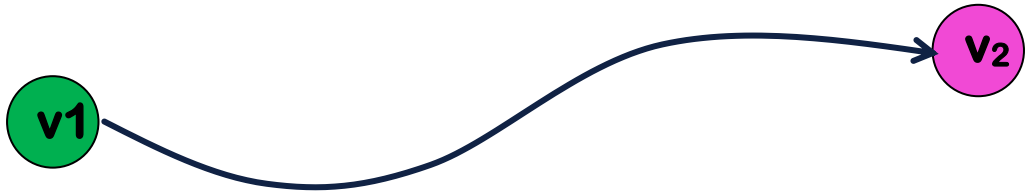
For i=2,3,..,n: P[i]=index of the vertex right before i on the shortest path from 1 to i:

$P[i]=k: \quad SP(v_1,v_i)=(1,\ldots,k,i)$

1. **How to Calculate Path matrix, P?**
2. **How to use P to find the best order?**

DIJKSTRA: PATH MATRIX

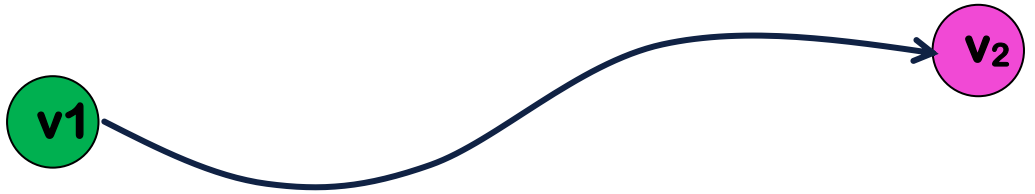
	v_2	v_3	v_4	v_5
P	3	1	5	1



SP (v_1 , v_2)=(1,…,2)

DIJKSTRA: PATH MATRIX

P	v_2	v_3	v_4	v_5
	3	1	5	1

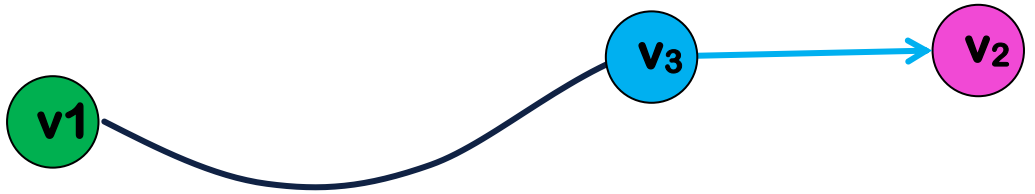


SP (v_1 , v_2)=(1,…,2)

DIJKSTRA: PATH MATRIX

P

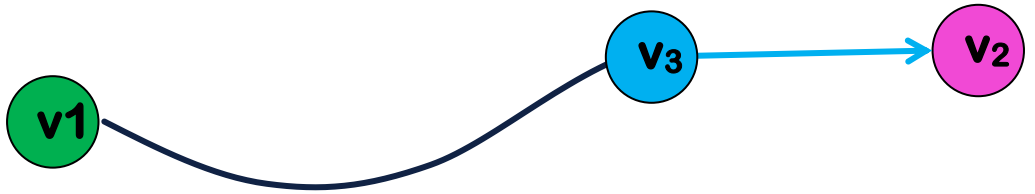
v_2	v_3	v_4	v_5
3	1	5	1



SP (v_1 , v_2)=(1,…,3,2)

DIJKSTRA: PATH MATRIX

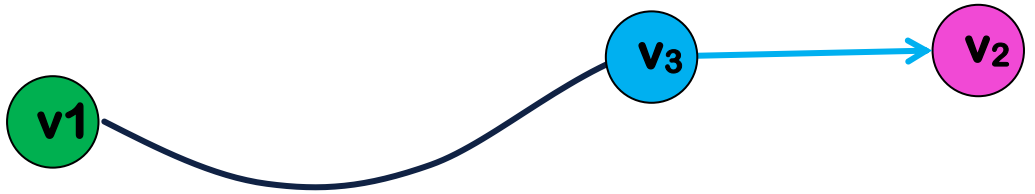
	v_2	v_3	v_4	v_5
P	3	1	5	1



SP (v_1 , v_2)=(1,…,3,2)

DIJKSTRA: PATH MATRIX

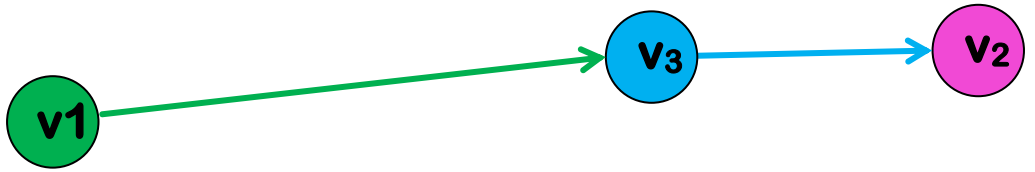
P	v_2	v_3	v_4	v_5
	3	1	5	1



SP (v_1 , v_2)=(1,…,3,2)

DIJKSTRA: PATH MATRIX

P	v_2	v_3	v_4	v_5
	3	1	5	1



SP (v_1 , v_2)=(1,3,2)

DIJKSTRA: PATH MATRIX

	v_2	v_3	v_4	v_5
P	3	1	5	1

$SP(v_1, v_2)=(1,\ldots,3,2)=(1,3,2)$

$SP(v_1, v_3)=(1,3)$

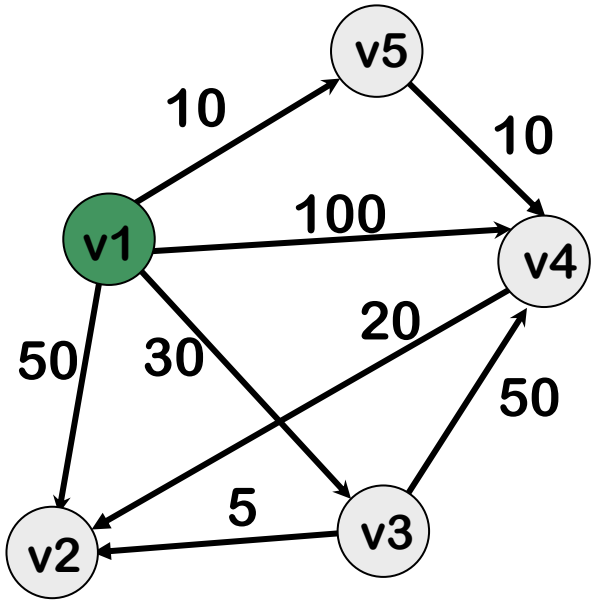
$SP(v_1, v_4)=(1,\ldots,5,4)=(1,5,4)$

$SP(v_1, v_5)=(1,5)$

DIJKSTRA: PATH MATRIX

For i=2,3,..,n: P[i]=index of the vertex right before i on the shortest path from 1 to i:

P[i]=k: $SP(v_1,v_i)=(1,\ldots,k,i)$



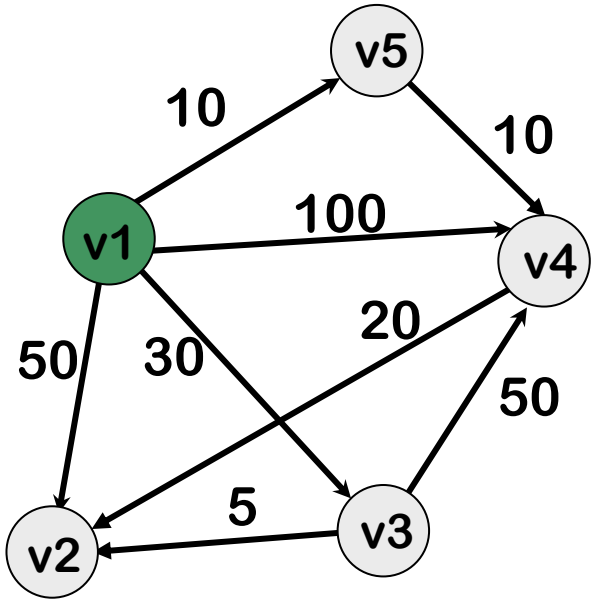
	1	2	3	4	5
P	-	1	1	1	1

	1	2	3	4	5
D	-	50	30	100	10

	1	2	3	4	5
P	-	3	1	5	1

	1	2	3	4	5
D	-	35	30	20	10

DIJKSTRA: IMPLEMENTATION



	v_2	v_3	v_4	v_5
D	50	30	100	10

	v_2	v_3	v_4	v_5
P	1	1	1	1

DIJKSTRA: PATH MATRIX

Old New

m=5

D[2]=min{D[2],D[5]+W[5][2]}=50

D[3]=min{D[3],D[5]+W[5][3]}=30

D[4]=min{D[4],D[5]+W[5][4]}=20

	v_2	v_3	v_4	v_5
D	50	30	100	10

	v_2	v_3	v_4	v_5
P	1	1	1	1

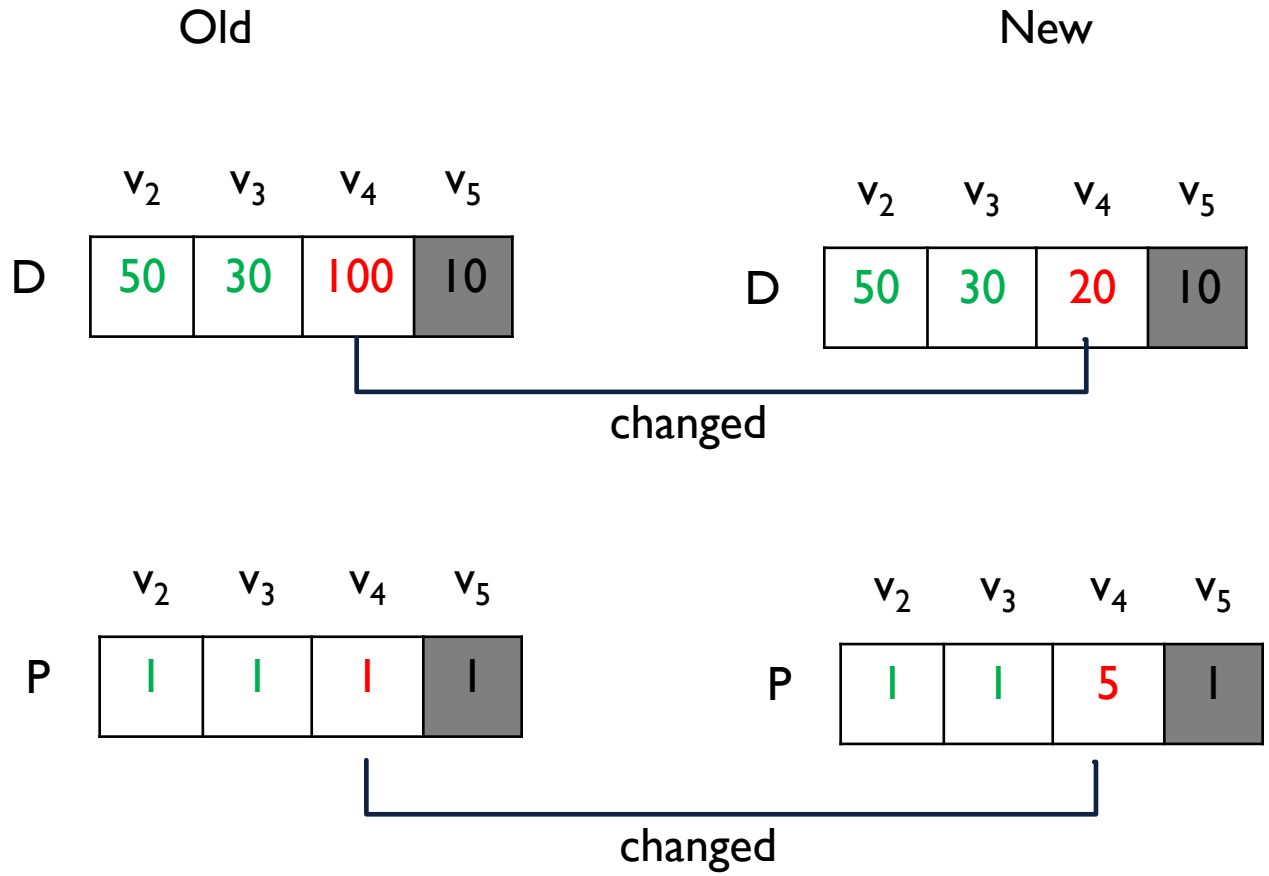
DIJKSTRA: PATH MATRIX

m=5

D[2]=min{D[2],D[5]+W[5][2]}=50

D[3]=min{D[3],D[5]+W[5][3]}=30

D[4]=min{D[4],D[5]+W[5][4]}=20



DIJKSTRA: PATH MATRIX

Old New

m=4

D[2]=min{D[2],D[4]+W[4][2]}=40

D[3]=min{D[3],D[4]+W[4][3]}=30

	v_2	v_3	v_4	v_5
D	50	30	20	10

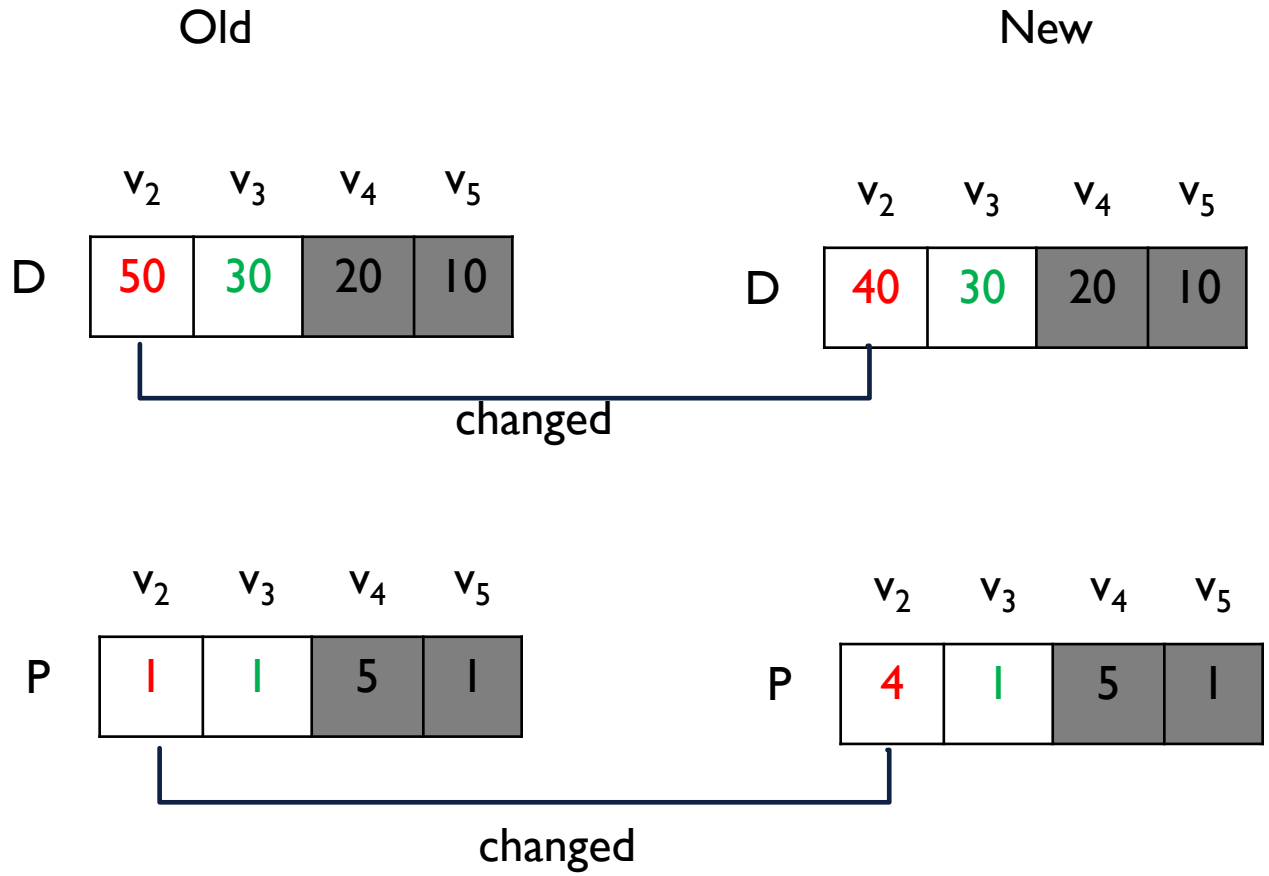
	v_2	v_3	v_4	v_5
P	1	1	5	1

DIJKSTRA: PATH MATRIX

m=4

D[2]=min{D[2],D[4]+W[4][2]}=40

D[3]=min{D[3],D[4]+W[4][3]}=30



DIJKSTRA: PATH MATRIX

Old New

m=3

	v_2	v_3	v_4	v_5
D	40	30	20	10

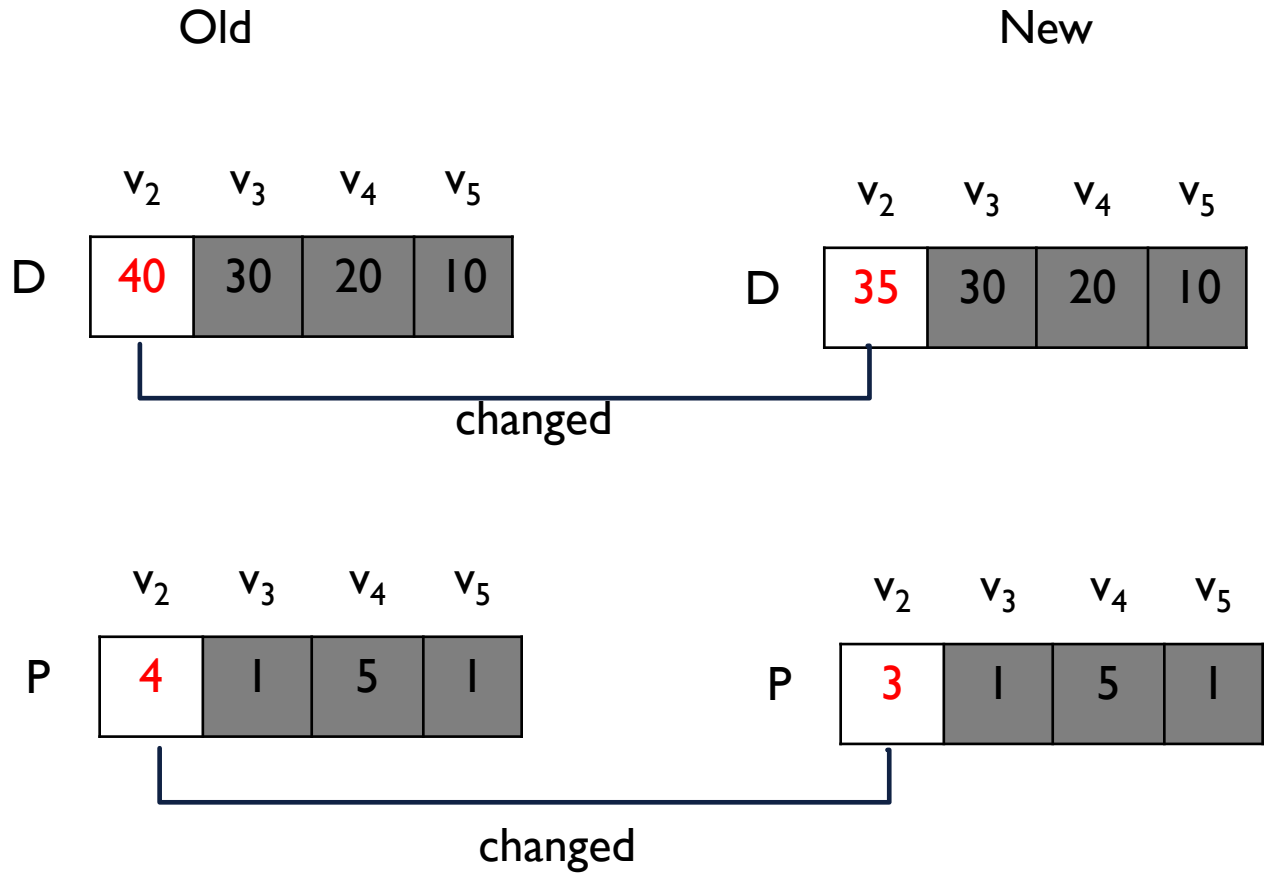
D[2]=min{D[2],D[3]+W[3][2]}=35

	v_2	v_3	v_4	v_5
P	4	1	5	1

DIJKSTRA: PATH MATRIX

m=3

D[2]=min{D[2],D[3]+W[3][2]}=35



DIJKSTRA'S ALGORITHM

```
Dijkstra(W)
      FOR i=2 TO n DO
         D[i]=W[1][i]
         P[i]=1
      FOR i=1 TO n-1 DO
        m=index of minimum in D
        FOR EACH j in D:
            IF D[m]+W[m][j]<D[j] THEN
                 D[j]=D[m]+W[m][j]
                 P[j]=m
        PRINT D[m]
        D.delete(m)
```

ANY QUESTION??

HAVE A NICE DAY!